Combining Self-Motivation with Planning and Inference in a Self-Motivated Cognitive Agent Framework

by

Daphne H. Liu

Submitted in Partial Fulfillment

of the

Requirements for the Degree

Doctor of Philosophy

Supervised by

Professor Lenhart Schubert

Department of Computer Science
Arts, Science & Engineering
Edmund A. Hajim School of Engineering & Applied Sciences

University of Rochester
Rochester, New York

2012

To my family

Biographical Sketch

Ms. Liu attended Simon Fraser University from 2000 to 2004, graduating with a Bachelor of Science degree in Honors Computing Science, with a rank of 2nd of 130 in her class, and a minor in Business Administration. During her undergraduate years, she completed an 8-month full-time internship as a software engineer, and later worked as a research assistant in artificial intelligence with Prof. James Delgrande. For her academic and research performance, she was awarded the Undergraduate Research Award by the Natural Sciences and Engineering Research Council of Canada (NSERC) as well as an Outstanding Undergraduate Award Honorable Mention by the Computing Research Association (CRA). Also a recipient of the Master's Level Canada Graduate Scholarship by NSERC, Ms. Liu went on to complete her M.Sc. thesis under the guidance of Prof. Delgrande, and earned her first Master of Science degree in Computing Science from Simon Fraser University in 2006. In the fall of 2006, she commenced her Ph.D. studies in Computer Science at the University of Rochester, where she was awarded the Robert L. Sproull Fellowship for the first two years. She pursued her research in artificial intelligence under the direction of Prof. Lenhart Schubert. In 2008, she obtained her second Master of Science degree in Computer Science from the University of Rochester.

List of Publications and Articles Submitted for Publication

- Daphne H. Liu and Lenhart Schubert. An Infrastructure for Self-Motivated, Continually Planning Agents in Virtual Worlds. *Technical Report 2012-985*, Dept. of Computer Science, University of Rochester, December 2012.

- Daphne Liu and Lenhart Schubert. Towards Self-Motivated, Cognitive, Continually Planning Agents. Manuscript submitted in February 2012 for publication (under review at *Computational Intelligence*).

- Daphne Liu and L. K. Schubert. Combining Self-Motivation with Logical Planning and Inference in a Reward Seeking Agent. In *Proceedings of the International Conference on Agents and Artificial Intelligence (ICAART 2010), vol. 2*, Valencia, Spain, January 2010.

- Daphne Liu and Lenhart Schubert. Incorporating Planning and Reasoning into a Self-Motivated, Communicative Agent. In *Proceedings of the Second Conference on Artificial General Intelligence (AGI 2009)*, Arlington, VA, March 2009.

- Daphne Hao Liu. A Survey of Planning in Intelligent Agents: from Externally Motivated to Internally Motivated Systems. *Technical Report 2008-936*, Dept. of Computer Science, University of Rochester, June 2008.

- Daphne Hao Liu. A Consistency-Based System for Knowledge Base Merging. *Master of Science thesis*, School of Computing Science, Simon Fraser University, August 2006.

- J. Delgrande, D. H. Liu, T. Schaub and S. Thiele. COBA 2.0: A Consistency-Based Belief Change System. In *Proceedings of the Eleventh International Workshop on Non-Monotonic Reasoning (NMR 2006)*, June 2006.

- James Delgrande, Daphne H. Liu, Torsten Schaub and Sven Thiele. COBA 2.0: A Consistency-Based Belief Change System. *Technical Report TR 2005-17*, School of Computing Science, Simon Fraser University, November 2005.

Selected Honors and Awards

- Robert L. Sproull Fellowship, University of Rochester (2006 - 2008)
- Natural Sciences and Engineering Research Council of Canada (NSERC) Canada Graduate Scholarship - Masters (2005 - 2006)
- Computing Research Association (CRA) Outstanding Undergraduate Award Honorable Mention (2005)
- Natural Sciences and Engineering Research Council of Canada Undergraduate Research Award (2004)
- The Governor General's Academic Bronze Medal of Canada (2000)

Acknowledgments

My sincere gratitude goes to my advisor, Prof. Lenhart Schubert, for his invaluable insights, guidance and feedback on my work. Len is not only a meticulous and extraordinarily knowledgeable advisor, but also a kind and encouraging person. I have learned and benefitted considerably from many memorable discussions with him over the years. Even under the pressure of impending deadlines and course-teaching obligations, Len is very generous with his time and attention when it comes to advising students. Len's passion for research is inspiring, and his impeccable attitude towards research is worth emulating for one to work hard and remain dedicated to one's craft.

I am also indebted to my thesis committee as a whole: Prof. Lenhart Schubert, Prof. James Allen, Prof. Greg Carlson, and Prof. Daniel Gildea. Thank you all for your suggestions, feedback, and guidance throughout my PhD studies.

I appreciate the opportunity to have worked as an intern with Dr. Eric Bier of the Palo Alto Research Center (PARC). In the summer of 2011, I worked in the Knowledge, Language and Interaction group at PARC, on simulating human work practices and identifying tasks that could be automated or streamlined with tools. Eric's wonderful mentorship, as well as the privilege to collaborate with a capable and cordial group of coworkers, made for an rewarding and unforgettable internship experience.

The prestigious Sproull Fellowship awarded by the University of Rochester, together with my advisor's National Science Foundation grants IIS-0535105, IIS-

0916599, and IIS-1016735 and the Department of Defense/ONR grant N00014-11-10417, have graciously supported me financially during my PhD studies. I also acknowledge the entire URCS faculty and staff for their teaching and assistance, respectively, over the years.

On a more personal level, I am thankful to the following people for their friendship over the years in Rochester: Jerry Charipar, Mary Charipar, Xiao Zhang, Xiaohua Zhang, Huiling Zhang, Yang Gao, Tongxin Bai, Xiaoming Gu, Zhuan Chen, and Li Lu.

To my parents, brother, auntie Joyce and grandma, thank you for love and encouragement. Your contribution to this dissertation cannot be quantified, but it is forever etched in my heart and my memory.

Lastly, to my husband, Qi Ge, thank you for your patience, understanding and support. As you and I both know, obtaining one's PhD degree is an arduous journey, but having you alongside me through this journey has certainly made it immeasurably more enjoyable. I am blessed to have you in my life, and I am tremendously grateful for the loving home we have built together.

Abstract

Most work on self-motivated agents in Artificial Intelligence has focused on acquiring utility-optimizing mappings from states to actions. However, such mappings do not allow for explicit, reasoned anticipation and planned achievement of future states and pay-offs, based on symbolic knowledge about the environment and about the consequences of the agent's own behavior. In essence, such agents can behave only reflexively, not reflectively. Conversely, planning and reasoning have been viewed as aimed at fulfillment of explicitly specified user goals, without regard for the long-range utility of the planner or reasoner's choices. Moreover, most work assumes either a completely known environment or a predictable environment altered only by the agent's actions.

We strive to integrate self-motivation into logical planning and reasoning, in a general self-motivated cognitive agent framework. Our self-motivated cognitive agents are capable of thinking ahead and planning flexibly in an incompletely known world, question-answering dialogues, mental modeling, introspection and reasoning over a substantial knowledge base, all while in unceasing pursuit of optimizing their own cumulative utility. While such agents employ reasoned exploration of feasible sequences of actions and corresponding states, they also behave opportunistically, seizing opportunities and recovering from failures, thanks to their continual plan updates and quest for rewards. Our framework allows for both specific and general (quantified) knowledge, and for epistemic predicates such as *knowing−that* and *knowing−whether*. Since realistic agents often possess in-

complete knowledge of their world, the reasoning of our agents uses a prudently restricted version of the closed world assumption; this has consequences for epistemic reasoning, in particular positive and negative introspection. The planning operators allow for quantitative, gradual change and side effects such as the passage of time, changes in distances and rewards, and language production, using a uniform procedural attachment technique. Question-answering (involving introspection and reasoning) and experimental runs are shown for our particular agent ME in a simulated world, demonstrating the benefits of self-awareness as well as continual, deliberate and reward-driven planning. Our agents can also be easily configured as single or multiple goal-seeking agents, and as such perform quite favorably compared with recent experimental agents.

Contributors and Funding Sources

The PhD research culminating in this dissertation was supervised by the author's dissertation committee: Prof. Lenhart Schubert of the Department of Computer Science, Prof. James Allen of the Department of Computer Science, Prof. Greg Carlson of the Department of Linguistics, and Prof. Daniel Gildea of the Department of Computer Science, all at the University of Rochester.

Chapter 2 of this dissertation is organized around several systems the author originally surveyed in (Liu, 2008). Chapters 3 through 5 describe work the author collaboratively accomplished with her advisor, Prof. Lenhart Schubert. Parts of Chapter 3 elaborate on some material originally published in (Liu and Schubert, 2010) and (Liu and Schubert, 2009). The contents of Chapter 4 have not yet been published. Chapters 5 and 6 and Section 7.1, their contents partially done jointly with Prof. Schubert and also appearing in the Computer Science Department *Technical Report 2012-985*, have been submitted in a journal manuscript that is currently under review.

This material is based upon work supported by the Sproull Fellowship of the University of Rochester, the National Science Foundation grant IIS-0535105, National Science Foundation grant IIS-0916599, National Foundation grant IIS-1016735 and the Department of Defense/ONR grant N00014-11-10417. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author and do not necessarily reflect the views of above named organizations.

Table of Contents

Biographical Sketch **iii**

Acknowledgments **vi**

Abstract **viii**

Contributors and Funding Sources **x**

List of Tables **xiv**

List of Figures **xv**

List of Algorithms **xvi**

1 Introduction **1**

2 Internally Motivated Systems **7**

2.1 Views of Self-Awareness 8

2.2 Introspection 15

2.3 BDI Architecture 17

2.4 Homer 24

3 Integrating Self-Motivation into Logical Planning and Inference 30

3.1 Self-Motivated Cognitive Agent Framework 31

3.2 Overview of Framework 32

3.3 Actions, Planning, and Simulation 35

3.4 Reasoning about World States and Mental States 40

3.5 Deliberate Self-Motivation and Self-Awareness 46

4 Designing and Interacting with a Simulated World 49

4.1 Syntax Preliminaries 50

4.2 Defining and Creating a Roadmap 52

4.3 Defining and Creating Types and Entities 53

4.4 Defining an Action Operator 55

4.5 Interacting with ME 58

5 Experimental Agent and Results 60

5.1 Simulated World 61

5.2 Question-Answering 64

5.3 Results 69

6 Related Work 76

6.1 Classical Planning and Our Results 77

6.2 Continuous Planning and Our Results 81

6.3 Cognitive Robots 98

6.4 Behavioral Agents 102

6.5 Agents That Plan and Communicate Verbally 104

6.6 Some General Architectures for Intelligent Autonomous Systems . 109

7 Conclusion **113**

7.1 Future Work 116

Bibliography **120**

List of Tables

6.1 Working with or without Full Contingent Plans 84

6.2 CLG in Execution Mode on Colorballs-9-i 86

6.3 Our Framework on Colorballs-9-i 87

6.4 Our Framework on Multiagent-4-x-b 89

6.5 Our Framework on Multiagent-5-x-b 90

6.6 Our Framework on Multiagent-6-x-b 91

6.7 Our Framework on Multiagent-7-x-b 92

6.8 Our Framework on Multiagent-8-x-b 93

6.9 Our Framework on Multiagent-9-x-b 95

6.10 Our Framework on Multiagent-10-x-b 97

List of Figures

2.1 A Taxonomy of Knowledge 12
2.2 The System Structure of PRS 22
2.3 The System Block Diagram of Homer 26
2.4 One Sense of Action "give" 27

3.1 The *sleep* Model Operator 36
3.2 The *sleep* Actual Operator 38
3.3 The *answer_user_ynq* Model Operator 40

5.1 The Experimental World 62
5.2 Handling *tell* Tidbits 65
5.3 Answering *yes/no* Questions 66
5.4 Answering *wh* Questions 67
5.5 Answering Questions Using Introspection 68
5.6 Example of an Opportunistic Run 71
5.7 Example of a Goal-Directed Run 75

List of Algorithms

1 Rao and Georgeff's BDI Interpreter 19
2 Evaluating Non-epistemic Atoms Using a Restricted CWA 43
3 Evaluating Epistemic Atoms Using Introspection 45

1 Introduction

General human-like intelligence entails planning and reasoning; essentially all human activities, but for lower level ones like perception and reflexes, are guided by planning and reasoning. The ability to plan and reason is a requisite for any agent, artificial or not, wishing to claim intelligence in thought and behavior. Given a goal, a planning agent typically persists in pursuing the goal to attainment as long as the situation justifies the agent's perseverance. Additionally, an *intelligent* planning agent must be equally adept at responding to failures, opportunities, and threats in the environment.

Traditionally, planning and reasoning have been viewed within Artificial Intelligence (AI) as geared towards satisfaction of user-specified goals, without consideration of the long-range utility of the planner or reasoner's choices. We consider such systems externally motivated. Deprived of self-awareness and such mental attitudes as beliefs, desires, and intentions, these systems have been constrained to act and react in accord with user-specified goals, constraints, and preferences, but with total disregard for what they themselves stand to gain in cumulative utility from the fulfillment of user-given goals.

On the other hand, most work in self-motivated agents to date has focused on acquiring utility-optimizing mappings from states to actions. However, neither the

learning of these mappings from past experiences, nor the mappings themselves, allow for reasoned anticipation and planned achievement of future states and rewards, based on symbolic knowledge about the environment and about the consequences of the agent's own behavior. Furthermore, in such systems, states are not represented by symbolic descriptions; therefore, such states cannot be reasoned about, nor can the beliefs, desires, and intentions of such agents be fully realized. In essence, such agents can behave only reflexively, rather than reflectively.

We are ultimately interested in creating linguistically competent, intelligent, human-like agents. Such agents will be able to converse, plan flexibly in an incompletely known world, and make inferences over a substantial knowledge base, all while striving to optimize their own cumulative utility. Although many AI systems, as will be described in Chapter 6, have shown *some* of these abilities, none, to our knowledge, have integrated them all. In this dissertation, we present a general framework for creating *self-motivated cognitive agents* and exemplify our framework with a protagonist agent dubbed ME, pronounced "em-ee" for Motivated Explorer. ME can plan and think ahead where actions can take up time and produce gradual change, cope with unforeseen environmental events, make inferences about states including its own and other agents' knowledge and wants, engage in limited question-answering with the user and other agents, and do so indefinitely and autonomously, driven by the expected cumulative rewards or costs of its contemplated sequences of actions and anticipated future states.

It may seem that the type of self-motivation and self-awareness espoused in our framework would lead to single-minded, self-indulgent behavior, but this is not at all so. If the agent enjoys acquiring knowledge, exploring its environment, conversing, and satisfying perceived user goals, then it can appear to be inquisitive, interactive, and helpful. It may also seem that such an agent could do no more than typical reinforcement learning agents. In fact, however, the kind of agent

envisaged and illustrated here conceives of its world and its actions in logical, descriptive terms. It is a *cognitive* agent, able to entertain beliefs about itself and other agents, to engage in simple question-answering dialogues, and to plan ahead sufficiently well not only to thrive in its world, but also (with some minor tweaks) to solve classical STRIPS-like planning problems.

As a preview, we first succinctly enumerate our intended research contributions in this dissertation:

1. We seek to bridge the dichotomy between the planning and reasoning agent paradigm and the self-motivated agent paradigm.

2. We aim to demonstrate the feasibility of combining planning, inference and dialogue in a completely self-motivated cognitive agent.

3. We offer a general, flexible, and easy-to-use self-motivated cognitive agent framework.

We will substantiate these contributions in the ensuing chapters. Finally in the Conclusion chapter, we will elaborate on how we have accomplished each of these contributions.

We provide an outline of the remaining chapters in this dissertation.

- Chapter 2, as related work and background, explores various views of self-awareness by Schubert (2005), Minsky (1968), and McCarthy (1968). Notably, Schubert (2005) defined and differentiated *explicit self-awareness* from other weaker notions of self-awareness including self-monitoring. Additionally, we consider introspection in planning agents. We also investigate the integral role such mental attitudes as beliefs, desires, and intentions play in the design of intelligent agents, by studying the belief-desire-intention (BDI) architecture as realized in PRS (Georgeff and Lansky, 1987). We then discuss Homer (Vere and Bickmore, 1990), an exemplar of a conscious, albeit

still externally motivated, agent exhibiting both perceptual awareness and self-awareness.

- Chapter 3 elucidates what we mean by a self-motivated cognitive agent and our development framework for creating such agents. We provide details of how actions are defined for an agent, the planning process, and the syntax and semantics of action definitions when we are specifying actual actions as carried out when running the simulated world. We then turn our attention to a prudently restricted version of the closed world assumption used by the agent to perform reasoning including introspection. We close this chapter with an analysis of how the agent displays self-motivation and self-awareness.

- Chapter 4 lends credence to our framework's ease of use. We specify precisely the steps required in designing one's own self-motivated cognitive agent and simulated world from scratch. These steps encompass defining and creating a roadmap, entity types and properties, agent instances, object instances, and the self-motivated cognitive agent's action repertoire, as well as setting such parameters as the forward inference bound, search beam, manually added general-knowledge inference rules, and the reward and cost metrics for actions and state properties. We also explicate how a human user can interact with the self-motivated cognitive agent.

- In Chapter 5, we instantiate our agent ME in a particular simulated world, for the purpose of demonstrating and experimenting with our framework. The sample world is a simple one, but suffices for illustrating and supporting the points we wish to make. We then provide a range of examples of question-answering by ME (including questions involving attitudes) to demonstrate its self-awareness and epistemic knowledge, before reporting the results of various runs of the agent in the simulated world. One set of runs is aimed at demonstrating the default, opportunistic yet future-

directed behavior of the agent when all its faculties are intact. In another set of runs, the agent is deprived of its ability to plan and reason ahead (or of its self-awareness, from another perspective), and this leads to the expected deterioration in its behavior. In a third experiment, the agent is rendered doggedly goal-directed towards one specific consumption goal by upward adjustment of the rewards it subjectively assigns to the chosen goal and to types of actions and states that lead to it, while zeroing out most other reward estimates. This shows that such goal-directedness and ME's resulting oblivion and failure to exploit opportunities will greatly reduce its total utility.

- Chapter 6 compares and contrasts our framework with related work in classical planning, continuous planning, behavioral and game agents, agents that reason and converse, and other general architectural proposals for autonomy and self-awareness. The capabilities of our agents evidently intersect those of these agents. Not only does our framework combine self-motivation with logical planning and reasoning, but we also view it as an integration of situated, reactive behavior, basic dialogue capabilities, continuous planning for utility optimization, with classical planning (including first-order reasoning, extended to allow reasoning about attitudes). Notably in Sections 6.1 and 6.2, we discuss how we successfully constructed and solved such classical STRIPS-like planning problems as the Towers of Hanoi and a Logistics domain, and more recent continuous planning problems such as a Colorballs domain and a Multiagent traveling domain in incompletely known worlds. In the Colorballs domain, our framework achieved considerably better performance than some state-of-the-art contingent planners that construct full contingent plans, even finishing problems that they failed to solve. In presenting our Multiagent traveling domain experiments, we also describe the multiagent extensions we have made to our framework to enable multiple,

coexisting self-motivated cognitive agents.

- Chapter 7 recapitulates the motivation for our work and elaborates on our research contributions. Lastly, we conclude with a discussion of possible future directions for this framework.

2 Internally Motivated Systems

We devote this chapter, as related work and background, to a discussion of *internally motivated* systems, broadly including planning agents. Specifically, we focus on several systems endued with self-awareness and with such mental attitudes as beliefs, desires, and intentions.

We review several views of self-awareness in machines, due to Schubert (2005), Minsky (1968), and McCarthy (1968), in Section 2.1. An investigation of how McCarthy's *mental situation calculus* (McCarthy, 1995) and Scherl, Levesque, and Lespérance's situation calculus with sensing and indexical knowledge (Scherl et al., 1995) support a machine's introspection is included in Section 2.1.3. This is followed by Section 2.2, which explores introspection in machines. In Section 2.3, we examine Rao and Georgeff's framework (Rao and Georgeff, 1991, 1995) founded on beliefs, desires, and intentions (BDI), before looking at the BDI architecture as realized in Georgeff and Lansky's PRS system (Georgeff and Lansky, 1987). Lastly in Section 2.4, we conclude this chapter with Homer (Vere and Bickmore, 1990), an exemplar of a conscious agent exhibiting both perceptual awareness and self-awareness.

2.1 Views of Self-Awareness

Self-awareness, with its multi-disciplinary connections, has been the focus of much research effort in various communities ranging from philosophers, cognitive scientists, to computer scientists. As a consequence of the fertile research, widely varied definitions of self-awareness have been proposed.

In the ensuing three subsections, we review three perspectives of self-awareness, due to Schubert (2005), Minsky (1968), and McCarthy (1968). Notably, Schubert's notion of *explicit self-awareness* is compared and contrasted with other weaker notions of self-awareness commonly found in the research literature. Minsky's proposed taxonomy of knowledge, in conjunction with self-awareness, is discussed. As well, logically-oriented perspectives of self-awareness are presented, both in the form of McCarthy's *mental situation calculus* (McCarthy, 1995) and in the form of Scherl, Levesque, and Lespérance's situation calculus with sensing and indexical knowledge (Scherl et al., 1995).

2.1.1 Schubert's Perspective

In (Schubert, 2005), Schubert reviewed several competing notions of self-awareness and defined the notion of *explicit self-awareness*. Schubert characterized *explicit self-awareness* as being both human-like and explicit. Specifically, it is human-like in that an explicitly self-aware agent must have a well-elaborated human-like model of the world, including a model of itself and its relationships to the world. The self-model encompasses its beliefs, desires, intentions, knowledge, abilities, autobiography, the current situation, etc.; furthermore, the self-model must be amenable to inferences in conjunction with world knowledge, and the system must be capable of goal-directed and utility-directed planning.

In addition to prescribing the aforementioned human-like capabilities, *explicit self-awareness* is explicit in three respects (Schubert, 2005). First, an agent's

representation of self-knowledge must be amenable to self-observation and self-interpretation by the agent. Second, *explicit self-awareness* must be conveyable by the agent, through language or other modalities. Third, such self-awareness is pervious to the same inferential processes as is all other knowledge.

Schubert further enumerated reasons motivating the need for explicit self-awareness. First, given its bootstrapping potential with respect to meta-control, error recovery, autoepistemic reasoning, and learning of skills or facts, explicit self-awareness would help expand the frontiers of Artificial Intelligence. Additionally, an explicitly self-aware agent can interact with human users in a transparent, natural, and engaging manner by having a shared context. Lastly, operational explicitly self-aware agents can serve as exemplars of entities whose internal basis for self-awareness can be analyzed by consciousness theorists wishing to better understand self-awareness.

Schubert also acknowledged several related, albeit weaker, notions of self-awareness. First, some cognitive scientists, including Anderson and Perlis (2005), equate self-monitoring with self-awareness. Even though *self-monitoring* agents monitor, evaluate, and adapt their internal processes purposefully (e.g., for performance optimization or error recovery), they need not have a self-model or general reasoning mechanisms. By way of a simple example, an automatic temperature control system can be considered a self-monitoring agent which takes a room temperature as input, compares the input to a certain threshold, and either turns on the heat if the input drops below the threshold, turns off the heat if it exceeds the threshold, or does nothing otherwise. Another example is Anderson and Perlis's metacognitive loop (Anderson and Perlis, 2005).

Another weaker form of self-awareness is captured by *self-explaining* agents. Schubert (2005) suggested that these agents are able to both recount and justify their actions, and substantiate their inferences; however, they fail to integrate their self-model with general reasoning mechanisms. A prime example is Winograd's

SHRDLU (Winograd, 1971). SHRDLU examines its stack of actions, along with their corresponding preconditions and effects, to extract explanations for its actions. Nonetheless, SHRDLU does not genuinely understand its explanations for actions, as it would be stumped by a question probing whether the effects of certain actions could have been achieved by some other actions. Yet another example is Cox's Meta-AQUA system (Cox, 2007) for learning from failures in story comprehension.

A third, weaker from of self-awareness is embodied in the *global workspace systems*. Such systems have a global blackboard which numerous processes share to interact and communicate with one another. The Opportunistic Planning Model invented by Hayes-Roth and Hayes-Roth (1979) exemplifies such a system. Schubert (2005) argued that although such a structural feature seems to be reflected in Baars's proposed architecture for human consciousness (Baars, 1988), such a feature does not entail reflective thought or any inferential mechanisms.

The last weaker form of self-awareness surveyed by Schubert (2005) is exhibited by *adaptive, robust, goal-directed* systems. As with the other weaker forms of self-awareness, such a system does not imply a self-model or inferential mechanisms, and thus, either lacks a self-model, or fails to integrate its self-model into general reasoning. For instance, although an antibody system is highly adaptive, robust, and goal-directed in combatting viral intrusions, one would hardly deem an antibody system self-aware in any sense of the word.

Having compared and contrasted these various notions of self-awareness, Schubert then proposed augmentations to McCarthy's logically-oriented enterprise (McCarthy, 1995) of enabling an agent to reason about its own internal states. Also detailed in (Schubert, 2005) are specific knowledge representation and reasoning requirements for explicit self-awareness. In addition to a basic logical framework, these requirements include logical formulations of events, situations, attitudes, autoepistemic inferences, generic knowledge, and various meta-syntactic devices such as axiom schemas, knowledge categorization, knowing or deriving a

value, and experience summarization. Readers interested in the details are referred to (Schubert, 2005); as well, McCarthy's proposal (McCarthy, 1995) is reviewed in our Section 2.1.3.

2.1.2 Minsky's Perspective

Minsky (1968) contended that in order for a machine to answer questions about the world or about itself in the world, it must be able to simulate the world and itself procedurally. The pivotal thrust of Minsky's proposal is that if an agent can answer questions regarding a hypothetical experiment without actually executing it, then the agent demonstrates its knowledge of the world; moreover, if this hypothetical experiment involves the agent itself, then the agent also demonstrates its knowledge of itself. Central to this proposal is a computational model of the world which the machine possesses and can simulate to answer questions about actions and events in the world, regardless of whether these actions and events actually occur in the world. Additionally, to be able to answer questions about itself in the world, the machine must also have a model of itself in the world model.

Figure 2.1, recreated from (Cox, 2005), illustrates the taxonomy of knowledge proposed by Minsky (1968). Schematically, this anticipates the possession of a self-model entailed by Schubert's explicit self-awareness (Schubert, 2005). The machine M exists as the modeler in the world W. M possesses a model of the world $W*$, which, in turn, encompasses a model of the machine $M*$. Whereas M simulates $W*$ to understand and answer questions about actions and events in the world, M simulates $M*$ to answer questions about itself. Accordingly, Cox (2005) conceived of $W*$ and $M*$ as the machine's knowledge of the world and its reflective knowledge of itself, respectively. Moreover, embedded in this knowledge taxonomy are $W**$ and $M**$, which Cox (2005) coined *meta-knowledge* and *introspective knowledge*, respectively. The machine requires the former in order to answer

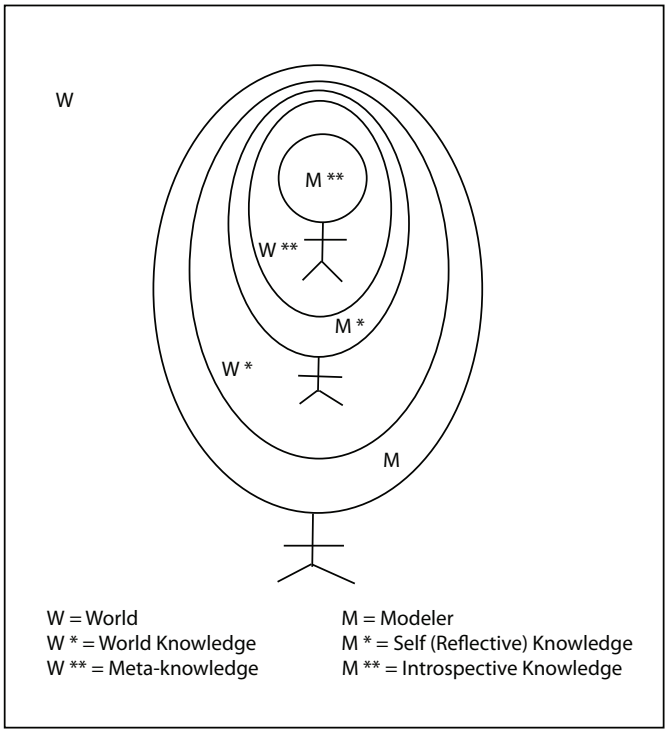


Figure 2.1: A Taxonomy of Knowledge

questions about and reason about its model of the world $W*$, as well as the latter in order to answer questions about and reason about its self-knowledge $M*$.

2.1.3 McCarthy's Perspective

In (McCarthy, 1968), McCarthy postulated that a machine must declaratively represent its mental states in order to reason about them. This postulation figures prominently in McCarthy's proposal to imbue machines with self-awareness, in the ultimate objective to attain human level intelligence in machines. Also entailed in this proposal is the assignment, to machines, of mental qualities, such as beliefs, wants, intentions, knowledge, abilities, free will, and consciousness, in the *mental situation calculus* (McCarthy, 1995). This is the situation calculus with the amendment that many fluents now are knowledge properties, and that various intensional notions are introduced. A proposition or individual concept

is distinguished as a first-class object, from the truth value of the proposition or the value of the concept. The former is denoted by a word beginning with an uppercase letter; the latter, by a word in all lowercase letters.

As an example, consider the following sentences from (McCarthy, 1979) expressing that although Mike and Mary have the same telephone number, Pat knows Mike's telephone number but not Mary's:

$$\left\{ \begin{array}{l} denotation(Telephone(Person)) = telephone(denotation(Person)), \\ denotation(Mike) = mike,\ telephone(mike) = telephone(mary), \\ knows(pat, Telephone(Mike)),\ \neg knows(pat, Telephone(Mary)) \end{array} \right\}$$

Knowing creates an intensional context such that what is in the context cannot be freely substituted for based merely on the equality of actual objects.

Furthermore, McCarthy introduced context as a first-class object, and as such, variables, as well as arguments and results of functions, can assume contexts as their values. $Ist(c, p)$, or $c : p$, asserts that some proposition p is true in some context c, where p is reified as a term in first-order logic. The uses McCarthy seemed to intend for contexts are either (a) to provide a "here, now, I" context for all propositions that currently hold for the machine, or (b) to assist in hypothetical reasoning.

This formalism enables a machine to observe its own body, its relative position to the environment and its internal variables. As an instance of (a), as given by $C(Here, Now, I) : Lowbattery \wedge In(Screwdriver, Hand3)$ from (McCarthy, 1995), the machine is aware of its low battery as well as its holding a screwdriver in hand. Additionally, the machine can observe whether it does or does not know the value of a certain term. For example, $C(Now, I) : \neg Know\text{-}whether(Sitting(Clinton))$ from (McCarthy, 1995) allows the machine to assert that it does not know whether Clinton is currently sitting. Deciding that it does not know and cannot infer whether Clinton is currently sitting is the motivator that could impel the machine to take actions to find out the answer.

The mental situation calculus envisions a system in which *knowledge* is not closed under deductive inference, whereas $Holds$ is. Specifically, $Holds(p, s)$ asserts that some term p representing a proposition holds in some situation s. $Know(q)$ and $Believe(q)$ are among the reified propositions taking some term q. For instance, $Holds(Know(p), s)$ expresses that the machine knows p in situation s, while $Holds(Know(NotKnow(p), s)$ states that the machine knows that it does not know p. Assertions of the latter kind are useful in prompting the machine to take actions to acquire knowledge that it currently lacks.

Several kinds of mental events can be represented in the mental situation calculus, including the sequential occurrence of mental events, learning, forgetting, assuming, inferring, and seeing. Let's review an example from (McCarthy, 1995), making use of $F(p)$, $Occurs(e, s)$, and $Next(p, s)$, where s is a situation, e is an event, and p is a reified proposition. Particularly, $F(p)$ is a reified proposition that p will hold at some time in the future; $Occurs(e, s)$ is a point fluent asserting that e occurs in s; $Next(p, s)$ denotes the next situation following s in which p holds. For example, $Occurs(Learn(p), s) \rightarrow Holds(F(Know(p)), s)$, together with the axiom $Holds(F(p), s) \rightarrow Holds(p, Next(p, s))$, would express that if the machine learns p in situation s, then the machine will know p in the next situation following s in which p holds. The imprecision in how many time steps after s this next state is, nevertheless, unresolved by this formulation.

A similar, logically-oriented endeavor to endow machines with self-awareness was realized by the cognitive robotics project at the University of Toronto. Scherl et al. (1995) proposed extending the situation calculus with sensing and indexical knowledge. In this framework, both the knowledge prerequisites and effects of actions can be specified in terms of indexical rather than objective knowledge. This representation was motivated by several observations (Lespérance and Levesque, 1995) about the unsuitability of objective knowledge alone for the formalization of actions. Specifically, objective knowledge is neither necessary nor sufficient for

actions, nor is it produced by perceptual actions. For example, a robot may not know an object's absolute location, but knows the object's relative location to itself; conversely, a robot may not know its current location, but knows an object's absolute location. As well, Scherl et al. (1995) claimed that a robot's perceptual actions, at best, inform the robot of an object's relative location to the robot.

Both the mental situation calculus (McCarthy, 1995) and the situation calculus augmented with sensing and indexical knowledge (Scherl et al., 1995) readily lend themselves to a machine's introspection, which McCarthy claimed to be an essential requirement for a machine to attain human level intelligence. Section 2.2 discusses introspection, illuminating the connection with the mental situation calculus by an example.

2.2 Introspection

Cox (2007) suggested that an intelligent agent confronted with a choice in what to do in the world must make three kinds of decisions. First, it must decide which of several available actions is the most suitable in the current situation. Second, it must decide whether its decision in the first step is sufficiently justified to warrant its commitment; i.e., it must cogitate about its cognition. Third, in further evaluating its decisions in the first two steps, it must reflect on its similar, past experiences and weigh reasons for and against its decisions. All three decision processes involve *introspection*, or *metacognition*.

Anderson and Oates (2007) gave a fictitious story featuring Rosie the robot maid from the TV show *The Jetsons*, in which Rosie uses introspection to monitor, model and control her cognition. In one episode, Rosie forgets to buy food for the family pet dog and decides to feed the dog human food instead. However, realizing that human food is relatively more expensive and that she is forgetful, Rosie sticks a spare dog collar on her shopping list to serve as a reminder next time

she does groceries. In another episode, Rosie consults her to-do list prioritized by task importance, and decides to allocate the computationally intensive but relatively less important task of making arrangements for the Jetsons' vacation to the evening, when her workload is less. The episodes reveal that Rosie employs introspection to know her capabilities and deficiencies, take steps to address the latter, and make the best use of her limited resources.

Cox (2007) asserted that being self-aware is not simply perceiving the world, nor is it merely a logical interpretation mapping symbols to relations, objects, and functions in the world. Cox seemed to presuppose that some authors regard self-awareness as a property of denotational semantics; however, we are not aware of anyone that holds such a view. According to Cox, *self-interpretation* is identified with *introspection*, or *metacognition*, and it behooves a self-awareness machine to understand itself well enough. Introspection entails understanding oneself well enough to not only explain oneself to others, but also generate a set of learning goals in order to improve the knowledge one uses in decision-making (Cox, 2007).

Similarly, McCarthy (1995) defined *introspection* as the observation and reasoning about one's own mental states, including one's own beliefs, desires, intentions, abilities, knowledge, and consciousness. One can, thus, propound that all planners are introspective in a way. A planner has knowledge of its capabilities afforded by the repertoire of operators at its disposal, in addition to knowledge of what operators it can invoke and what goals it can readily achieve in a given situation. Furthermore, in solving a given planning problem, a planner has knowledge of its knowledge deficiencies and can remedy such knowledge gaps. For example, to establish a solvable goal, a planner attempts to identify and perform a sequence of actions whose fulfillment leads to goal attainment.

Introspection also enables a planner to infer non-knowledge and do nonmonotonic reasoning. Under the closed world assumption, a planner presumes false anything it cannot prove. As another example, a Prolog planner based on Horn

clauses may treat non-knowledge as failure (McCarthy, 1995); i.e., for an arbitrary proposition p, if the planner can prove neither p nor $\neg p$, then the planner can infer that it does not know whether p holds. Moreover, an introspective machine reasons about the correctness and consistency of its knowledge base, revising and contracting it appropriately in the face of new information. For example, $C(Now, I) : \neg Know(Telephone(Clinton))$ in the mental situation calculus (McCarthy, 1995) allows a machine to assert that it does not know Clinton's telephone number. This determination by the machine can, in turn, motivate the machine to plan and act accordingly to get the number. Once the machine has acquired the number, it revises its knowledge base to reflect its knowledge of Clinton's telephone number.

Anderson and Oates (2007) proclaimed introspection as a pillar upon which the robustness of an intelligent agent rests. That is, introspection provides a level of decision-making that makes the agent error-tolerant, robust, and responsive to a dynamically changing environment. However, Cox (2005) deplored the lack, as of year 2005, of implementation of any truly introspective mechanisms. As McCarthy noted in (McCarthy, 1995), although reason maintenance systems record the justifications for their beliefs and perform belief revision and contraction appropriately, they cannot observe or reason about their justification structures, due to a lack of introspective subroutines.

2.3 BDI Architecture

The integral role played by such mental attitudes as beliefs, desires, and intentions in the design of intelligent agents has been extensively investigated and well acknowledged in both Philosophy and AI literature. Specifically, these attitudes are instrumental in determining the behavior of an agent as it strives to accomplish a given goal. Furthermore, intention has been singularly noted as being primarily

responsible for maintaining a "rational balance" among the beliefs, commitments, plans, goals, intentions, and actions of an autonomous agent (Cohen and Levesque, 1990a), whereby by "rational" is meant that the agent should be committed to achieving a given goal, but be able to change its focus and pursue a new goal when the situation warrants it.

In (Liu, 2008), we surveyed the distinct role of intention in governing an intelligent agent's behavior, and in how an agent endowed with beliefs, desires, and intentions can benefit from this endowment. One perspective surveyed is Bratman's view of intention (Bratman, 1987), informed through a philosophy of mind and action perspective. In contrast to prior philosophical literature reducing intention to some combination of belief and desire, Bratman proposed several characteristic functional roles of intentions distinguishing intention from belief and desire. Additionally, Bratman showed how one can adopt intentions relative to a background of relevant beliefs, goals, and other intentions.

In the spirit of Bratman's work on intention (Bratman, 1987, 1990) but coming from an AI perspective, Cohen and Levesque (1990a) analyzed various properties of intention and the significance of such properties in the specifications of intelligent agents that aim to approximate a theory of human action. Among Cohen and Levesque's contributions is their formalization of the conditions admitted by an agent's intentions under which an intelligent agent should and can abandon its goals. Cohen and Levesque's theory also shows how an intelligent agent can avoid intending all the anticipated side-effects of its intended action. Cohen and Levesque's work was also thoroughly reviewed in (Liu, 2008).

Extending work in belief-desire-intention (BDI) agents still further is Rao and Georgeff's logical framework founded on the primitive modalities beliefs, desires, and intentions (Rao and Georgeff, 1991, 1995). Their framework formalizes intentions based on a branching-time possible-worlds model, leading to a BDI architecture for rational reasoning. An improved version of this architecture, in turn,

evolved into the goal-directed and reactive PRS (Georgeff and Lansky, 1987), one of the first practical, agent-oriented systems implemented based on the BDI architecture.

The BDI architecture consists of an input queue of events, and three dynamic data structures representing the agent's beliefs, desires, and intentions. All four data structures are assumed to be globally shared, and the system can recognize both internal events and external events. Moreover, both events and the agent's actions are assumed to be atomic. Central in the architecture are the update operations on the agent's beliefs, desires, and intentions to ensure the agent's mental attitudes are as specified in the theory. Algorithm 1 outlines the architecture.

Algorithm 1 Rao and Georgeff's BDI Interpreter

Procedure BDI-Interpreter[ ]

```
1: initialize-state();
2: Loop {
3: options ← options-generator(event-queue);
4: selected-options ← deliberate(options);
5: update-intentions(selected-options);
6: execute();
7: get-new-external-events();
8: drop-successful-attitudes();
9: drop-impossible-attitudes();
10: }
```

Each cycle begins with the option generator reading the event queue and providing a list of options. However, one wonders how options are really generated. After all, the point of planning is to discover actions that will advance the agent towards its goals, and this in general requires exploring and evaluating sequences of possible actions, extending well beyond the current time point. Without such extrapolation to the future, the agent is in effect purely reactive, or at least short-sighted.

From this list of options, the deliberator selects a subset of options to be adopted, and adds them to the intention structure. At this point, if the agent

has an intention to execute an atomic action, then the agent executes it. Next, any external events that have occurred during the cycle are appended to the event queue. In contrast, internal events are appended to the event queue as they occur. The agent then modifies both the desire and intention structures by dropping not only all successful desires and satisfied intentions, but also all impossible desires and unattainable intentions.

Rao and Georgeff (1995) contended that although the architecture conforms to the theory faithfully, it is not a practical system for rational reasoning for two reasons. First, since the architecture is based on a deductively closed set of beliefs, desires, and intentions, the provability procedures required are undecidable. Second, it fails to specify how the option generation and deliberation procedures can be made sufficiently fast so as to meet the system's real-time demands. To provide a more practical system for rational reasoning, Rao and Georgeff added more stringent representational choices to the architecture. The resulting practical system is the PRS, one of the first agent-oriented systems implemented based on the BDI architecture.

PRS (Procedural Reasoning System) (Georgeff and Lansky, 1987), proposed by Georgeff and Lansky in 1987, is an agent architecture based on the BDI framework for intelligent agents. At any instant, the actions being considered by PRS depend both on its current desires or goals and on its beliefs and previously formed intentions. Since PRS interleaves planning and performing actions in the world, at any instant of time, the system has a fluid and partially specified plan. Furthermore, PRS is able to reason about and modify its own internal state, which is composed of its own beliefs, desires, and intentions; thus, PRS offers reactivity crucial for survival in dynamic and uncertain worlds.

PRS is more computationally feasible and responsive to real-time demands than is the basic BDI architecture. The improvement realized in PRS can be attributed to three particular representational choices (Rao and Georgeff, 1995).

First, only beliefs about the current state of the world are explicitly represented in PRS, and only ground sets of literals are considered. Second, each intention that the system forms by adopting certain plans of action is represented implicitly using a process stack of hierarchically related plans. Third, information regarding the means of attaining certain future world states and the options available to the agent are represented as *plans*.

Additionally, to ensure that PRS can satisfy real-time demands while functioning in a dynamic environment, the process of option generation is streamlined by inserting an additional procedure, post-intention-status(), at the end of the interpreter loop (Rao and Georgeff, 1995). Aside from this, PRS's main interpreter loop is identical to the loop in Algorithm 1. This procedure allows the system to defer appending events to the event queue regarding any changes to the intention structure until the end of the interpreter loop. Rao and Georgeff (1995) stated that this way of posting appropriate events to the event queue enables the system to specify which changes to the intention structure will be noticed by the option generator. As a result, various notions of commitment can be modeled, yielding different behaviors of the agent (Rao and Georgeff, 1992) as warranted by the circumstances in the current situation.

The system consists of a *database* of current *beliefs* and facts about the world expressed in first-order predicate calculus, a set of current *goals* or *desires* to be achieved, a set of *procedures* or *knowledge areas (KAs)* describing how certain sequences of actions and tests may be executed either to accomplish given goals or to react to particular situations, and an *interpreter* or *inference mechanism* for manipulating these components. At any instant of time, the system also has a *process stack* of all the currently active KAs, which can be considered the system's current *intentions* for accomplishing its goals or reacting to an observed situation. Figure 2.2 shows the PRS system structure; refer to (Liu, 2008) for details regarding the database, goals, and KAs of PRS.

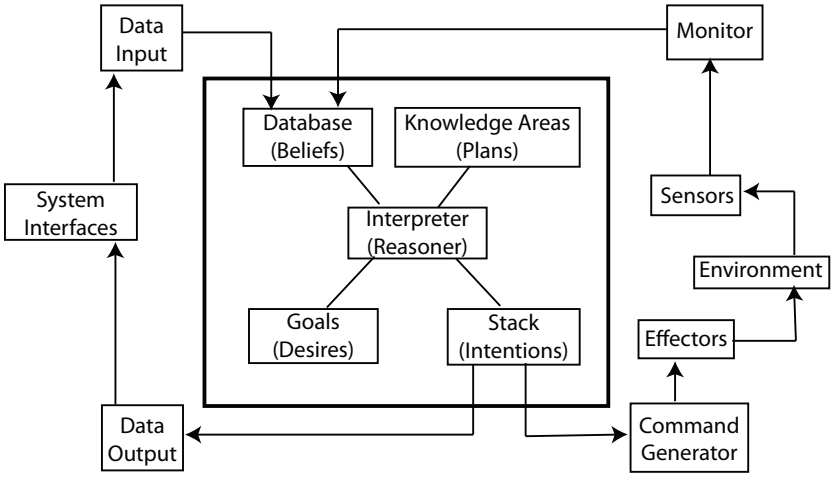


Figure 2.2: The System Structure of PRS

The interpreter in PRS runs the whole system, interacting with the database (beliefs), the goals (desires), the process stack (intentions), as well as the library of KAs (plans). The interpreter maintains beliefs about the current world state, determines which goal to strive to achieve next, decides which knowledge area to apply in the current situation for the achievement of the selected goal. At any particular time, extant beliefs in the database and active goals in the system trigger a set of applicable KAs. Once the interpreter has selected a goal to pursue, it selects one of the applicable KAs relevant to the achievement of the selected goal and places the selected KA on the process stack. The interpreter then executes this KA.

As the interpreter executes the selected KA, new subgoals that emerge are posted to the goal stack, and new beliefs derived are incorporated into the database. When a new belief is added to the knowledge base, the interpreter will perform necessary consistency maintenance procedures, possibly activating other relevant KAs. Similarly, When a new goal is posted to the goal stack, the interpreter checks to see if any new KAs are now applicable, selects an applicable KA, places it on the process stack, and executes it. Georgeff and Lansky (1987) stated that

various meta-level KAs may be invoked by the interpreter to choose a KA from a set of applicable KAs and make choices among alternative paths of execution. This results in an interleaving of plan selection, formation, and execution.

In our review of the PRS-related literature, we were unable to find the complete formal semantics of the original PRS system; the most related coverage, though, is in (Georgeff et al., 1985) on the semantics of a procedural logic. Furthermore, several issues are unclear due to details omitted from the PRS-related papers. These include the criteria the PRS interpreter uses to select a goal to pursue among the goals on the goal stack, the criteria the PRS interpreter uses to select a KA to execute among the currently applicable KAs for its selected goal, how the PRS interpreter determines the applicability of a KA (whether theorem proving as in the original STRIPS (Fikes and Nilsson, 1971) is involved), and so on.

PRS is one of the first practical systems for rational reasoning. Specifically, it has been used in a variety of applications including the fault diagnosis in a space shuttle (Georgeff and Lansky, 1986) and the control of a mobile robot (Georgeff et al., 1986). (Georgeff and Lansky, 1986; Georgeff et al., 1986) showcased PRS's dual role as a reactive planner and reasoner; a detailed example involving the route planning and navigation of a robot is included in (Liu, 2008; Georgeff and Lansky, 1987). Georgeff and Lansky (1987) attributed the success of PRS to its partial planning strategy, its reactivity, its use of procedural knowledge, and its meta-level, reflective capabilities.

The PRS architecture is innovative in that it demonstrates how the attribution of beliefs, desires, and intentions to an autonomous system can assist both in interacting with the system and in specifying complex system behaviors (Georgeff et al., 1986). Note that, however, Georgeff et al. was not concerned with the actual implementation of PRS, for example, whether distinct data structures were incorporated to explicitly represent psychological attitudes. Rather, they suggested viewing the specification of the PRS system, along with its KAs, as a description

of the desired behavior of an autonomous system.

Despite the innovative and practical features of the original PRS (Georgeff and Lansky, 1987), it has several limitations that need to be addressed. First, there is a large class of facts that the PRS system must be told in order to function correctly. For instance, if the system does not have knowledge of the agent's starting location, the interpreter aborts. Such problem could be remedied perhaps by incorporating knowledge areas which automatically ask for crucial missing information. Second, reasoning about process interference and synchronization is essential for concurrency control in multi-agent planning. The system should be augmented to integrate more complex reasoning about interprocess communication, such as that described in (Cohen and Levesque, 1990b; Levesque et al., 1990). Third, some goals are not made as explicit as one would like, but are implicit in the KAs used by the agent. For example, while it is straightforward to handle a goal like "move forward as fast as possible," the agent can only attempt to approximate a goal like "travel along the center of the hallway;" an approximation of the latter goal is given in (Georgeff et al., 1986).

2.4 Homer

Homer was Vere and Bickmore's endeavor to construct a conscious agent "embedded in time and functioning in a simulated dynamic environment" (Vere and Bickmore, 1990). The agent is an integration of temporal planning and reasoning, reactive replanning, action execution, equipped with natural language understanding and generation, episodic memory and reflection, and symbolic perception. The creation of the agent had been inspired and rendered possible by the progress in computer hardware and such AI components as parsers, semantic interpreters, generators, and inference systems (Vere, 1991).

Homer's simulated, two-dimensional sea-world environment consists of Homer,

such fixed elements as a land, an island, a dock, a breakwater, and a pier, as well as such floating objects as logs, icebergs, and mines, ships, and boats. The environment is, for the most part, non-dynamic; almost all objects in the world are motionless, with the exception of a moving ship which patrols back and forth on a fixed path. Rather, changes to the environment, when they do occur, arise as a result of arbitrary manipulation of objects specified by human users. At (re)initialization, Homer knows only its own location and orientation and that of the fixed objects, and most movable objects are placed randomly in the world.

Homer's action repertoire includes the following capabilities: swimming about within the world; picking up, holding, dropping, shooting, photographing, or perceiving objects; refueling; hearing and generating natural language utterances over a simulated radio-telephone link. Homer's actions are distinguished from external events in the world by their coordinates in the two-dimensional plane and the time at which they occur. Initially, Homer does not know of objects in the environment, and learns about them only through conversation and perception.

Homer has limited natural language understanding and perception capabilities. He has a vocabulary of roughly 800 words. In his simulated perception system, Homer's line of sight spans 120 degrees and measures a radius of 150 feet. Attributes of perceived objects are fed directly into Homer's episodic memory.

To facilitate interactions with users, a graphical representation of the environment is displayed on a terminal, and the agent communicates with human users via a natural language text interface. Through this interface, Homer answers user-posed questions and responds to user-given tasks such as taking a picture of an object, or picking up and delivering an object.

Homer's architecture consists of five main components. These components together interpret and execute user commands, consulting and maintaining two repositories of information – the episodic memory and the dynamic plan. These architectural components are the text interpreter, the reflective processes, the

planner based on DEVISER (Vere, 1983), the plan interpreter, and the text generator. In this design, components may share information with one another either directly, or indirectly through memory.

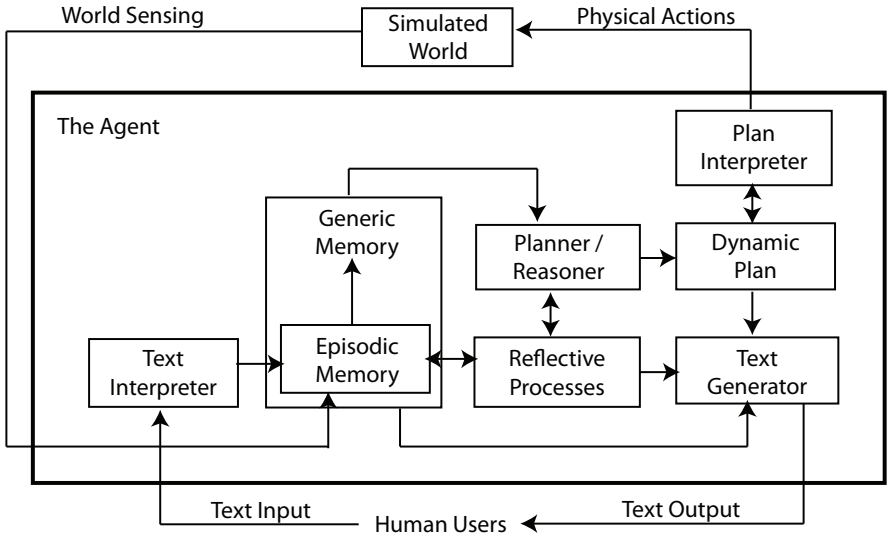


Figure 2.3: The System Block Diagram of Homer

Figure 2.3, recreated from (Vere and Bickmore, 1990), shows the major architectural components and their interactions. Homer's parser, natural language interpreter, sentence generator, declarative and procedural knowledge, dual action representations, episodic memory, and reflection system are all detailed in (Liu, 2008).

Homer's state-transition representation is a frame-like description of verbs complete with a description of the effect of the action on the world state (Vere and Bickmore, 1990). It assists in the integration of natural language processing and planning. To plan to use an action, one must have a representation of what the action does to the world state, what the preconditions of the action are, and how the action procedurally expands into subactions if it is a macro-action.

Consider the following definition in Figure 2.4 for one sense of "give" given by Vere and Bickmore (1990), as a bitransitive verb. It specifies that with both the

giver and receiver being animals, if the giver wants the receiver to own an object currently in the giver's possession, then ownership of the object will be transferred to the receiver as a result of the "give" action.

```
(ACTION (.agent .recipient .thing)
    (CONSTRAINTS  (ISA .agent ANIMAL)
            (ISA .recipient ANIMAL)
    )
    (ANTECEDENT   (POSSESS .agent .thing)
            (GOAL .agent (POSSESS .recipient .thing))
    )
    (CONSEQUENT   (POSSESS .recipient .thing)
            (NOT (POSSESS .agent .thing))
    )
)
```

Figure 2.4: One Sense of Action "give"

Homer's planner operates in two modes. The first mode is for general commands. In this mode, the planner first generates a sequence of actions to achieve a given goal using its knowledge of the world in memory. The planner then stores this information in the dynamic plan ready to be passed on to the plan interpreter for execution. A temporal plan may consist of actions, events, or inferences, and goal protections are implemented and monitored to allow for both detection of plan violation and explanation of motivation for actions of interest. To answer a user-posed question, the planner works in the second, inference mode, using only inferences and not real actions to obtain its goal, i.e., the answer to the question. Next, the text generator, by accessing the dynamic plan, verbalizes the extracted answer from the inference plan generated by the planner.

The plan interpreter performs plans stored in the dynamic plan. In addition, it ensures that plans are executed in the proper sequence and at the proper time. When a temporal plan is formed, nominal start times for each action are determined. An action is initiated when all its predecessors have finished and when its start time has arrived. If the desired start time has arrived but some predecessors have not finished, then initiation of the action is delayed until all predecessors have

finished. When Homer's reactive capabilities (to be described below) are invoked, however, plan execution is temporarily suspended, actions in execution halted and the relevant world state updated. Plan execution resumes once replanning is complete.

Dynamic by nature, the planning is responsive and always subject to the circumstances; e.g., plans can be modified to reflect a more efficient way to perform the task. In addition, plans can also be revised to accommodate changes in the environment caused by other plans or external events. Homer supports some reactivity via two means – replanning and obstacle avoidance. By the latter means, when confronted with an obstacle in its path, Homer will instead follow a path around the obstacle and recompute a path to the original destination from its new location.

Specifically, Homer employs a combination of reactive features:

(1) detection and reaction to plan violation;

(2) detection and reaction to subplan obviation;

(3) excuse-making; and

(4) perseverance after failure to synthesize a plan.

In (1), new information received by Homer, through conversation or perception, is checked to see if it clobbers an old fact that must be protected for the current plan. If so, the relevant portion of the plan is pruned back to the goal, and the planner is reinvoked to recreate the plan. In (2), each new fact entering Homer's memory is matched against the original, uninstantiated form of goals of future planned actions and events. When a match is detected, the action pyramid above the goal is destroyed, and the planner is reinvoked to achieve the matched goal. In (3), when Homer fails to form a plan in response to a command, he gives a natural language excuse to explain the problem; e.g., "Sorry, I don't know

the location of David." In (4), Homer will reattempt previously impossible goals should the situation become favorable later. To do this, Homer keeps a record of blocking goals, those which themselves could not be achieved and none of whose predecessor goals were achieved either. Not only do blocking goals provide the basis for a verbalized excuse for failing to generate a plan, but they also serve as the trigger for dynamically generated goal reactivation demons.

Homer is a paragon of an explicitly self-aware agent. Equipped with an episodic memory, Homer possesses a self-model integrated with general reasoning mechanisms. Furthermore, Homer exhibits coherent behavior in that all of his actions are aimed at accomplishing tasks specified by users. Homer's integrated range of capabilities enable him to respond appropriately to user-posed assertions, commands, and questions. For a given command, Homer's temporal planner first constructs a plan, resolving temporal references with respect to agent time, and then the plan interpreter executes the plan. Homer can also answer questions regarding his experiences, general knowledge, perceptions, present activities, and future intentions.

Homer, albeit personified, can hardly be considered a fully alive, humanoid agent. Notably, Homer lacks desires, preferences, and initiatives. Homer, being highly goal-directed, acts only in response to user-specified commands, questions, or assertions, nor does Homer learn from his experiences. Moreover, Homer is not easily extensible or adaptable to new environments, probably due to the mixing of internal variable management with domain symbolism in the procedural representation. Consequently, Homer has not, so far, become a springboard for development of a more sophisticated agent.

3 Integrating Self-Motivation into Logical Planning and Inference

We formally present our self-motivated cognitive agent framework that unifies self-motivation with logical planning and reasoning. Not only can our self-motivated cognitive agent introspect and reason, thanks to its mental modeling and inferential capabilities, but it can also think ahead, plan and act opportunistically in an incompletely known world. By opportunistically, we mean that the agent can recover from unexpected action failures, seize unexpected favorable opportunities, and avert unexpected threats, all while striving to optimize its own cumulative utility.

In Sections 3.1 and 3.2, we elucidate what constitutes a self-motivated cognitive agent and our development framework for creating such agents. Section 3.3 details the planning process and how actions are defined for an agent, and delineates the syntax and semantics of action definitions when we are specifying actual actions as executed when running the simulated world. We delve into a prudently restricted version of the closed world assumption used by the agent to perform reasoning including introspection in Section 3.4. Lastly, in Section 3.5, we analyze the deliberate self-motivation and self-awareness exhibited by our agent.

3.1 Self-Motivated Cognitive Agent Framework

We conceive of a self-motivated cognitive agent as one whose activity is governed by continual, reasoned elaboration and execution of a life plan, much as first suggested in (Morbini and Schubert, 2008). The agent is not assumed to have complete knowledge about the world, either about the current state or about exogenous events, but rather, it observes new or altered facts as it goes. By its nature, such an agent will manifest *opportunistic*, or *situated* behavior. That is, as new facts become apparent to the agent, or actions fail, its continual planning will cope appropriately with unexpected opportunities and hazards, factoring them in so as to effectively exploit newly recognized opportunities or avoid newly recognized threats.

One naturally wonders that if the agent is self-motivated, in the sense of attempting to maximize its own long-term satisfaction or utility, what should its goals be? It has sometimes been proffered that one might implant an explicit goal or desire such as "thrive" or "achieve happiness," perhaps with Asimov-like "do no harm" constraints added. However, we believe that such an approach raises more questions than it answers, and suggest instead that goals are a *by-product* of reward-based planning, rather than its driving force. In other words, goals are not pursued for their own sake, but for their anticipated contribution to the sum of agent's subjective rewards over the long term.

Accordingly, the framework and LISP-based platform we have built treat planning as continual construction, evaluation, and partial execution of sequences of potential actions. Both actions and states can be intrinsically rewarding or odious to the agent, and the extent of such utility or disutility depends on the exact action parameters involved and on arbitrary descriptive numerical features of the world and the agent. Consequently, the specification of the LISP functions that measure action and state utility is left open to the person using our framework to

design his or her own self-motivated cognitive agent and simulated world. Actions and states with a large anticipated utility will tend to be favored by the planning process. The planning process is designed to aggregate total utilities over courses of action, and then propagate these back to allow a current choice of action that is seemingly optimal "in the long run." Rewards and costs need not be limited to intrinsically pleasurable or unpleasant actions and states, but may perfectly well be associated with actions or states that tend to foreshadow good or bad outcomes, akin to joyous anticipation or dread, respectively.

To qualify as a *cognitive* system, the agent should not only plan, but do so within an expressively rich language for describing, and making inferences about, actions and the world. Instead of choosing a situation calculus-based framework like Golog (e.g., (Ferrein et al., 2004)), we adopted a more STRIPS-like representation of actions and states, both for ease of use in implementing a set of operators and for planning efficiency. Our action representations allow for quantitative preconditions and effects (including variable elapsed time), and these are handled by a very simple, general procedural attachment syntax to be expounded later in Section 3.3. Moreover, the logic of state descriptions allows for propositional attitudes such as *knowing−that*, *knowing−whether*, and *wanting*. This is essential for formalizing knowledge-acquisition actions, such as asking a question, and for answering questions about the agent's own or other agents' attitudes.

3.2 Overview of Framework

Our self-motivated cognitive agent framework confines our protagonist agent, to a simulated "grid world". We dubbed our exemplary self-motivated cognitive agent ME for Motivated Explorer, and will refer to it as ME for clarity in this dissertation. ME can plan and think ahead (where actions produce gradual change over time), deal with unforeseen events, make inferences about states (including its

own and other agents' knowledge and wants), engage in question-answering with the user and with other agents. With seamless integration, ME does all of the above indefinitely and autonomously, driven by the expected cumulative rewards or costs of its contemplated sequences of actions and anticipated future states.

The geography of such a simulated world consists of an arbitrary designer-defined[1] locations, and named roads connecting some or all of these locations. This is enabled with a *def-roadmap* function. With function *place-object*, the designer can not only place designer-defined animate entities (*agents*) and inanimate entities (*objects*) of designer-specified types at various locations, but also associate objects that an entity has (as possessions or parts) and list miscellaneous ground literals describing the entity. For an *agent*, the specified ground literals may include ones like beliefs and preferences. The types ascribed to entities are separately specified with a *def-object* function, and this also allows shared properties of entities of that particular type to be enumerated. For *agents*, the designer may include properties such as being able to talk. While ME is mobile and capable of exploring the world and interacting with the user and other entities, all other entities are stationary and merely reactive in their interactions with ME.

In addition to the general knowledge implicit in type specifications (*def-object* declarations), our framework also allows the designer to specify arbitrary conditional knowledge. Examples of general knowledge may include properties of certain entity types (e.g., that every sasquatch has an IQ of 50), or that certain conditions entail others (e.g., that being asleep implies not being awake, or that p is true if someone knows that p is true). These *general knowledge inference rules* are stated by the designer in Horn clause-like form, with the important generalizations that the (implicitly conjoined) antecedent conditions of the clause may be positive or negative literals, and that the literals may be based on propositional attitude

[1]The designer intended here is any person who uses our framework to define ME's action repertoire and create a simulated world in which to situate ME and other entities.

predicates such as *knows*, with reified arguments such as (*that* (*is_edible* *fruit*3)). These rules are used for bounded forward inference in elaborating world state and mental state descriptions. The designer of the simulated world can set the depth of bounded forward inference. When the antecedent of such a rule unifies with a fact in a given world state or mental state, the resulting grounded consequent is derived in the world state or mental state.

ME's knowledge base is automatically initialized so that it contains the geographical knowledge about the world and the general quantified conditional facts. Its world model is also augmented with specific facts about itself, its initial location, and about the entities at that location. Notably, *ME* has only partial knowledge about the world, even about entities at its current location. In particular, the designer of *ME*'s world may designate certain predicates as being *occluded*, and these are predicates for local facts that the designer wants to regard as not immediately known to any entity other than possibly the certain agent or agents named in the predication. For example, the predicate *is_in* might be occluded, so if (*is_in* *key*1 *box*1) holds, *ME* does not know this even when standing next to an unopened *box*1. Similarly, *knows* would generally be occluded, so that what another agent colocated with *ME* knows is not apparent to *ME*. There are two types of exceptions, however. First, if a certain argument (usually the subject) of the predication is *ME*, then the predication is not occluded from *ME*. For example, self-referential facts like (*has* *ME* *key*1) and (*not* (*knows* *ME* (*whether* (*is_edible* *fruit*3)))) would be evident to *ME*, and thus added to *ME*'s world model, despite the general occlusion of *has* and *knows*. Second, *ME* may find out and remember a fact that would otherwise be occluded, perhaps because it asked a question, read a book containing the fact, or inferred it.

The incompleteness of *ME*'s knowledge has several important consequences for the design of the cognitive agent framework. One is that a distinction needs to

be made between the simulated world and ME's model of it, in terms of both the action effects and the action termination conditions (which may be determined by external events, not anticipated by ME). Another is that under conditions of incomplete knowledge, ME cannot use a full closed-world assumption (CWA), and consequently must be careful not only in its evaluation of the truth or falsity of action preconditions and effects, but also in its introspections concerning what it knows and does not know. We discuss ME's limited use of closed-world reasoning and its introspective reasoning in Section 3.4.

3.3 Actions, Planning, and Simulation

The designer can define ME's action repertoire of operators, or action types, in a STRIPS-like syntax, with a list of parameters, a set of preconditions, and a set of effects; an action also has an anticipated reward value and an expected duration. As an example, consider the following *sleep* operator in Figure 3.1 with formal parameters $?f$ for ME's fatigue level and $?h$ for ME's hunger level. From ME's perspective, if it is at home, more tired than hungry with a fatigue level of at least 8.0, and there is no fire, then it can sleep for a duration of $(* \ 2 \ ?f)$ time units and, as a result, it will completely relieve its fatigue at the expense of increasing its hunger level by half its starting fatigue level $?f$. As given by the value $(* \ 4 \ ?f)$, execution of this particular action alone is estimated by ME to contribute to an increase in its cumulative utility equal to four times its starting fatigue level $?f$.

Instantiating an operator into an action requires replacing its formal parameters with actual values obtained by unifying the preconditions with the facts in ME's conception of the current state along with the permanently true facts. That is, an action can be created in a given state only if ME finds its preconditions to be true according to ME's knowledge in that state; ME considers such an action *applicable* although ME might not elect to perform this action in that state.

```
(setq sleep
   (make-op
   :name 'sleep
   :pars '(?f ?h)
   :preconds '( (is_at ME home)
                (is_tired_to_degree ME ?f)
                (>= ?f 8.0)
                (is_hungry_to_degree ME ?h)
                (> ?f ?h)
                (not (there_is_a_fire))
              )
   :effects '( (is_tired_to_degree ME 0.0)
               (not (is_tired_to_degree ME ?f))
               (not (is_hungry_to_degree ME ?h))
               (is_hungry_to_degree ME (+ ?h (* 0.5 ?f)))
             )
   :time-required '(* 2 ?f)
   :value '(* 4 ?f)
   )
)
```

Figure 3.1: The *sleep* Model Operator

ME accomplishes planning by first doing a forward search from a given state s, followed by back-propagation to s of the anticipated rewards and costs of the various applicable actions and states reached, to determine a/the seemingly most rewarding sequence of actions. The forward search is constrained by a search beam, preset by the designer of the simulated world to stipulate the number of lookahead levels (i.e., the length of each contemplated sequence of applicable actions), as well as the branching factor and the allowable operators at each level. Informed by the projective lookahead, ME will then execute the first action of a/the seemingly best plan and update its knowledge with (a) the action effects, (b) its new observations of the non-occluded, local facts, and (c) new facts (if any) derived by unifying its knowledge with the antecedents of designer-specified general knowledge inference rules in bounded forward reference. The state of the simulated world will also be updated with (a) and (c), except that (c) is derived by unifying the facts in the world state with the antecedents of the same inference

rules in bounded forward reference, as was first broached in the previous section. ME is then ready to plan forward from the new state.

As noted at the end of the previous section, when an agent's knowledge of the world is incomplete, the actual effects of its actions may diverge from the effects expected by the agent. In particular, the simulated world can be a dynamic one in which optional, designer-defined exogenous events such as fire and rain can spontaneously begin and end with some probability at each time step. For example, if we model traveling (from ME's perspective) as a multi-step action, but also allow for spontaneous fires that bring travel to a halt, then ME may not reach its expected destination. Given this disparity between expectations and reality, we clearly need to distinguish ME's conception of its actions from the "actual" executed actions in the simulated world. In stark contrast, this is not an issue in most classical planners, as they assume that their work is done when they have found a path to the goal in their conception of the problem domain.

The need for separate modeling of changes in the simulated world, as opposed to modeling of changes anticipated by the agent, would be patently obvious if one imagined a more elaborate simulated world, with other mobile agents, graphical animation, etc. Since ME's behavior is driven uniformly by a continually evolving plan at each time step, not only should the designer specify a stepwise version of each operator that ME can execute, but the framework should (and does) run an instantiated stepwise version of the selected operator in actual execution. The designer defines "actual" action types much like ME's models of these actions, but allowing for stepwise updates and possible interruptions. Consider the following example in Figure 3.2 of the "actual" stepwise version of the *sleep* operator. Note the addition of $(there_is_a_fire)$, indicating an exogenous fire, to the *stopconds* field.

In the execution, actual actions and exogenous events (also defined with the same syntax) may be tracked in parallel in unit time steps. An active actual

```
(setq sleep.actual
   (make-op.actual
   :name 'sleep.actual
   :pars '(?f ?h)
   :startconds '( (is_at ME home)
                  (is_tired_to_degree ME ?f)
                  (>= ?f 8.0)
                  (is_hungry_to_degree ME ?h)
                  (> ?f ?h)
                )
   :stopconds '( (is_tired_to_degree ME 0.0)
                  (there_is_a_fire)
                )
   :deletes '( (is_tired_to_degree ME ?#1)
               (is_hungry_to_degree ME ?#2)
             )
   :adds '( (is_tired_to_degree ME (- ?f (* 0.5 (elapsed_time?))))
            (is_hungry_to_degree ME (+ ?h (* 0.25 (elapsed_time?))))
          )
   )
)
```

Figure 3.2: The *sleep* Actual Operator

action will continue for another time step if and only if none of its stop conditions as given by *stopconds* are true in the current world state. If at least one of them is true in the current world state, then the action will terminate immediately. In either case, the world state will be updated by computing the effects as given by *deletes* and *adds*, followed by bounded forward inference. At the same time, ME's knowledge will be updated by the computed action effects, ME's observation of the non-occluded local facts in the new state, and lastly by ME's bounded forward inference.

When an action comes to a stop, whether successfully or unsuccessfully, ME performs its projective lookahead plan search and chooses a step (namely, an action) of its most promising plan to execute, as already described. This is what makes ME's planning situated, or opportunistic, or robust in the event of failure – it always bases its choice of the next action on a seemingly best course of action

that starts in the situation at hand.

The STRIPS-like action representation allows for quantitative preconditions and effects, handled by a uniform procedural attachment technique. This technique is used in preconditions and effects (and value and duration, if applicable) of both the model actions and the actual actions, and it enables not only quantitative reasoning but also dialogue. The designer may define action preconditions and effects (and value and duration, if applicable) containing *evaluable terms*, and these are precisely the functions or predicates whose operators are among $+$, $-$, $*$, $/$, $<$, $<=$, $>$, $>=$, $=$, *random*, and designer-defined function or predicate names ending with the ? symbol.

The system will evaluate the evaluable terms when verifying preconditions and when computing effects (and when contemplating the net utility and duration, if applicable). For example, $(is_tired_to_degree\ ME\ (-\ ?f\ (*\ 0.5\ (elapsed_time?))))$, an effect of of *sleep.actual* in Figure 3.2, specifies that the decrease in ME's fatigue level as a result of sleeping will be half the number of time steps it sleeps, where the designer-defined function $(elapsed_time?)$ returns the number of time steps ME has spent on the current sleeping action thus far.

Not only does procedural attachment handle quantitative preconditions and effects (and the value and duration of an action, if applicable), but it also handles side-effects such as ME producing a printed answer to a question straightforwardly. For instance, consider the following operator for answering a yes-no-question given by the user in Figure 3.3.

In the process of instantiating this operator's effect
$(knows\ USER\ (that\ (answer_to_ynq?\ ?q)))$ in ME's knowledge base, the evaluable term $(answer_to_ynq?\ ?q)$ is evaluated by applying the designer-defined LISP function *answer_to_ynq*? to the value bound to variable $?q$. This function consults ME's knowledge base to determine an appropriate answer, such as $(not\ (can_fly\ guru))$ for the question $?q$ being $(can_fly\ guru)$, and also outputs

```
(setq answer_user_ynq
   (make-op :name 'answer_user_ynq :pars '(?q)
   :preconds '( (wants USER (that (tells ME USER (whether ?q)))) )
   :effects '( (not (wants USER (that (tells ME USER (whether ?q)))))
               (knows USER (that (answer_to_ynq? ?q)))
             )
   :time-required 1
   :value 10
   )
)
```

Figure 3.3: The *answer_user_ynq* Model Operator

this answer to the user. Note that the particular effect stored for this example would be $(knows\ USER\ (that\ (not\ (can_fly\ guru))))$, which correctly anticipates the user's resultant state of mind, on the assumption that the user trusts answers offered by ME.

3.4 Reasoning about World States and Mental States

ME does not in general know all the current facts. Apart from geographical facts, it does not know the facts at other, unvisited locations; furthermore, even facts at visited locations may be occluded, or may have changed. Among the occluded facts we would normally have possessions and propositional attitudes of other entities, whereas ME presumably knows all of its own possessions and attitudes. Many traditional planning systems make a closed-world assumption (CWA), according to which absence (or non-inferrability) of a positive fact from the world model is viewed as supporting the negation of that fact. For example, if the world model does not contain $(has\ guru\ key1)$, the negation $(not\ (has\ guru\ key1))$ is assumed. But this amounts to a tacit assumption that all positive facts (at least for the predicates of interest) are provided by ME's world model, contrary to the

observations just made. Therefore, ME can only use a restricted version of the CWA. We do not want to abandon the CWA altogether, as it would otherwise be very inefficient to enumerate and maintain the very large number of negative predications that hold even in simple worlds.

Specifically, ME uses the CWA for every non-epistemic literal in which ME is the first (namely, subject) argument. Epistemic literals, discussed below, are ones with predicate *knows*, any subject, and a *that*-clause or *whether*-clause as object. For example, ME takes statements such as that ME is hungry, or has money, or likes Guru, or wants to answer the user's question, etc., to be false unless the corresponding positive literal appears in ME's current state of knowledge, or synonymously, ME's representation of the current state. In other words, ME's positive self-knowledge is assumed to be complete for such self-referential sentences.

However, when the literal concerns a non-ME subject, ME requires explicit negative knowledge to draw a negative conclusion, except for the following two cases, where it applies the CWA:

1. literals about road connectivity and navigability (and even here the CWA could easily be weakened); e.g., the absence of (*road path*5) from ME's knowledge base, regardless of whether (*not* (*road path*5)) is present, would suffice for ME to conclude that *path*5 is not a *road*;

2. when (a) the subject is a local entity currently colocated with ME or one ME has visited, and (b) the predicate is non-occluded; e.g., a literal expressing that *piano*3 is at ME's current location would be considered false unless explicitly present in ME's representation of the current state.

In all other cases concerning a non-ME subject, the mere absence of a literal from ME's representation of the current state is not interpreted as supporting its negation – its truth value may simply be unknown to ME. Of course, ME may

learn that (*not* ϕ) holds for some literal ϕ even if ϕ is not subject to the CWA. In such a case, it stores that negative literal in its world model. Therefore, in checking whether a literal ϕ is true or false, where ϕ is not subject to the CWA, ME will not conclude "unknown" merely because ϕ is absent from its world model – it also checks whether the negation is explicitly known. ME updates its knowledge of the non-occluded properties of an entity only when it (re)visits its location, and its knowledge of the occluded properties of that entity when it takes appropriate actions to (re)discover them. Algorithm 2 formally summarizes the method of judging the truth value of a non-epistemic ground atom. A negative literal is judged to be true, false, or unknown accordingly as the atom that it negates is judged to be false, true, or unknown.

Epistemic literals require special treatment, because an agent that is committed to a proposition ψ should also be committed to the proposition that it *knows* that ψ, and that it *knows* whether ψ, without having to assert these conclusions separately. Similarly, if it is committed to the falsity of ψ, then it should also be committed to the falsity of the proposition that it knows that ψ, and to the truth of the proposition that it knows whether ψ. And finally, if it committed to neither the truth nor falsity of ψ, it should be committed to the propositions that it does not know whether ψ, does not know that ψ, and does not know that not ψ.

As a result, ME judges the truth value of epistemic literals by an introspection algorithm rather than closed-world inference. The algorithm deals with both positive introspection (about knowledge ME possesses) and negative introspection (about knowledge it lacks). In particular, in the evaluation of a predication of the form ($knows\ SUBJ\ (that\ \psi)$) (e.g., ($knows\ ME\ (that\ (can_talk\ guru))$)) with ψ being a literal, the algorithm considers the two cases $SUBJ = ME$ and $SUBJ \neq ME$. In the first case, ME recursively determines whether the literal ψ that is the object of $knows{-}that$ is true, false, or unknown, and accordingly judges the autoepistemic predication to be true, false, or false respectively (i.e.,

Algorithm 2 Evaluating Non-epistemic Atoms Using a Restricted CWA

1: Let $(P\ t_1\ t_2\ ...)$ be a non-epistemic ground atom.
{ * We assume that t_1, t_2, ... are either constants, or *evaluable terms* which have been evaluated to constants. Either t_1, t_2, ... are constants (in which case even a *knows*-predication such as $(knows\ ME\ Guru)$ is allowable here), or else P is a predicate other than *knows* (but we allow here, e.g., *wants*, *wonders*, or *contains_a_message*), and t_1, t_2, ... are variable-free terms (constants, or reified propositions formed by applying *that* to a ground literal, or reified yes-no questions formed by applying *whether* to a ground literal). * }

{ * We judge the truth value of $(P\ t_1\ t_2\ ...)$ as *true*, $false$, or *unknown* in a given state (as modeled by ME) according to the following rules. * }

2: **If** $(P\ t_1\ t_2\ ...)$ is in the state description
3: **return** *true*

4: **If** $(not\ (P\ t_1\ t_2\ ...))$ is in the state description
5: **return** $false$

{ * To reach this point in the algorithm must mean that neither $(P\ t_1\ t_2\ ...)$ nor $(not\ (P\ t_1\ t_2\ ...))$ is in the state description. *}

6: **If** P is one of *road*, *connects*, *navigable*, ..., i.e., a roadmap predicate
7: **return** $false$
8: **Else if** $t_1 = ME$
9: **return** $false$

10: **If** P is not marked as an occluded predicate, and t_1 denotes a local entity (colocated with ME) or an entity local to a place that ME has visited
11: **return** $false$

12: **Return** *unknown*

in the latter two cases ME does not know ψ to be true).

In the case $SUBJ \neq ME$, ME judges the epistemic predication as true if the predication is explicitly available in the world model, or $SUBJ$ is identical to the subject of ψ (thus making an assumption that other agents are similar to itself in having complete positive self-knowledge – a type of simulative inference), and false otherwise. The latter assumption implies that other agents know only what ME *knows* they know. Although this rather strong negative closure assumption could be weakened, we were more concerned with realistic epistemic modeling of ME than of other agents.

The method for judging the truth value of predications of the form $(knows\ SUBJ\ (whether\ \psi))$ is much the same. However, in the case of $SUBJ = ME$, when ψ is found to be true, false or unknown, the autoepistemic predication is judged to be true, true, or false respectively. In the case $SUBJ \neq ME$, ME again judges the epistemic predication as true only if $SUBJ$ is identical to the subject of ψ, and false otherwise. The method of evaluating epistemic predications is depicted in Algorithm 3.

These inference capabilities are especially important in two respects. First, they are instrumental in ME's attempt to confirm or disconfirm action preconditions, including knowledge preconditions in actions like asking an agent whether ψ is true (viz., not knowing whether ψ is true, but believing that an agent knows whether ψ is true). Second, the inference capabilities are crucial in answering questions, including ones that test ME's introspective capabilities. Given a question, ME will either inform the user if it does not know the answer, or otherwise verbalize its answer(s) as English sentence(s). Extensive examples of ME's dialogue with the human user are in Section 5.2. They include $knows{-}whether$, $knows{-}that$, yes/no, and wh-questions in a simple simulated world set up in Section 5.1.

Our restricted version of the CWA may seem similar to the idea of *local closed*

Algorithm 3 Evaluating Epistemic Atoms Using Introspection

Let ϕ be of the form $(knows\ t_1\ (that\ \psi))$ or $(knows\ t_1\ (whether\ \psi))$.
{ * t_1 is a constant, and ψ is a ground, possibly epistemic literal. We assume that any embedded terms such as $(+\ account - balance?\ 100)$ have been evaluated to constants. * }

{ * We judge the truth of ϕ as $true$, $false$, or $unknown$ in a given state (as modeled by ME) according to the following rules. * }

If ϕ is in the state description
    return $true$

If $(not\ \phi)$ is in the state description
    return $false$

{ * To reach this point in the algorithm must mean that neither ϕ nor $(not\ \phi)$ is in the state description. * }

Evaluate ψ to $true$, $false$, or $unknown$ recursively using the method of Algorithm 2 or the method of Algorithm 3, depending on whether ψ is non-epistemic or epistemic.

If $t_1 = ME$, or the subject of ψ (the first argument appearing in this literal) is also t_1
    If ϕ is of the form $(knows\ t_1\ (that\ \psi))$
        If value of ψ is $unknown$
            return $false$
        Else { * Value of ψ is either $true$ or $false$. * }
            return value of ψ
    Else { * ϕ is of the form $(knows\ t_1\ (whether\ \psi))$ * }
        If value of ψ is $unknown$
            return $false$
        Else { * Value of ψ is either $true$ or $false$. * }
            return $true$
Else
    return $false$

world (LCW) introduced by Golden et al. (1994) and later adopted by Bacchus and Petrik (1998). In both approaches, a proposition can have a true, false, or unknown truth value to an agent. Whereas we designate certain predicates (and only these predicates) as being occluded to an agent, their approaches allow for a more precise specification by naming certain formulas (and only these formulas) as being subjected to the CWA by an agent. However, our primary purpose is to enable ME to reason about world states as well as its own and other agents' mental states so ME can also be an active knowledge-seeker and an engaging question-answerer. The LCW has largely been used in purely goal-directed planning or partial-order planning with focus on execution-time effects.

Besides ME's capacity for inferentially checking action preconditions and answering questions, it also performs bounded forward inference for any state that it reaches in its simulated world or in its lookahead, by unifying its current factual knowledge with the antecedents of the designer-specified general knowledge inference rules to derive grounded consequents. For example, from $(knows\ guru\ (that\ p))$, ME can infer both p and $(knows\ guru\ (whether\ p))$; from $(sasquatch\ moe)$ and general knowledge $((sasquatch\ ?x)\ =>\ (has_IQ\ ?x\ 50))$ where variable $?x$ is assumed to be universally quantified over the domain, ME can infer that $(has_IQ\ moe\ 50)$.

3.5 Deliberate Self-Motivation and Self-Awareness

We have emphasized the integration of self-motivation with planning, inference, and simple dialogue capabilities as a distinctive characteristic of our agent framework. In this section, we reiterate and expand upon the features of the type of agent we have developed that justify its description as *deliberately self-motivated*, and as self-aware.

The most obvious feature that renders ME self-motivated is its reliance on

anticipated rewards and penalties in its choice of actions. Of course, as noted at the outset, examples of reward-driven agents abound in the AI literature, particularly in the area of reinforcement learning (RL) and behavioral robotics. However, such systems generally are non-cognitive systems: their policies, learned through repetitive randomized exploration, simply map features of situations to actions, without entertaining or communicating any thoughts about themselves., the world they live in, their actions, or future states. Thus, ME differs from such agents by being *deliberately* self-motivated. Conversely, cognitive agents that plan and reason deliberately have lacked self-motivation. They are standardly conceived of as serving some user purpose; goals are generally handed to them, and even where their behavior is reactive, the design ensures that the reactions will comply with preconceived notions of what the agent's priorities *should* be.

ME's self-motivation does not necessarily imply selfishness. While ME may find certain self-involved states and actions rewarding (e.g., eating) or displeasing (e.g., being tired), and this will definitely affect its behavior, ME can also experience vicarious satisfaction, for example by answering the user's questions or by helping another agent in the simulated world. Notably, the anticipatory satisfaction in the projective lookahead can also be used by the designer to implement ME's curiosity, by making the acquisition of new knowledge intrinsically satisfying for ME.

Another important consideration in ascribing self-motivation to an agent, at least in the context of a quest for human-like AI, is that self-motivation should be coupled with self-awareness. One can imagine a system that reasons, plans and chooses its actions in accordance with a utility metric, yet makes absolutely no reference to itself in any of its thinking or its communication (if any). Consider, for example, a sophisticated AI system that helps manage an extensive truck fleet, seeking to maximize the cost-effectiveness of the requisite pickup and delivery of goods. Despite its utility-driven deliberations and decisions, such a system is

scarcely *self*-motivated.

We contend that the various agent capabilities implemented in our framework endow the agent with a significant degree of self-awareness, sufficient to describe its planning and actions as self-motivated. In fact, ME meets most of the requirements for explicit self-awareness (Schubert, 2005). First, the framework enables any number of facts about the agent itself to be represented, which the agent then uses in its planning, question-answering, and forward-inferencing. For instance, besides providing basic self-knowledge about ME's possessions and initial location in the grid world, the initialization process also allows for miscellaneous current state facts and propositional attitudes, such as $(owns\ ME\ bert)$, $(is_a_bird\ bert)$, $(likes\ ME\ bert)$, and $(knows\ ME\ (whether\ (is_hungry\ bert)))$, so that ME will know that it owns and likes bird *bert*, and that it knows whether *bert* is hungry. Also, the introspection algorithm enables ME to make inferences about what it knows and does not know. Furthermore, in a given state, ME knows what operators it can invoke, their anticipated effects, and what goals are readily achievable. Finally, ME keeps a history list of all actions it has performed in the world, and both the list and ME's knowledge base are open to introspection. Moreover, these aspects of ME's self-awareness are not merely theoretical ascriptions: ME can overtly demonstrate its self-awareness through its ability to answer questions about itself, including its own mental states. The experiments we will report in Sections 5.3.2 and 5.2 similarly illustrate that self-awareness is not only of theoretical interest, but operationally significant.

4 Designing and Interacting with a Simulated World

Our self-motivated cognitive agent framework prominently features an integration of self-motivation, logical planning and reasoning, with a capacity for simple question-answering dialogues. Capabilities aside, two equally important characteristics of our framework we would like to emphasize are its generality and ease of use. Two observations lend credence to the generality of our framework. First, as depicted in Sections 5.3.3 and 6.1, we have successfully simulated strictly goal-directed planning behavior, and constructed and solved such classical planning problems as the Towers of Hanoi and a Logistics domain using our framework. Additionally, our framework has been an invaluable education tool in three consecutive offerings of the Machines and Consciousness course taught by Prof. Lenhart Schubert at the University of Rochester; students have been able to create and experiment with their own self-motivated cognitive agents and various simulated worlds in their term projects.

As a precursor to our experimental agent and empirical results in Chapter 5, and in order to help corroborate our framework's ease of use, we specify precisely the steps entailed in designing one's own self-motivated cognitive agent and simulated world from scratch. The steps range from defining and creating a roadmap, entity types and properties, agent instances, object instances, and the self-motivated cognitive agent's action repertoire, to setting such parameters

as the search beam, forward inference bound, manually added general-knowledge inference rules, and the reward and cost metrics for actions and state properties. We also explain how a human user can interact with the self-motivated cognitive agent.

4.1 Syntax Preliminaries

To design a simulated world or to interact with ME, the designer must have the source files `gridworld-planning.lisp`, `gridworld-definitions.lisp`, and `simulation-and-function-evaluation.lisp`. The designer is expected to have basic knowledge of LISP including its syntax. Interested readers may obtain these files by emailing the author at the email address listed on her Webpage at `www.cs.rochester.edu/u/daphnel`

LISP accepts almost all alphanumerical strings for predicate names and function names; while a predicate name or function name cannot be a single dot or a number, syntactically speaking, it can be any string of non-space and non-delimiter characters. That said, it is recommended that the designer abide by the following rules when choosing a name for a *predicate*:

- It should be easily translatable into English (see paragraph below);
- Any verb(s) appearing in it should be in the third person, singular, present tense.

Our framework has already implemented the rendering of predications into English via the **verbalize(wff)** and related LISP routines in `simulation-and-function-evaluation.lisp`. A positive literal is translated into an English sentence by pulling out the subject (i.e., the first argument of the predicate) and placing it first, by replacing underscores and dashes and question

marks with spaces, and by replacing plus signs with arguments in the list of arguments after the first argument of the predicate. For any predicate of arity 2 or higher, its last argument is always inserted at the end of the rendered English sentence, so the designer need not append a question mark to the end of its predicate name unless the designer intends for it to appear as an *evaluable term* (see a few paragraphs below) somewhere in his or her code. For instance, the literal $(is_at\ ME\ (the_point{+}units_from{+}towards{+}on_road?\ ?d\ ?x\ ?y\ ?z))$ is rendered as "ME is at the point d units from x towards y on road z." On the other hand, a negative literal is translated in the same way except that the *not* operator is verbalized by prepending "it is not the case that" to the verbalization of the remainder of the literal. For example, $(not\ (has_IQ\ Grunt\ 50))$ is verbalized as "it is not the case that Grunt has IQ 50."

A *literal* (lit) is either an atomic formula (predication) enclosed in parentheses of the form $(pred\ t_1\ t_2\ ...\ t_n)$ where *pred* is an n-ary predicate name and $t_1, t_2, ..., t_n$ are *terms*, or the negation of such an atom $(not\ (pred\ t_1\ t_2\ ...\ t_n))$.

A *term* is either a number, constant, variable, function, or a *reified predication*. We adopt LISP's inherent definitions of numbers, constants, variables, and functions here, except that the designer should prefix each variable name with the ? symbol to indicate its being a variable in the framework. A *reified predication* has either the form $(whether\ lit)$ or the form $(that\ lit)$ with *lit* being a literal; it typically appears as the object argument of a *wants* or *knows* predicate. For instance, $(knows\ ME\ (whether\ (is_bored\ ME)))$ expresses that ME knows whether ME is bored, and $(wants\ ME\ (that\ (eats\ ME\ pizza)))$ states that ME wants to eat *pizza*.

As previously stated, the logic of state descriptions allows for propositional attitudes such as *knowing-that*, *knowing-whether* and *wanting*. This is essential for formalizing knowledge-acquisition actions and for answering questions about the agent's own attitudes. To be syntactically correct, the object argu-

ments of such (epistemic or not) predicates as *wants* and *knows* must be reified with (*whether* (...)) or (*that* (...)), if they are literals themselves. For instance, (*wants AG* (*that* (*drinks AG juice*))) expresses that *AG* wants to drink *juice*, and (*knows AG* (*whether* (*is_alive AG*))) states that *AG* knows whether *AG* is alive. On the other hand, the object argument of (*wants AG juice*) and (*knows AG juice*) need not be reified with (*whether*(...)) or (*that*(...)) since *juice* is not a literal.

We further distinguish some terms as being *evaluable terms*. These are precisely functions or predicates whose operators are among +, −, ∗, /, <, <=, =, >=, >, *random*, and designer-defined function or predicate names ending with the ? symbol. Evaluable terms often appear in operator specifications and are featured prominently in the procedural attachment technique detailed in Section 3.3.

4.2 Defining and Creating a Roadmap

The designer can define a set of named locations, as well as named roads connecting some or all of these locations. Roads start at, pass through, and end at specified locations, and named roads and named locations have atomic labels.

To define and create a roadmap, the designer should invoke the LISP routine
def-roadmap(locations, roads)
defined in `gridworld-definitions.lisp`. Argument **locations** is a list of distinct location names, such as *point*1, *point*2, *dead−end*3, etc. Argument **roads** is a list of lists, each beginning with a road name, followed by the first point on this road, the distance from the first point to the second point on this road, the second point on this road, and so on. A road must not reach any point twice, because we would like "taking road x from location y to location z" to correspond to a unique path. Theoretically, **locations** may include locations not reached by

any road; a minimalist roadmap would be one that has just one location (where we place ME), and no roads.

For example, the following defines and creates a roadmap with named locations *home*, *grove* and *plaza*, and named roads *path*1 and *path*2:
$(def{-}roadmap\,'(home\,grove\,plaza)\,'((path1\,home\,3\,grove)\,(path2\,home\,2\,plaza)))$
Furthermore, *path*1 connects *home* and *grove*, which are 3 distance units apart; while *path*2 connects *home* and *plaza*, which are 2 distance units apart.

This roadmap definition would automatically add the following specific geographical facts to the world state and to ME's knowledge base:

- $(point\ home)$, $(point\ grove)$, $(point\ plaza)$
- $(road\ path1)$, $(road\ path2)$
- $(navigable\ path1)$, $(navigable\ path2)$
- $(is_on\ home\ path1)$, $(is_on\ grove\ path1)$, $(is_on\ home\ path2)$, $(is_on\ plaza\ path2)$

The above definition, however, does not automatically add connectivity predications to ME's knowledge base. Rather, connectivity information, including length and road name, between each point and its immediate neighbor is directly stored in the *next* property of each point. Therefore, the *next* properties of *home*, *grove*, and *plaza* are $((path1\ 3\ grove)\ (path2\ 2\ plaza))$, $((path1\ 3\ home))$, and $((path2\ 2\ home))$, respectively.

4.3 Defining and Creating Types and Entities

Specific entities are defined and placed in a simulated world in two stages. First, the designer has to define some types of entities, and their associated general,

permanent properties. This is done by using the LISP routine

def-object(obj-type, properties)

of file `gridworld-definitions.lisp`. Argument **obj-type** is an atomic identifier for a type, e.g., *sasquatch*. Argument **properties** is a list of abbreviated general, permanent properties such as $(is_animate\ is_furry\ (has_IQ\ 50))$. This is automatically expanded so as to produce a conditional whose antecedent applies the type predicate to a variable and whose consequent applies the given predicate in each property first to the variable and then to any additional arguments that are given. Thus, for the aforementioned example, the conditionals automatically generated and added as general knowledge inference rules (see Section 3.2) are as follows:

- $((sasquatch\ ?x)\ =>\ (is_animate\ ?x))$
- $((sasquatch\ ?x)\ =>\ (is_furry\ ?x))$
- $((sasquatch\ ?x)\ =>\ (has_IQ\ ?x\ 50))$

Next, the designer invokes a function to create a named entity of a particular type, placing it at a location along with its associated things, and some facts about its current state and about its current propositional attitudes. The function call is of the form

place-object(name, obj-type, loc, assoc-things, curr-facts, prop-attitudes)

in file `gridworld-definitions.lisp`. The first three arguments are each an atomic label and serve self-explanatory purposes. Argument **associated-things** is a list of typed entities such as $((key\ key1)\ (sword\ sword1)\ (pen\ pen3))$; these may be regarded as possessions in the case of animate entities (agents), or as contained or attached objects in the case of inanimate entities (objects). For example, $(key\ key1)$ supplied under this heading means that the named entity has $key1$ and $key1$ is a key. Argument **curr-facts** is a list of current state facts about

the named entity; e.g., ((*is_hungry Grunt*) (*likes Grunt Tweety*)). Argument **prop-attitudes** is a list of propositional attitudes such as ((*knows Grunt* (*that* (*has Rob pen*3))) (*wants Grunt* (*that* (*has Grunt pen*3)))).

The type fact, location fact, possession or association facts, current state facts, and propositional attitude facts about the entity are all automatically placed in the world state; furthermore, if and only if the named entity is ME, they are also added to ME's knowledge base as ME's extra initial knowledge (in addition to roadmap knowledge, self-knowledge and any locally non-occluded facts).

4.4 Defining an Action Operator

For each physical action-based ability the designer wishes to endow ME with, the designer needs to define both the *model version* and the *actual version* of the corresponding action operator. We elaborated on both the semantics and the reasons for this distinction in Section 3.3.

The following is the schema for the model version of an operator named *action_name*, with requisite fields :*name*, :*pars*, :*preconds*, :$effects$, :$time{-}required$, and :*value* :

```
(setq action_name
   (make-op
      :name 'action_name
      :pars '(?var1 ?var2 ... ?varn)   ; where ?vari's are variables
      :preconds '( lp1 lp2 ... lpn ) ; where lpi's are literals
      :effects '( le1 le2 ... len )  ; where lei's are literals
      :time-required 'term1  ; where term1 evaluates to some number
      :value 'term2          ; where term2 evaluates to some number
   )
)
```

The following is the schema for the corresponding actual version of the same

operator, with requisite fields :*name*, :*pars*, :*startconds*, :*stopconds*, :*deletes*, and :*adds* :

```
(setq action_name.actual
   (make-op.actual
        :name 'action_name.actual
        :pars '(?var1 ?var2 ... ?varn)   ; where ?vari's are variables
        :startconds '( lstt1 lstt2 ... lsttn ) ; where lstti's are literals
        :stopconds '( lstp1 lstp2 ... lstpn ) ; where lstpi's are literals
        :deletes '( ld1 ld2 ... ldn ) ; where ldi's are literals
        :adds '( la1 la2 ... lan ) ; where lai's are literals
   )
)
```

4.4.1 Setting Other Parameters

In addition to the conditionals automatically generated and added as general knowledge inference rules as a result of defining and creating an entity type as described in Section 4.3, the designer can also manually add more inference rules to the predefined global variable ***general-knowledge***. To add a new inference rule, the designer should use the LISP line
($push$ ($list$ ϕ `'=>` ψ) $*general-knowledge*$)
where antecedent ϕ is either a single positive literal or a conjunction of positive literals (i.e., of the form (and ϕ_1 ϕ_2 ... ϕ_k)), consequent ψ is a positive literal, and `'=>` are three consecutive characters (namely, a single quote, an equal sign, and a greater-than sign). These inference rules play a pivotal role in the bounded forward inference performed by ME as explained in Section 3.2. This LISP line should appear in the LISP source file for the simulated world. The default depth of bounded forward inference is set at 2, but the designer can reset this by replacing the value in the LISP line **(defvar *inference-limit* ...)** in file `gridworld-planning.lisp`.

The projective lookahead search as detailed in Section 3.3 is bounded by a *search beam*, which specifies the number of lookahead levels (the length of the contemplated sequences of actions), the branching factor and the allowable operators at each level. The designer should set the search beam desired accordingly, by appropriate assignment to the global variables ***operators*** and ***search-beam*** in the LISP source file for the simulated world. For example, the following LISP lines set the search beam to a three-level lookahead, with branching factors 5, 4, and 3 for the first, second, and third levels, respectively, with the same set of allowable operators (referred to by their model version names) to consider at each level (though one can specify a different set of allowable operators to consider for each level of the lookahead):

- $(setq\ *operators*\ '(walk\ eat\ answer_ynq\ answer_whq\ sleep))$
- $(setq\ *search-beam*$
  $(list\ (cons\ 5\ *operators*)\ (cons\ 4\ *operators*)\ (cons\ 3\ *operators*)))$

Lastly, the designer should specify the rewards and costs of the various actions and potential states reached, as follows. First, the designer should prescribe the anticipated net utility of each model operator by setting its :*value* field accordingly; this value could either be a static numerical measure or be dependent on (i.e., be a function of) the specific input arguments that would be substituted into the formal parameters in instantiating the model operator into a contemplated action. Second, the designer should also assign values to specific properties that could be found in a state by modifying the contents of LISP routine
(defun state-value (additions deletions prior-local-value) ...)
in file `gridworld-planning.lisp`. This function essentially allocates positive points for addition of desirable properties to a state, and subtracts away these points for deletions of those properties; analogously for addition of undesirable

properties to, and removal of undesirable properties from a state. The header comments of this LISP routine give a clear example.

4.5 Interacting with ME

To interact with ME in a designer-specified simulated world, the user executes the following steps:

1. Open a terminal window and make sure to change directory into the one containing the LISP source files. Type and enter "*acl*" (without quotes) at the command prompt to start Allegro Common LISP (or whichever Common LISP-compatible interpreter is installed on the user's machine).

2. Load the four LISP source files in this order: `gridworld-definitions.lisp`, `gridworld-planning.lisp`, `simulation-and-function-evaluation.lisp`, and lastly the desired simulated world file (e.g., `world.lisp`).

3. Type and enter at the prompt $(initialize{-}state{-}node)$ to initialize the loaded simulated world.

4. Enter $(go!)$ to begin/continue ME's exploration, or $(listen!)$ to input wh- and yes/no-questions to ME. The user repeats this step until he or she wants to stop interacting with ME.

5. Type and enter $(exit)$ to exit Allegro Common LISP.

To experiment with question-answering in the framework, the user issues the command $(listen!)$ at the prompt and should become familiarized with the syntax for posing questions to ME. Let *lit* denote a literal of either the form $(arg_1\ pred\ arg_2\ ...\ arg_n)$ or the form $(not\ (arg_1\ pred\ arg_2\ ...\ arg_n))$, where arg_1

is the subject argument of the n-nary predicate relation *pred*. A wh-question to ME has the form ($ask{-}wh$ *lit*), and a yes/no-question to ME has the form ($ask{-}yn$ *lit*). After typing (*listen*!), if the user has no input to impart, then the user should simply enter () at the prompt and then hit the enter key. Otherwise, an example user input would be (($ask{-}yn$ (*AG is_bored*))
($ask{-}wh$ (*AG likes ?wh*))). Details and extensive examples are in Section 5.2.

(*listen*!) is not a command to ME, but to the executive routines running the agent. The system's response to the command, inviting a fact or question from the user, is also a mere reflex output not seen by ME. The effect on ME when the user subsequently supplies a fact or question is just the appearance of the new fact, or the user's desire for an answer to the question, in its knowledge base; its inferences and planning are affected accordingly.

Similarly, (*go*!) at the prompt is not a command to ME, but to the executive routines running the agent. These routines allow ME to consider its applicable actions in the current state and evaluate contemplated action sequences, select a/the seemingly best plan, and then perform the first action of the chosen plan to achieve effects and derive further inferences.

ME's dialogue interaction is handled uniformly via ME's planning and procedural attachment capabilities. Accordingly, the projective lookahead identifies a seemingly best action for ME to take at each step; thus, to answer a user question, ME must choose to do so as a/the seemingly most rewarding action. The designer can enhance ME's curiosity and helpfulness by making ME favor acquiring knowledge and answering questions as rewarding actions.

5 Experimental Agent and Results

We instantiated our agent ME in a particular simulated world, for the purpose of illustrating and experimenting with our framework. The sample world, although simple, suffices for substantiating the points we wish to make. After describing the design of this sample world in Section 5.1, we give a variety of examples of question-answering by ME to demonstrate its self-awareness and epistemic knowledge in Section 5.2.

Section 5.3 reports the empirical results of various runs of the agent in the simulated world. In one set of runs, the agent manifests its default, opportunistic yet future-directed behavior. In another set of runs, the agent is deprived of its ability to plan and reason ahead (or of its self-awareness, from another perspective). In a third experiment, the agent is rendered tenacious in sole pursuit of one specific consumption goal; this is effected by upward adjustment of the rewards it subjectively assigns to the chosen goal and to types of actions and states that lead to it, while zeroing out most other reward estimates. The latter two experiments show that ME's oblivion and failure to think ahead and exploit opportunities greatly reduce its total utility, in contrast with its default behavior. Our results affirm that ME benefits from its self-awareness, deliberate self-motivation, and opportunistic choices.

5.1 Simulated World

ME is situated in a simulated world simple yet sufficiently complex to illustrate its aforementioned distinctive features. The simulated world consists of five named locations, connecting roads, and both animate entities (agents) and inanimate entities (objects) at various locations. Figure 5.1 depicts the simulated world at initialization. Specifically, road *path*1 connects locations *home*, *school*, and *gym*; road *path*2, locations *home*, *plaza*, and *school*; road *path*3, locations *home*, *company*, and *school*. Additionally, each road segment is annotated with its length in units.

At the outset, object *pasta_ingredients*, of cost 2.0, object *pepperoni_pizza*, edible and of cost 6.0 by virtue of being of type *pizza*, and *apple_juice*, potable and of cost 2.0 by virtue of being of type *juice*, are all at *plaza*; agents ME and *guru*, both communicative by virtue of being agents, are at *home* and *school*, respectively; object *piano*, playable by virtue of being of type *music_instrument*, is at *home*; object *self_note*, readable by virtue of being of type *note*, is at *company*. Only predicates *contains*, *knows*, *is_edible*, and *is_potable* are designated as being occluded.

Initially, ME is penniless, has no swimming or cooking knowledge, is not tired, and has a hunger level of 4.0 and a thirst level of 2.0. In addition to knowledge of the road network and the existence of the object *piano* at *home* and its playability, ME knows whether *piano* is playable, the cost and location of *pasta_ingredients*, that *self_note* is readable and at *company*, and nothing more. *self_note* contains three pieces of knowledge, namely, that *guru* can talk, knows whether *apple_juice* is potable and whether *pepperoni_pizza* is edible; the latter two pieces of knowledge are attributed to *guru*.

The kinds of behavior we wish to enable in this simulated world include indulging in various pleasurable activities such as eating, drinking, cooking, swim-

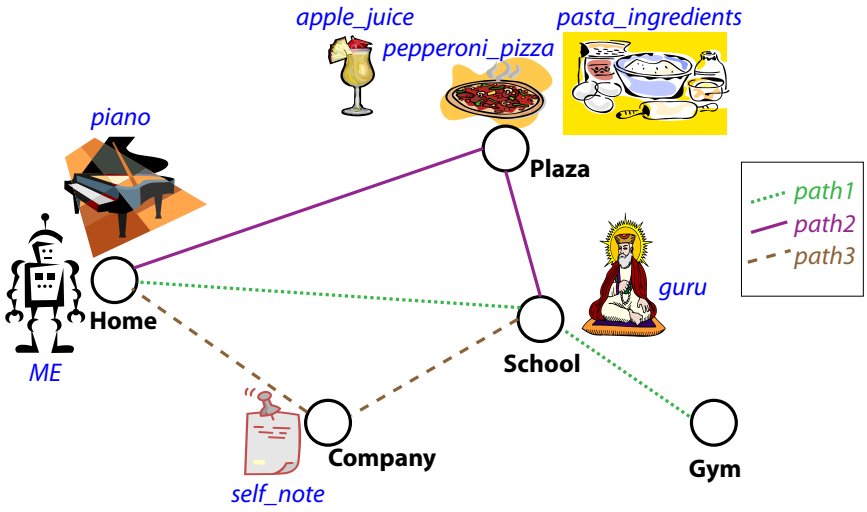


Figure 5.1: The Experimental World

ming, and playing the piano. But actions have preconditions; for example, eating and drinking require a consumable item to be at hand and knowledge about the item's edibility and potability, cooking requires cooking knowledge as well as suitable ingredients and being at home, and swimming requires swimming knowledge and being at the gym (assumed to include a pool). In turn, finding out about edibility and potability may require reading and/or consultation of the guru; obtaining ingredients for cooking may require working for money and buying them, and acquiring cooking and swimming skills by taking lessons may require working for money; we therefore enable all of these activities as well in ME's repertoire of operators.

In order to ensure that ME has a general disposition towards action, we do not limit its reward system so that only bodily indulgence is rewarding. Rather, ME reaps positive rewards from all of its activities, including answering user questions, working for money, and taking cooking and swimming lessons. The only contributors to negative utility are hunger, thirst, fatigue, boredom, and financial loss, but of course these are inevitable consequences of the passage of

time, of certain actions, or (in the last case) of purchases. Since in the stated initial conditions ME is penniless and can neither cook nor swim, we should expect to see behavior where ME works for money repeatedly, uses the earned money to take swimming and cooking lessons and buy ingredients, and eventually cooks meals, and of course partakes repeatedly of other rewarding activities – eating, drinking, sleeping, swimming, playing the piano, and answering user questions.

Specifically, ME has the following operators (where the anticipated rewards are indicated parenthetically) at its disposal: *walk* (5 - ME's fatigue level), *eat* (4 * ME's hunger level), *drink* (4 * ME's thirst level), *work_and_earn_money* (10), *buy* (7 - the cost of the purchase), *cook* (7 * ME's hunger level), *swim* (12), *read* (7 per piece of knowledge contained that ME does not know, and -7 per piece of knowledge contained that ME already knows), *play* (3), *answer_user_ynq* (10), *answer_user_whq* (10), *ask + whether* (5), *take_swimming_lesson* (10 - ME's fatigue level), *take_cooking_lesson* (10 - ME's fatigue level), and *sleep* (4 * ME's fatigue level).

Any state s reached by ME in its projective lookahead has an inherent value, calculated using the following criteria comparing s to its parent state. Acquiring knowledge or a possession earns ME 1 utility point; gaining swimming knowledge, cooking knowledge, or money yields the commensurate amount of increase in points; answering a user question adds 50 points; becoming less tired (hungry, thirsty) gains ME points equal to the decrease in ME's fatigue (hunger, thirst) level; becoming not bored contributes 1 point, as does becoming liked by some entity. Conversely, losing swimming knowledge, cooking knowledge, or money contributes the commensurate number of penalty points; leaving a user question unanswered subtracts 20 points; becoming more tired (hungrier, thirstier) costs ME points equal to the increase in ME's fatigue (hunger, thirst) level; becoming bored costs ME 1 point, as does becoming not liked. These criteria, together with the operators' anticipated rewards, indicate that ME favors cooking and

answering user questions the most.

Any potentially multi-step action is subject to possible interference by two types of exogenous events, namely fire and rain, although only fire can disrupt the action (e.g., traveling). A spontaneous fire has a 5% chance per time step of starting; once started, it has a 50% chance per time step of dying by itself, and it is extinguished as soon as there is rain. Spontaneous rain has a 33% chance of beginning; once it has begun, it has a 25% chance of stopping.

5.2 Question-Answering

ME's dialogue interaction is handled uniformly via *ME*'s planning and procedural attachment capabilities. Accordingly, the projective lookahead identifies a/the seemingly best action for *ME* to take at each step; thus, to answer a user question, *ME* must elect to do so as a/the seemingly most rewarding action. In support of our vision of *ME* as an inquisitive question-answering explorer, we induce its curiosity and helpfulness by making *ME* favor acquiring knowledge and answering questions as rewarding actions.

Using command (*listen*!), the user can interact with *ME* by issuing a yes/no or wh-question, or by imparting knowledge. After the user types (*listen*!) at the prompt, the system requests the user's input question(s) and/or knowledge. User input has a fixed format where an input question or knowledge item must be presented as a parenthesized expression of either the form (arg_1 *pred* arg_2 ... arg_n) or the form (*not* (arg_1 *pred* arg_2 ... arg_n)), with arg_1 denoting the subject argument and *pred* being an n-ary predicate relation. Furthermore, a yes/no question is prefixed with $ask{-}yn$; a wh-question, with $ask{-}wh$; a piece of knowledge, with *tell*. Although the user can enter more than one question or knowledge tidbit) at a time, *ME* will handle the input items piecewise and independently.

Among user inputs, knowledge tidbits are immediately processed, while it may

be some time until ME chooses to answer a user question as a/the seemingly next best action to perform. In a given state, ME may have incorrect or incomplete knowledge about the world when the knowledge pertains to nonlocal objects or occluded object properties. Consequently, the intended use of functionality *tell* is for the user to inform ME of facts in the simulated world which ME may not know. To prevent the user from tampering with the consistency of the simulated world, the system first checks the truth of any proffered knowledge tidbit p in the knowledge base of the simulated world (i.e., world KB). If and only if the check succeeds, it adds p to ME's KB, while removing its negation (if present), and then performs bounded forward inference. The check in the world KB uses a uniform closed world assumption (CWA), based on the presumption that the description of the simulated world is complete (a somewhat inaccurate presumption because of the bound on forward inference). Here we adopt the philosophical stance that when using *tell* to impart to ME facts in the simulated world, the user himself is in fact mentally subscribing to the simulated world and thus accepts it as *the* real world in which ME lives.

In Figure 5.2's example, the user conveys three knowledge tidbits, only the first of which is true in the world KB. The system displays informative warnings immediately after processing the tidbits.

```
>> (listen!)
You're welcome to ask ME a question or tell him a fact.
((tell (apple_juice is_at plaza))
 (tell (not (guru is_at school)))
 (tell (ice is_edible)))

The negation of your assertion (NOT (IS_AT GURU SCHOOL)) is
true in the simulated world so your assertion is not added to avoid
introducing inconsistency.

Your assertion (IS_EDIBLE ICE) has an unknown truth value in the
simulated world so it is not added under the closed world assumption.
```

Figure 5.2: Handling *tell* Tidbits

Given a yes/no question of the form $(ask{-}yn\ q)$, ME must consider two cases. If ME knows either q or $\neg q$ to be true, then ME will impart that knowledge as an English sentence. On the other hand, if ME knows neither q nor $\neg q$ to be true, then it must be that q is about a nonlocal entity or about an occluded property of a local non-ME entity, and ME informs the user accordingly. The example in Figure 5.3 illustrates ME's responses for the two cases of yes/no questions.

```
>> (listen!)
You're welcome to ask ME a question or tell him a fact.
((ask-yn (not (piano is_playable)))
 (ask-yn (apple_juice is_tasty)))

>> (go!)
STEP TAKEN: (ANSWER_USER_YNQ (IS_TASTY APPLE_JUICE))
ANSWER: ME DOES NOT KNOW WHETHER APPLE JUICE IS TASTY.
      For the question (IS_TASTY APPLE_JUICE), ME does not currently
      know the answer as it may be about an occluded property of a local
      object or about a non-local object.

>> (go!)
STEP TAKEN: (ANSWER_USER_YNQ (NOT (IS_PLAYABLE PIANO)))
ANSWER: PIANO IS PLAYABLE
      For the question (NOT (IS_PLAYABLE PIANO)), according to ME's
      current knowledge base, ME offers the answer above.
```

Figure 5.3: Answering yes/no Questions

A wh-question of the form $(ask{-}wh\ r)$ must have at least one variable (which is indicated by prefix ?) in r. For instance, $(is_at\ ?x\ ?y)$ translates to "where is every entity located?" while $(not\ (?x\ is_bored))$ corresponds to the question "who is not bored?". In computing the answer(s) to r, ME attempts to unify r with facts in ME's current knowledge base; for each set s of bindings found for the variables in r, ME formulates the corresponding answer by replacing the variables in r with bindings in s. As Figure 5.4 illustrates, ME uses the restricted CWA (see Section 3.4) in verbalizing its responses.

Figure 5.5, including $knows{-}whether$ and $knows{-}that$ questions, is a concatenation of several dialogue exchanges between ME and the user, showing only

```
>> (listen!)
You're welcome to ask ME a question or tell him a fact.
((ask-wh (?x is_tasty))
 (ask-wh (not (ME is_hungry_to_degree ?y)))
 (ask-wh (?z is_at home)))

>> (go!)
STEP TAKEN: (ANSWER_USER_WHQ (NOT
                        (IS_HUNGRY_TO_DEGREE ME ?Y)))
ANSWER: ME IS HUNGRY TO DEGREE 4.0.
    For the question (NOT (IS_HUNGRY_TO_DEGREE ME ?Y)), other
    than the above positive instance(s) that ME knows of, ME assumes
    everything else as the answer.

>> (go!)
STEP TAKEN: (ANSWER_USER_WHQ (IS_TASTY ?X))
ANSWER: ME DOES NOT KNOW WHETHER ANYTHING-non-ME IS TASTY.
    For the question (IS_TASTY ?X), there are no positive instances that
    ME knows of, so ME assumes nothing as the answer.

>> (go!)
STEP TO TAKEN: (ANSWER_USER_WHQ (IS_AT ?Z HOME))
ANSWER: ME IS AT HOME.
            PIANO IS AT HOME.
    For the question (IS_AT ?Z HOME), other than the above positive
    instance(s) that ME knows of, ME assumes nothing else as the answer.
```

Figure 5.4: Answering *wh* Questions

ME's actions to answer user questions and *ME*'s corresponding verbalized English answers (while omitting system output). For clarity, we further annotate some of the action-answer pairs with justifications for the answers. These examples demonstrate *ME*'s ability to introspect positively and negatively using the restricted CWA; see also Section 3.4.

```
>> (go!)
STEP TAKEN: (ANSWER_USER_YNQ
              (KNOWS GURU (WHETHER (LIKES ME PASTA))))
ANSWER: ME DOES NOT KNOW WHETHER GURU KNOWS WHETHER ME LIKES PASTA.
//Reason: Guru's knowledge is still occluded from ME since ME has not acquired
//such information about Guru.

>> (go!)
STEP TAKEN: (ANSWER_USER_YNQ (GURU KNOWS TWEETY))))
ANSWER: ME DOES NOT KNOW WHETHER GURU KNOWS TWEETY.
//Reason: Guru's knowledge is still occluded from ME since ME has not acquired
//such information about Guru.

>> (go!)
STEP TAKEN: (ANSWER_USER_YNQ
              (KNOWS GURU (WHETHER (KNOWS GURU TWEETY))))
ANSWER: GURU KNOWS WHETHER GURU KNOWS TWEETY.
//Reason: Although ME does not know whether Guru likes Tweety, Guru must know
//this about itself by simulative inference.

>> (go!)
STEP TAKEN: (ANSWER_USER_YNQ (CAN_DANCE GURU))
ANSWER: IT IS NOT THE CASE THAT GURU CAN DANCE.
//Reason: ME knows the answer since ME has visited Guru and Guru's dancing
//ability is not an occluded property.

>> (go!)
STEP TAKEN: (ANSWER_USER_WHQ (IS_AT ?X ?Y))
ANSWER: ME IS AT GYM.
      GURU IS AT SCHOOL.
      SELF NOTE IS AT COMPANY.
      PASTA INGREDIENTS IS AT PLAZA.
      PIANO IS AT HOME.
For the question (IS_AT ?X ?Y), other than the above positive instance(s) that ME
knows of, ME assumes nothing else as the answer.

>> (go!)
STEP TAKEN: (ANSWER_USER_YNQ (CAN_DANCE ME))
ANSWER: IT IS NOT THE CASE THAT ME CAN DANCE.

>> (go!)
STEP TAKEN: (ANSWER_USER_YNQ (NOT (IS_NAVIGABLE PATH3)))
ANSWER: PATH3 IS NAVIGABLE.

>> (go!)
STEP TAKEN: (ANSWER_USER_YNQ
              (KNOWS GURU (THAT (IS_NAVIGABLE PATH3))))
ANSWER: ME DOES NOT KNOW WHETHER GURU KNOWS THAT PATH3 IS NAVIGABLE.
//Reason: Even though ME indeed knows that path3 is navigable, since ME has not
//learned of Guru's knowledge about the navigability of path3, this particular
//piece of Guru's knowledge is still occluded from ME.

>> (go!)
STEP TAKEN: (ANSWER_USER_YNQ (NOT (IS_BORED ME)))
ANSWER: IT IS NOT THE CASE THAT ME IS BORED.

>> (go!)
STEP TAKEN: (ANSWER_USER_YNQ (KNOWS GURU (THAT (IS_BORED ME))))
ANSWER: IT IS NOT THE CASE THAT GURU KNOWS THAT ME IS BORED.
```

Figure 5.5: Answering Questions Using Introspection

5.3 Results

We present empirical results of three distinct cases illustrating the advantages of *ME*'s self-awareness and opportunistic behavior, resulting from its self-motivated, deliberate, continual planning, in the context of the simulated world described in Section 5.1. The first case is *ME*'s default, self-aware and opportunistic behavior; the second case, its unthinking and non-self-aware behavior by ablating its projective lookahead; the third case, its purely goal-directed behavior by ablating its opportunistic tendencies.

With the exception of the purely goal-directed (non-opportunistic) case, each case includes ten independent runs of 40 *steps* apiece, where at each *step* *ME* chooses and executes a/the next best action. The lookahead used in each case is three levels (i.e., three steps into the future from the current state) and has branching factors of 5, 4, 3 at the three successive levels, respectively. Variable behaviors could result from random occurrences of fire and rain, or from randomly and with uniform probability selecting a best action (among equally attractive best action candidates) to perform at a step. Therefore, we show the average in comparing these three different cases.

5.3.1 Opportunistic Behavior

We report experiments in which *ME*'s exhibits (default) opportunistic and foresighted behavior with all its faculties intact. The success of a run is measured in terms of the net utility (NU), accumulated over the entire sequence of actions and corresponding states attained. In subsequent Sections 5.3.2 and 5.3.3, we then describe ablation experiments and investigate *ME*'s performance when its self-awareness and deliberate planning are supplanted by unthinking search, and when its opportunistic inclinations are replaced by single-minded pursuit of a sole beneficial yet distant goal.

For the opportunistic case, we conducted ten independent runs of ME's opportunistic behavior, using a three-level lookahead with branching factors of 5, 4, and 3 for the three successive levels. Each run was for 40 steps. ME achieved an NU of 1260.85 averaged over the ten runs ranging from 1204.5 to 1389.0.

As an example, Figure 5.6 shows ME's sequence of actions for one of the ten runs, and ME's NU was 1288.0. Each action, which is the inner parenthesized expression on a line, was logged with a number indicating which iteration of this action was listed, its anticipated duration, and the time in the simulated world when that iteration began. An inner parenthesized expression represents an action performed by ME, and it consists of the action name as well as its specific input arguments. To economize on space required for enumerating all iterations, for a multi-step action (i.e., one that ran for more than one iteration), we only enumerate its first iteration and its final iteration on the same line; i.e., each multi-step action below, such as ($WORK_AND_EARN_MONEY$ 4.0 0.0 1.0) on the 2nd line running for five iterations, has exactly one pair of listings on the 2nd line. The exceptions are the three $TAKE_SWIMMING_LESSON$ multi-step actions, whose pairs of first and final iterations are too long to list on one line. On the other hand, a one-step action below, such as ($READ$ 6.5 $SELF_NOTE$ $COMPANY$) on the 3rd line, has exactly one listing.

We reiterate that ME's chosen seemingly best action alone in itself may not seem or prove immediately rewarding to ME; however, it is chosen because it constitutes the first action in a/the most rewarding sequence of actions that ME anticipates and foresees as executable and attainable from ME's current state. For instance, although ME might not receive a reward for walking from *home* to *plaza*, but ME may very well foresee a substantial reward resulting from traveling to *plaza* to buy *pasta_ingredients*, taking *pasta_ingredients* back *home*, cooking with *pasta_ingredients* to produce *pasta*, and ultimately eating the cooked *pasta*. Therefore, the use of a reasoned projective lookahead enables ME to exhibit both

```
((WALK HOME COMPANY PATH3 0.0) 1 2 0)          ((WALK HOME COMPANY PATH3 0.0) 2 2 1)
((WORK_AND_EARN_MONEY 4.0 0.0 1.0) 1 5 3)      ((WORK_AND_EARN_MONEY 4.0 0.0 1.0) 5 5 7)
((READ 6.5 SELF_NOTE COMPANY) 1 1 9)
((WALK COMPANY SCHOOL PATH3 6.0) 1 3 11)       ((WALK COMPANY SCHOOL PATH3 6.0) 3 3 13)
((ASK+WHETHER GURU ((IS_POTABLE APPLE_JUICE) SCHOOL) 1 1 15)
((TAKE_COOKING_LESSON 0.0 7.5) 1 4 17)         ((TAKE_COOKING_LESSON 0.0 7.5) 4 4 20)
((TAKE_COOKING_LESSON 4.0 9.5) 1 4 22)         ((TAKE_COOKING_LESSON 4.0 9.5) 4 4 25)
((TAKE_COOKING_LESSON 8.0 11.5) 1 4 27)        ((TAKE_COOKING_LESSON 8.0 11.5) 4 4 30)
((TAKE_COOKING_LESSON 12.0 13.5) 1 4 32)       ((TAKE_COOKING_LESSON 12.0 13.5) 4 4 35)
((TAKE_COOKING_LESSON 16.0 15.5) 1 4 37)       ((TAKE_COOKING_LESSON 16.0 15.5) 4 4 40)
((ASK+WHETHER GURU ((IS_EDIBLE PEPPERONI_PIZZA) SCHOOL) 1 1 42)
((WALK SCHOOL HOME PATH1 17.5) 2 2 45)         ((WALK SCHOOL HOME PATH1 17.5) 1 2 44)
((PLAY 2.0 18.5 PIANO HOME) 1 1 47)
((SLEEP 19.0 7.0) 1 38.0 49)                   ((SLEEP 19.0 7.0) 38 38.0 86)
((WALK HOME SCHOOL PATH1 0.0) 1 2 88)          ((WALK HOME SCHOOL PATH1 0.0) 2 2 89)
((WALK SCHOOL GYM PATH1 1.0) 1 1 91)
((TAKE_SWIMMING_LESSON 16.5 0.0 1.5 2.5) 1 3 93)
                                               ((TAKE_SWIMMING_LESSON 16.5 0.0 1.5 2.5) 3 3 95)
((TAKE_SWIMMING_LESSON 18.0 6.0 4.5 3.5) 1 3 97)
                                               ((TAKE_SWIMMING_LESSON 18.0 6.0 4.5 3.5) 3 3 99)
((TAKE_SWIMMING_LESSON 19.5 12.0 7.5 5.0) 1 3 101)
                                               ((TAKE_SWIMMING_LESSON 19.5 12.0 7.5 5.0) 3 3 103)
((WALK GYM SCHOOL PATH1 10.5) 1 1 105)
((WALK SCHOOL GYM PATH1 11.0) 1 1 107)
((WALK GYM SCHOOL PATH1 11.5) 1 1 109)
((WALK SCHOOL HOME PATH1 12.0) 1 2 111)          ((WALK SCHOOL HOME PATH1 12.0) 2 2 112)
((WALK HOME PLAZA PATH2 13.0) 1 2 114)           ((WALK HOME PLAZA PATH2 13.0) 2 2 115)
((BUY 15.0 PASTA_INGREDIENTS PLAZA 2.0) 1 1 117)
((WALK PLAZA HOME PATH2 14.0) 1 2 119)           ((WALK PLAZA HOME PATH2 14.0) 2 2 120)
((COOK 21.0 20.0 15.0)  1 1 122)
((EAT 21.0 PASTA) 1 1 124)
((SLEEP 16.0 0.0) 1 32.0 126)                    ((SLEEP 16.0 0.0) 32 32.0 157)
((WALK HOME PLAZA PATH2 0.0) 1 2 159)            ((WALK HOME PLAZA PATH2 0.0) 2 2 160)
((BUY 13.0 PASTA_INGREDIENTS PLAZA 2.0) 1 1 162)
((WALK PLAZA HOME PATH2 1.0) 1 2 164)            ((WALK PLAZA HOME PATH2 1.0) 2 2 165)
((COOK 8.0 20.0 2.0) 1 1 167)
((EAT 8.0 PASTA) 1 1 169)
((WALK HOME PLAZA PATH2 3.0) 1 2 171)            ((WALK HOME PLAZA PATH2 3.0) 2 2 172)
((BUY 11.0 APPLE_JUICE PLAZA 2.0) 1 1 174)
((DRINK 6.5 APPLE_JUICE) 1 1 176)
((WALK PLAZA HOME PATH2 4.0) 1 2 178)            ((WALK PLAZA HOME PATH2 4.0) 2 2 179)
((WALK HOME COMPANY PATH3 5.0) 1 2 181)          ((WALK HOME COMPANY PATH3 5.0) 2 2 182)
((WORK_AND_EARN_MONEY 0.0 9.0 6.0) 1 5 184)      ((WORK_AND_EARN_MONEY 0.0 9.0 6.0) 5 5 188)
```

Figure 5.6: Example of an Opportunistic Run

foresight and opportunism in its behavior.

The relatively high NU of opportunistic behavior results from ME's better choices of actions; rather than blindly pursuing any one goal, ME seizes various favorable opportunities encountered while exploring the simulated world. Specifically, the opportunities ME seized include working to earn money, sleeping to

relieve fatigue, playing *piano* to assuage boredom (an effect of reading, though ME is not initially bored), taking cooking lessons to learn to cook, taking swimming lessons to learn to swim, reading $self_note$ to discover that *guru* is communicative and knowledgeable about $apple_juice$'s potability and $pepperoni_pizza$'s edibility, asking *guru* in order to acquire such knowledge, buying $pasta_ingredients$ with which to cook, cooking *pasta*, eating *pasta* to sate its hunger, and drinking $apple_juice$ to quench its thirst.

5.3.2 Unthinking, Non-Self-Aware Behavior

To simulate unthinking, non-self-aware behavior, we suppressed ME's projective lookahead and instead made ME randomly choose applicable actions (with uniform probability) in any given state, without consideration of consequences or anticipated utility. Certainly the resulting behavior should show the disadvantages of abandoning ME's default reasoned lookahead. But we can also interpret the resulting behavior as showing the consequences of abandoning self-awareness. After all, much of ME's self-awareness is comprised of ME's knowledge about its own situation and about its action preconditions and effects, and its anticipation of the situations it may find itself in when performing a sequence of actions, and of the rewards and penalties associated with those actions and situations.

The average NU over these ten runs of 40 steps apiece is -627.65, and ME's behavior was extremely haphazard, yielding greatly varied NUs ranging from -1465.0 to -56.0. The average NU here, though unreliable, is significantly lower than in the opportunistic, reasoned, self-aware case in Section 5.3.1. Perhaps more importantly, the very low NU values obtained in some cases show that the unthinking behavior is extremely risky. These results are unlikely to surprise the reader, but they are not a foregone conclusion: one can certainly imagine environments in which random behavior is as good as any other. But it is precisely in worlds where certain formally statable regularities constrain the situations en-

countered by an agent and the effects produced by its actions, that thinking and self-awareness are beneficial.

5.3.3 Goal-Directed Behavior

To simulate non-opportunistic behavior, we made ME purely goal-directed by skewing its "subjective" metrics of rewards and penalties for actions and states in the lookahead calculation. (But we evaluated actual rewards, and thus actual NUs, just as in the opportunistic case.) Specifically, we designated eating ME-cooked *pasta* as ME's sole goal and made ME uninterested in actions other than those essential to achieving that goal: taking cooking lessons to learn to cook, working to earn money with which to later buy *pasta_ingredients*, buying *pasta_ingredients* with which to cook *pasta*, cooking to produce *pasta*, traveling to enable performing these actions, and, of course, eating *pasta*. This was accomplished by (a) setting the anticipated rewards of *eat*, *take_cooking_lesson*, *buy*, *cook*, and *work_and_earn_money* each to 50, while nullifying the anticipated rewards of all others; and by (b) rewarding the acquisition of cooking knowledge, money, *pasta_ingredients*, and *pasta* in a state reached, rewarding the consumption of *pasta* or *pasta_ingredients* in a state reached (i.e., the removal of either from a state reached), rewarding the decrease in hunger in a state reached, and penalizing the increase in hunger hunger in a state reached, while ignoring all other changes in a state reached.

The reader may wonder why we assigned positive values to actions and states that are needed to reach the goal of eating pasta, rather than simply rewarding the latter. The answer is that nontrivial problem-solving in combinatorially complex domains is impractical without heuristics. Most practical planners use heuristics that either estimate the distance to the goal, or (inversely) measure the similarity of a given state to the goal state. For regression planners, or mixed forward-backward planners, guidance can also be provided in the form of user-supplied or

automatically inferred invariants (logical relationships that hold in every reachable state), allowing avoidance of inconsistent states when reasoning backward from later states to earlier states. The method we have employed here is particularly simple, since it merely biases the agent towards types of actions likely to lead to the goal. This is sufficient to solve the pasta-eating problem, even though the solution is 14-25 steps long (e.g., involving multiple cooking lessons). The reason that such a simple strategy suffices here is that the effects of the actions needed to achieve the goal are generally persistent: Once ME has gained some cooking knowledge, this knowledge is not lost; once ME has pasta ingredients, this remains true till the ingredients are used to make pasta; and once ME has earned money from working, this remains available until ME has used it for cooking lessons or buying ingredients (assuming that taking swimming lessons has been made uninteresting). In many classical planning domains, gains made towards the goal are easily lost again when further steps are taken; as will be seen in Section 6.1, such domains require stronger heuristics than we employ here.

Given the goal of eating self-cooked *pasta*, ME's sequence of at least 14 actions was generally found with few missteps. The exceptions occurred in states where there were two disparate routes either of which ME could take to reach a desired new location from the ME's current location, and ME would randomly (with uniform probability) select one of the routes rather than taking into account the greater fatigue level of a longer route. Fatigue relative to distance traveled was purposely discounted in the goal-directed case. Because of the nondeterministic route choices in some states, six of the ten runs had an actual NU of 197.0, and the remainder had an actual value of 187.0; therefore, the average actual NU was 193.0. Each actual NU was calculated using ME's original metrics of rewards and penalties for actions and states reached.

Figure 5.7 shows a run that yielded an actual NU of 187.0.

The rationale behind ME's sequence of actions to reach the goal is eas-

```
((WALK HOME SCHOOL PATH1 0.0) 1 2 0)                    ((WALK HOME SCHOOL PATH1 0.0) 2 2 1)
((TAKE_COOKING_LESSON 0.0 1.0) 1 4 3)                   ((TAKE_COOKING_LESSON 0.0 1.0) 4 4 6)
((TAKE_COOKING_LESSON 4.0 3.0) 1 4 8)                   ((TAKE_COOKING_LESSON 4.0 3.0) 4 4 11)
((TAKE_COOKING_LESSON 8.0 5.0) 1 4 13)                  ((TAKE_COOKING_LESSON 8.0 5.0) 4 4 16)
((TAKE_COOKING_LESSON 12.0 7.0) 1 4 18)                 ((TAKE_COOKING_LESSON 12.0 7.0) 4 4 21)
((TAKE_COOKING_LESSON 16.0 9.0) 1 4 23)                 ((TAKE_COOKING_LESSON 16.0 9.0) 4 4 26)
((WALK SCHOOL COMPANY PATH3 11.0) 1 3 28)               ((WALK SCHOOL COMPANY PATH3 11.0) 3 3 30)
((WORK_AND_EARN_MONEY 4.0 0.0 12.5) 1 5 32)             ((WORK_AND_EARN_MONEY 4.0 0.0 12.5) 5 5 36)
((WALK COMPANY SCHOOL PATH3 17.5) 1 3 38)               ((WALK COMPANY SCHOOL PATH3 17.5) 3 3 40)
((WALK SCHOOL PLAZA PATH2 19.0) 1 4 42)                 ((WALK SCHOOL PLAZA PATH2 19.0)  4 4 45)
((BUY 15.0 PASTA_INGREDIENTS PLAZA 2.0) 1 1 47)
((WALK PLAZA HOME PATH2 21.0) 1 2 49)                   ((WALK PLAZA HOME PATH2 21.0) 2 2 50)
((COOK 6.5 20.0 22.0)  1 1 52)
((EAT 6.5 PASTA)  1 1 54)
```

Figure 5.7: Example of a Goal-Directed Run

ily understood by considering them anti-chronologically. To eat ME-cooked *pasta*, ME must cook to produce *pasta*. Since cooking *pasta* required the use of *pasta_ingredients*, which ME did not initially possess, ME must travel to *plaza*, the initial location of *pasta_ingredients*, and buy *pasta_ingredients* there. To be able to afford *pasta_ingredients*, the initially penniless ME must travel to *company* and work to earn money. Additionally, to be able to cook *pasta* with initially no cooking knowledge at all, ME must travel to *school* to take sufficiently many cooking lessons.

The average NU of 193.0 in this case is considerably lower than it was in ME's opportunistic behavior in Section 5.3.1. As enumerated at the end of that section, instead of doggedly pursuing the sole goal of eating *pasta*, ME was able to see and exploit *additional* opportunities, such as sleeping to relieve fatigue, playing *piano* to assuage boredom, taking swimming lessons to learn to swim, gaining knowledge from reading and from *guru*, and eating and drinking foods other than *pasta*. The results establish that ME benefits from awareness and exploitation of opportunities for immediate rewards, and not just distant ones.

6 Related Work

In addition to seamlessly integrating logical planning and reasoning into self-motivation, our self-motivated cognitive agent framework combines situated, reactive behavior, basic dialogue capabilities, continuous planning for utility optimization, with classical planning (including first-order reasoning, extended to allow reasoning about attitudes). Consequently, in the following six sections, we relate our framework to classical planning, to continuous planning, to cognitive robots and behavioral agents, and to previous agent systems that combine planning, reasoning and dialogue to some degree.

In Sections 6.1 and 6.2, we discuss how we successfully modeled and solved such classical STRIPS-like planning problems as the Towers of Hanoi and a Logistics domain, and more recent continuous planning problems such as a Colorballs domain and a Multiagent traveling domain in incompletely known worlds. In the Colorballs domain, our framework performed substantially better than some state-of-the-art contingent planners that construct full contingent plans, even finishing problems that they failed to solve. Our empirical results affirm the generality and flexibility of our framework. In conjunction with our Multiagent traveling domain experiments, we describe the multiagent extensions we have made to our framework to enable multiple, coexisting self-motivated cognitive agents. Lastly in Section 6.6, we compare and contrast our framework with some general archi-

tectural proposals for autonomy and self-awareness.

6.1 Classical Planning and Our Results

Classical planners (e.g., (Weld, 1999)) are purely goal-directed, aiming all their actions at the attainment of some future goal state. Or, in the case of regression planners (e.g., (Penberthy and Weld, 1992)), they search backward from the goal state in order to find a course of action that leads to the goal state from the initial state. These strictly goal-directed planners possess no notion of intrinsically rewarding actions or situations that could be exploited along the way to the goal; admittedly, they may attempt to minimize their cost of reaching the goal state. Also, they are prone to assume that the planner's knowledge of the search space to be explored is sufficiently complete in all the relevant respects to enable advance "mental" construction of a complete plan that provably leads to the goal state. Even in probabilistic planning (e.g., (Kushmerick et al., 1995)), it is typically assumed that alternative outcomes of actions and certain exogenous events can be exhaustively enumerated and assigned probabilities in advance.

This differs considerably from planning in real life, where unanticipated events and situations can and do arise regularly, unveiling new opportunities for rewards or new risks or costs. These phenomena are especially evident in dialogues, where an interlocutor's utterances are mostly unpredictable (even probabilistically). These utterances can present opportunities for reciprocal speech acts which may produce short-term rewards or longer-term benefits. For instance, the former can include satisfaction at being able to provide or elicit useful information; the latter, earning others' good will. This is precisely the reason that allowing for opportunistic responses to situations encountered in a simulated world is an essential desideratum in the design of our self-motivated cognitive agent framework.

Nonetheless, it is relevant to ascertain whether our framework can successfully

tackle classical planning problems. The reason is that the ability to model and solve classical planning problems would provide evidence for the versatility of our framework. Therefore, we undertook experiments in the Towers of Hanoi domain, a classical Logistics domain involving parcel delivery by truck and plane.

6.1.1 Towers of Hanoi

We already illustrated goal-directed behavior by ME in a fairly complex simulated world in Section 5.3.3. However, as noted, overcoming the combinatorics of the search space in that case was facilitated by the fact that the effects of the actions needed to achieve the goal of pasta-eating were generally persistent. This is not the case in Towers of Hanoi or in the subsequent Logistics world, where inept moves can easily undo gains made previously. Consequently, we needed to apply stronger heuristics in Towers of Hanoi and in the subsequent Logistics world, at least for versions of the problems requiring more than four of five steps. The natural way to encode these problems in our framework is to restrict the world to a single place, at which the agent performs all the actions needed to solve the problems; it is as though the agent were performing paper-and-pencil problem solving at a fixed locale. A pivotal issue requiring deeper thought is how to incorporate search heuristics into our agent's view of the world, which is after all reward-based, not directly goal-driven.

We implemented the 3-disk and the 4-disk versions of the well-known Towers of Hanoi puzzle, knowing well that the number of requisite actions in the optimal solution is given by the formula $2^d - 1$ for any d-disk version. A weak yet sufficient heuristic we adopted looked solely at the positions of disks on peg 3 in any given state, rewarding each disk that is in its final place (i.e., correct in the sense of both being directly under the next smaller-sized disk if applicable and being directly atop the next bigger-sized disk if applicable) in the stack of disks on peg 3, and symmetrically punishing undoing a disk that is already in its final place in the

stack of disks on peg 3. The rewards and symmetric penalties are proportional to the size of the disk. The only available actions were a 0-utility *move* action and a 1-utility *do−nothing* action, with the latter being applicable only when all disks have been correctly placed on peg 3.

For the 3-disk version of the well-known Towers of Hanoi puzzle, we instituted a 4-step deep search horizon with consideration of 3 seemingly best actions at each depth, and our agent invariably reached the goal state in the optimal 7 steps, without any missteps and taking only about 0.31 seconds (averaged over 20 runs). Using a 3-step deep search horizon could not guarantee the optimal 7-step solution in all runs for the 3-disk version.

For the 4-disk version, using an 8-step deep search horizon with the best 3-action breadth at each depth, our agent always reached the goal state in the optimal 15 steps, without any missteps and taking about 55.35 seconds (averaged over 20 runs). A 7-step deep search horizon, combined with our heuristic, could not always guarantee the optimal 15-step solution for the 4-disk version. The average 55.35 seconds for 4-disk seemed rather slow, but understandably so, since the forward search tree at each step contained up to 9840 states, implying rampant duplication of the 81 distinct states in this puzzle.

The 5-disk version of the puzzle was not solvable with the above weak heuristic and either an 8-step or a 9-step search horizon. With a 10-step search horizon, the execution unfortunately ran out of memory and was halted.

We could certainly have obtained faster solutions by avoiding state duplication in the forward search. However, our goal was merely to demonstrate that our agent framework is flexible enough to allow representation and solution of classical planning problems, rather than to match the performance of state-of-the-art algorithms such as Graphplan (Blum and Furst, 1995), SATPLAN (Kautz and Selman, 1992), and FF (Hoffmann and Nebel, 2001). We also note that the avoidance of state duplication is more of an issue in solving combinatorial puzzles

than in guiding the behavior of a reward-seeking agent in a world where rewards are distributed over many states. For example, the optimal behavior for an agent similar to the one in our cooking world may well be to loop indefinitely through a sequence of actions and states where the means for earning rewards are repeatedly acquired, deployed, and depleted.

6.1.2 A Logistics World

In the Logistics world, there are two cities, each containing two locations, namely an airport and a post office. There is an airplane that commutes between the two cities and is located initially at either of the two airports, and in each city there are one or more trucks at the airport or the post office. The goal is to deliver one or more parcels, located initially at one or another of the four locations, to designated locations. As in our representation of the Towers of Hanoi puzzle, we restricted the world to a single place, at which the agent performs all actions needed to solve the logistics problem at hand.

We experimented with problems requiring from 3 to 13 steps. Our approach in this case was to use a uniform A*-like heuristic, (under)estimating the remaining number of steps. Specifically, if a parcel is in the wrong city, there will have to be a flight (and the accompanying loading onto and loading off the plane) to the destination city; if a parcel is at the wrong place in a city, there will have to be at least one truck-driving step (and the accompanying loading onto and unloading from the truck) to transfer parcel, and possibly one for the truck to travel to the parcel's pickup location; the accompanying loading steps (onto the plane or a truck) and unloading steps (from the plane or a truck) can similarly be estimated.

The remaining issue is then to incorporate the heuristic into the agent's reward system. We found that an effective reward strategy was to make dual use of the heuristic function. In any state, the agent receives a negative reward for any action

undertaken that fails to reduce the estimated distance to the goal state. Seemingly helpful actions carry a utility of 0, and a 0-utility *do−nothing* action is allowed to prevent pointless vehicle movements once the goal is achieved. Precisely, we assigned a negative reward to each state, equal to -1 times the heuristic estimate of the remaining number of steps needed to complete parcel delivery; and we assigned a cost of 0 to each action that appeared to reduce the distance to the goal, according to the heuristic function, and a cost of 1 (reward value of -1) otherwise, except for the cost-free *do−nothing* action. The negative value of unproductive actions ensures that once parcel delivery is complete, useless actions such as moving the (empty) trucks or plane back and forth would be dispreferred compared to the *do−nothing* action.

With this setup and compiled LISP code, employing a 2-step search horizon, our agent solved each of five Logistics problems requiring 3, 6, 9, 10, and 13 steps in under 0.4 seconds (0.04, 0.08, 0.12, 0.13, and 0.39 seconds, respectively) on a standard desktop computer.

6.2 Continuous Planning and Our Results

Our framework shares an affinity with conformant and contingent planners (e.g., (Baral and Son, 1997; Bonet and Geffner, 2000; Bryce et al., 2006)) in looking beyond the flawed presupposition made by most classical planners of possessing correct and complete world information. In a realistic, dynamic environment, a planning agent may have incorrect or incomplete information about the world state, exogenous events, or its or other agents' actions. In recent years, interests have surged in continual planning, which interleaves planning, execution and monitoring in a dynamic, and only partially observable, multiagent environment.

We undertook experiments in a Colorballs-n-x domain where the agent is to transport each of the x balls from its initial location to its destination location

based on the ball's color in the nxn grid, and a Multiagent-n-x-b traveling domain where all x agents are to navigate from their respective initial locations to their respective goal locations in the nxn grid with b obstacles. Our highly competitive empirical results in this section show that our framework can hold its own against some state-of-the-art contingent planners and recent continuous multiagent planners in incompletely known worlds, further showcasing the generality and flexibility of our framework.

6.2.1 The Colorballs-n-x Problem

Conformant and contingent planners emerged to tackle planning in the presence of incomplete information, namely, uncertainty about the initial state or action effects. While a conformant planner is not equipped with sensing capabilities during plan execution, a contingent planner senses the value of certain world aspects during plan execution and branches contingent upon the observed value. Accordingly, conformant and contingent planning has also been formulated as a path-finding problem in the space of belief states, i.e., the space whose elements are sets of possible world states; the planner looks for a sequence of actions which maps a given initial belief state into a target belief state.

Particularly challenging are the tasks of obtaining effective heuristics over beliefs and representing beliefs compactly. The former challenge has not been fully conquered, as heuristics for searching in belief space are still less successful than heuristics developed for classical planning. The latter challenge has been addressed by SAT or ordered binary decision diagram-based logical representations; though intractable in the worst case, they still scale up better than plain state representations.

To overcome the two aforementioned challenges, Palacios and Geffner (2007) introduced a translation-based approach to automatically convert conformant

planning problems into classical planning problems to then be solved by a classical planner, although their formulation is limited to conformant problems with deterministic actions and where all uncertainty lies in the initial situation only. Not only does this translation-based approach obviate the need for an explicit search in belief state space, but a classical heuristic over beliefs can be used as belief states are now represented as plain states. The solutions computed by a classical planner encode the conformant solutions.

Albore et al. (2009) extended the approach of (Palacios and Geffner, 2007) to translate a contingent planning problem P into a classical planning problem $X^+(P)$, although assuming again that P involves uncertainty in the initial state only and that all actions are deterministic. P, a non-deterministic problem in belief space (i.e., where beliefs are represented by sets of states), is compiled into $X(P)$, a non-deterministic fully observable problem in state space (i.e., where states are represented by sets of literals representing the beliefs over P). Although the compilation is typically linear in the number of possible initial states which is exponential in the number of fluents, a polynomial translation $X(P)$ can be obtained for P with *bounded contingent width* (Albore et al., 2009). One then obtains the relaxation $X^+(P)$ of $X(P)$ where *deletes*, *preconditions*, and actions with non-deterministic effects are dropped. $X^+(P)$ is thus a classical planning problem that can be solved by a classical planner (e.g., FF).

The Closed-Loop Greedy (CLG) planner introduced in (Albore et al., 2009) uses both $X(P)$ and $X^+(P)$ as the execution model and a heuristic model $H(P)$ to select the actions to do next, adopting the same search strategy as the classical FF planner. CLG can be used in either the on-line or off-line mode. In the on-line mode, which is used to capture single executions, the non-deterministic actions $obs(L)$ (i.e., observing the truth value of literal L) in $X(P)$ are applied by randomly selecting one of the outcomes, and CLG essentially becomes an action selection mechanism that outputs the next action to do given the past

Problem	Contingent FF		Pond		CLG		SMCAF	
	time	#acts	time	#acts	time	#acts	time	#acts
cb-4-1	0.27	277	0.98	102	0.35	295	6.31	22.18
cb-4-2	35.88	18739	40.92	1897	18.83	20050	8.70	36.14
cb-4-3	T		1063.11	28008	1537.99	1136920	11.72	45.14
cb-10-1	T		M		415.73	4445	313.89	246.94
cb-10-2	T		M		T		696.27	484.64

Table 6.1: Working with or without Full Contingent Plans

observation/action sequence. In the off-line mode, which is used to construct full contingent plans, both outcomes are entertained. In both modes, CLG is invoked recursively on the resulting states.

Contingent planners typically rely on constructing full contingent plans when striving to look for a sequence of actions mapping the initial belief state into the target belief state. However, the size of the solutions, which is exponential in the number of possible observations, is often a computational bottleneck. The numerical data in the 2nd - 7th columns in Table 6.1 were extracted from (Albore et al., 2009) running on a Linux machine at 2.33 GHz with 2Gb of RAM with a cutoff of 45 minutes and 1.8Gb of memory. Each *time* column shows the total time taken in seconds by the corresponding planner on a particular Colorballs-n-x problem, and each *#acts* column displays the total number of actions in the solution. Notably, Contingent-FF (Hoffman and Brafman, 2005) failed to solve 3 problems after spending 45 minutes on each, Pond 2.2 (Bryce et al., 2006) failed to solve 2 problems after exhausting the available memory on each, and CLG also timed out on Colorballs-10-2 after 45 minutes.

We conducted our experiments on a similar Linux machine at 2.40GHz with 2Gb of RAM. The numerical data we show in Table 6.1's last two columns under the planner heading SMCAF, standing for our self-motivated cognitive agent

framework, are the averages over 50 randomly generated configurations matching the particular Colorballs-n-x settings. In our scenario world, there is a sole agent in an n by n gridworld whose only goal is to carry each of the x balls from its initial position to its goal position based on the ball's color in the gridworld, and each ball is of one of the only four possible colors red, blue, yellow, and green. The agent's repertoire of action operators is composed of:

- *walk* from a location to one of its adjacent locations, with an expected net utility increase equal to the agent's degree of happiness;
- *pick−up* a ball from a location other than the four color-based goal positions, with an expected net utility increase of 100;
- *put−down−red* a red ball at the goal position for red balls, with an expected net utility increase of 100;
- *put−down−blue* a blue ball at the goal position for blue balls, with an expected net utility increase of 100;
- *put−down−yellow* a yellow ball at the goal position for yellow balls, with an expected net utility increase of 100;
- *put−down−green* a green ball at the goal position for green balls, with an expected net utility increase of 100;
- *announce−success* when the agent has placed all balls in their designated color-based locations, with an expected net utility increase of 100.

Other than the anticipated utility values of actions operators, states present no anticipated values to the agent in the Colorballs gridworld. Our agent uses a 3-step deep search horizon with a branching factor of 4 at each step, but aside from this projective lookahead to evaluate the anticipated cumulative utilities of

Problem	**CLG in Execution Mode**					
	Translation		Search		#acts	
	time	size (MB)	avg	max	avg	max
cb-9-1	20.9	16.5	1.21	7.80	33.7	197
cb-9-2	56.4	33.7	4.84	25.70	57.1	288
cb-9-3	113.7	51.4	46.26	122.19	76.3	367

Table 6.2: CLG in Execution Mode on Colorballs-9-i

applicable action sequences, our agent uses absolutely no heuristics or additional search strategies.

For any given n by n gridworld, the only variables we randomized were the initial position of the agent, the initial positions of the x balls, the colors assigned to the x balls, the designated goal positions for the four possible colors; furthermore, we made sure that the aforementioned randomly (with uniform probability) chosen and assigned positions were all unique and mutually exclusive to avoid trivial configurations. Our results in Table 6.1, showing our averages (each averaged over 50 randomly generated configurations matching the particular Colorballs-n-x setting), compare very favorably with the other three contingent planners which either timed out after 45 minutes or exceeded available memory on several problems.

We comment on the noteworthy ability of CLG, in its on-line or execution mode, to generate goal-oriented observation/action sequences without having to first build a complete contingent plan. The numerical data in Table 6.2 were extracted from (Albore et al., 2009), showing for a particular Colorballs-9-i problem, its translation time in seconds, size of the translation in MB, average and maximum search time in seconds, number of actions in the solution, all averaged over 50 samples.

Table 6.3 shows our empirical data on the same Colorballs-9-i problems, listing

Problem	**SMCAF**			
	Run Time		#acts	
	avg	min / max	avg	min / max
cb-9-1	150.30	4.57 / 516.61	168.5	5 / 543
cb-9-2	281.38	16.37 / 642.90	239.12	11 / 552
cb-9-3	345.33	62.60 / 799.17	333.58	51 / 694

Table 6.3: Our Framework on Colorballs-9-i

for each problem, its average, minimum and maximum run times, and its average, minimum and maximum number of actions in the solution. Again, we obtained these average, minimum and maximum figures from 50 randomly generated configurations for each of the Colorballs-9-i problems.

Our advantages are that no translation is required to compile the scenario world into another format, and that the file size of our scenario world for each Colorballs-9-i problem is under 16 KB (including some coding comments) – as i increases by 1, an additional ($place{-}object...$) call suffices. However, our higher run times and number of actions indubitably included meandering actions that repeatedly visited the same states. Again, as we noted earlier near the end of our Towers of Hanoi discussion, this kind of behavior is often beneficial for a reward-seeking agent, as the kind exemplified by our self-motivated cognitive agent framework. We could have obtained faster solutions by using heuristics and by avoiding state duplication in the forward search, but we were most keen to demonstrate the versatility of our framework to model and solve classical planning problems, some of which have stumped state-of-the-art contingent planners that must build full contingent plans.

6.2.2 A Multiagent-n-x-b Traveling Domain

Brenner and Nebel (2009) introduced a simulation environment for Continual Multiagent Planning, called MAPSIM, in which an agent can directly use actions from its plan and execute them in simulation, making perceptions described using the ontology defined by the planning domain and using these perceptions as input for plan monitoring or replanning. They implemented each agent as an individual and independent MAPSIM process, and thus, each agent knows only about its own goal state and its own perceptions.

Brenner and Nebel (2009) also devised a Multiagent-n-x-b domain, similar to the *grid* domain in the International Planning Competitions. In the Multiagent-n-x-b domain, there are b blocked locations, which cannot be traversed by any agent, in the n by n gridworld, and there are x agents each of which is to travel from its initial location to its goal location in the gridworld. As very little empirical data for this domain were given by Brenner and Nebel (2009) and we were unable to establish an email correspondence, we cannot provide a tabular comparison of their results. However, we found the Multiagent-n-x-b traveling domain compelling, and were motivated to augment our framework with extensions for multiple, coexisting self-motivated cognitive agents.

The multiagent extensions we made to our framework further bolster the generality and flexility of our framework. Specifically, our multiagent extensions endow each coexisting self-motivated cognitive agent with:

- its own repertoire of action operators with associated anticipated utility values,
- its own anticipated utility values of state properties,
- its own search beam,

Problem	**SMCAF**			
	Run Time		#acts	
	avg	min / max	avg	min / max
ma-4-1-2	0.73	0.16 / 2.35	3.18	1 / 15
ma-4-1-5	0.59	0.11 / 1.49	3.06	1 / 7
ma-4-2-2	0.85	0.19 / 2.0	6.66	2 / 25
ma-4-2-5	1.70	0.39 / 17.78	10.54	2 / 128

Table 6.4: Our Framework on Multiagent-4-x-b

- its own perception of time (separate from the universal time in the simulated world),
- its own projective lookahead,
- its own (epistemic or not) reasoning capabilities,
- inferences it derives based on its own set of general knowledge inference rules and all of its knowledge in bounded forward inference, and
- its own history of actions performed.

In the Multiagent-n-x-b domain, each agent starts out unaware of the existence of any other agents (let alone their locations), unaware of any blocked locations; but knowledgeable only about the adjacency information of locations in the gridworld (i.e., which are the neighbor locations for each location) , about its initial location and co-located local entities, and what its goal location is. All agents take turns identifying and executing a seemingly best action for their own utility optimization. Since at most one agent is allowed to occupy a location, immediately prior to identifying and executing a seemingly best action, the agent will do a look-around solely to learn whether each adjacent location is occupied (i.e., either one of the b blocked locations, or occupied by at least one entity).

Problem	**SMCAF**			
	Run Time		#acts	
	avg	min / max	avg	min / max
ma-5-1-2	0.84	0.08 / 2.79	3.88	1 / 13
ma-5-1-5	0.89	0.13 / 3.59	4.7	1 / 25
ma-5-2-2	2.40	0.51 / 28.12	10.84	3 / 104
ma-5-2-5	3.26	0.5 / 39.58	17.28	2 / 199
ma-5-3-2	4.72	1.09 / 43.59	20.9	5 / 178
ma-5-3-5	4.22	1.03 / 24.86	24.44	5 / 187

Table 6.5: Our Framework on Multiagent-5-x-b

Despite this look-around, the agent still may not know of all the entities at an adjacent location until it actually reaches it. Furthermore, the look-around gives the *is_occupied*? information of its adjacent locations only, not that of all the locations in the gridworld, so the agent may still retain absolutely no or outdated *is_occupied*? information of all other locations.

Each agent can stay put at any step, but this is dispreferred to walking unless the agent has reached its goal location, at which point staying put reaps a big net utility increase. Specifically, each agent's repertoire of action operators is composed of:

- *walk* from a location to one of its unoccupied adjacent locations, with an expected net utility increase either of 10 if the new location is the agent's goal location, or of 0 otherwise;
- *stay−put*, with an expected net utility increase of 10 if the current location is the agent's goal location, or of -1 otherwise.

Other than the anticipated utility values of actions operators, states present no anticipated values to the multiple agents in this domain. Each of the multiple

Problem	**SMCAF**			
	Run Time		#acts	
	avg	min / max	avg	min / max
ma-6-1-5	2.41	0.25 / 34.84	10.56	1 / 191
ma-6-1-10	3.33	0.18 / 36.47	15.56	1 / 233
ma-6-2-5	5.65	0.76 / 28.40	22.9	3 / 103
ma-6-2-10	6.48	0.5 / 38.83	27.58	2 / 162
ma-6-3-5	9.75	1.59 / 42.64	38.74	6 / 167
ma-6-3-10	13.29	0.95 / 74.37	59.84	4 / 347
ma-6-4-5	14.64	1.51 / 81.39	60.74	7 / 294
ma-6-4-10	15.63	2.19 / 63.10	70.86	10 / 288

Table 6.6: Our Framework on Multiagent-6-x-b

agents uses a 4-step deep search horizon with branching factors of 4, 4, 3, and 2 at each successive step; aside from this projective lookahead to evaluate the anticipated cumulative utilities of applicable action sequences, each agent uses absolutely no heuristics or additional search strategies.

We conducted our experiments in the Multiagent-n-x-b traveling domain for n ranging from 6 to 10. Tables 6.4 through 6.10 show our average, minimum and maximum results for a total of 80 different problems (i.e., of different n-x-b settings). We obtained the average, minimum and maximum figures for a particular Multiagent-n-x-b problem, by randomly generating and running 50 configurations matching the given n-x-b setting. For any given n by n gridworld, the only variables we randomized were the initial positions of the x agents, the goal positions of the x agents, and the b blocked positions; furthermore, we made sure that the aforementioned randomly (with uniform probability) chosen and assigned positions were all unique and mutually exclusive to avoid trivial configurations. We also took care to avoid using randomly generated configurations where it was im-

Problem	**SMCAF**			
	Run Time		#acts	
	avg	min / max	avg	min / max
ma-7-1-5	6.87	0.41 / 65.36	18.94	1 / 180
ma-7-1-10	8.92	0.27 / 87.65	29.38	1 / 257
ma-7-2-5	10.64	1.25 / 53.14	29.14	3 / 151
ma-7-2-10	13.99	1.51 / 115.80	41.6	4 / 351
ma-7-3-5	17.94	1.35 / 122.36	48.82	4 / 332
ma-7-3-10	28.35	2.44 / 151.47	84.98	7 / 469
ma-7-4-5	31.66	5.58 / 119.42	72.62	10 / 254
ma-7-4-10	56.26	5.11 / 266.79	115.82	10 / 527

Table 6.7: Our Framework on Multiagent-7-x-b

possible for at least one agent to travel from its initial position to its goal position, given the b blocked positions (but disregarding positions of all other agents).

Tables 6.4 through 6.10 display both the average, minimum and maximum run time in seconds taken by all agents, and the average, minimum and maximum number of actions performed by all agents to reach their respective goal positions. Table 6.4 is our results for 4 Multiagent-4-x-b problems; Table 6.5, for 6 Multiagent-5-x-b problems; Table 6.6, for 8 Multiagent-6-x-b problems; Table 6.7, for 8 Multiagent-7-x-b problems; Table 6.8, for 15 Multiagent-8-x-b problems; Table 6.9, for 18 Multiagent-9-x-b problems; Table 6.10, for 21 Multiagent-10-x-b problems.

Once an agent reaches its goal location, it much prefers staying put to walking about. As in (Brenner and Nebel, 2009), there is no inter-agent communication, coordination or collaboration, so it is possible for an agent to become trapped at a non-goal location when all its adjacent locations are occupied (i.e., either being a blocked location, or occupied by another agent similarly trapped itself

Problem	**SMCAF**			
	Run Time		#acts	
	avg	min / max	avg	min / max
ma-8-1-5	11.75	0.57 / 139.01	25.32	1 / 311
ma-8-1-10	7.71	0.57 / 57.45	17.48	1 / 137
ma-8-1-15	15.69	0.41 / 171.04	40.22	1 / 474
ma-8-2-5	12.22	1.51 / 42.09	25.56	3 / 88
ma-8-2-10	28.75	2.74 / 97.60	65.54	6 / 216
ma-8-2-15	35.12	0.98 / 364.39	86.4	3 / 312
ma-8-3-5	32.02	3.75 / 119.76	72.4	8 / 245
ma-8-3-10	33.72	1.82 / 141.53	77.8	4 / 345
ma-8-3-15	53.86	6.21 / 251.97	130.38	13 / 583
ma-8-4-5	33.53	3.74 / 162.88	73.48	8 / 356
ma-8-4-10	58.99	3.01 / 415.19	141.44	6 / 1000
ma-8-4-15	74.41	6.05 / 374.39	184.72	13 / 1045
ma-8-5-5	57.21	6.12 / 232.86	129.98	12 / 518
ma-8-5-10	94.19	10.59 / 271.47	226.98	23 / 662
ma-8-5-15	81.37	10.79 / 310.85	208.08	27 / 825

Table 6.8: Our Framework on Multiagent-8-x-b

or already in its goal location). Accordingly, using our knowledge of the size of the configuration, we imposed a sufficiently large upper bound on the number of actions and cutoff time limit for each agent, in order to filter out theoretically workable configurations where some agent(s) ultimately became trapped. Consequently, for each problem, the average, minimum and maximum figures were only from 50 configurations where all agents could in principle and, actually did, finish successfully.

Brenner and Nebel (2009) stated that they placed a 10-minute time limit on their runs, counting a run as successful if all agents reached their respective goal locations within that time and unsuccessful otherwise. They then normalized the success rate relative to the success rate for the version where all agents had perfect knowledge of the entire grid occupancy – the version that invariably gave the best performance. Therefore, having failed to establish a correspondence with either author, we cannot possibly infer what absolute success rates were attained. But, since the agents in their runs that could see only the immediately adjacent locations succeeded at a rate of only 37% - 62% *relative to* the full visibility case (and thus at a lower rate *absolutely*), it is safe to say that many of their runs failed within the 10 minute time bound. Their focus apparently was less on run times, but more on showing that a certain amount of forgetfulness (e.g., only remembering information gained over the last 5 steps) is helpful when the world is imperfectly known, presumably because remembering locations of other agents is deceptive – they also are moving.

If we assume that the one example they display (a 6 by 6 grid with 4 agents and 10 blocked locations) is somewhat typical, then our run times fall far below their 10-minute time limit. In fact, for that particular world, ma-6-4-10 in Table 6.6, our agents finished on average in 15.63 seconds (and the worst time was 63.10 seconds); and even for a 10 by 10 grid with up to 7 agents, our average run times were well under the 10 minute mark. Our results are thus quite remarkable, as the

Problem	**SMCAF**			
	Run Time		#acts	
	avg	min / max	avg	min / max
ma-9-1-5	16.51	0.64 / 146.80	23.58	1 / 216
ma-9-1-10	24.72	0.7 / 228.51	35.96	1 / 338
ma-9-1-15	32.43	0.63 / 314.98	50.92	1 / 520
ma-9-2-5	44.82	2.11 / 271.64	63.4	3 / 396
ma-9-2-10	39.82	3.57 / 160.56	59.64	5 / 251
ma-9-2-15	70.46	2.42 / 544.72	112.1	3 / 839
ma-9-3-5	61.86	5.53 / 313.17	89.22	8 / 467
ma-9-3-10	69.96	7.34 / 236.81	106.62	10 / 369
ma-9-3-15	96.16	3.13 / 538.91	153.14	4 / 846
ma-9-4-5	115.05	10.70 / 481.28	167.84	14 / 679
ma-9-4-10	126.53	11.57 / 758.04	194.5	16 / 1150
ma-9-4-15	187.15	8.26 / 1070.25	295.72	12 / 1707
ma-9-5-5	123.81	26.07 / 431.55	182.82	35 / 637
ma-9-5-10	178.05	30.06 / 685.04	274.76	45 / 1074
ma-9-5-15	234.98	9.97 / 941.56	385.84	14 / 1459
ma-9-6-5	140.93	22.93 / 565.87	207.96	31 / 859
ma-9-6-10	237.94	61.64 / 1116.05	372.44	88 / 1756
ma-9-6-15	256.21	28.51 / 878.77	411.8	43 / 1225

Table 6.9: Our Framework on Multiagent-9-x-b

planning technique employed by (Brenner and Nebel, 2009) is quite sophisticated, in effect similar to Hierarchical Task Network planning (as they explained in detail in their Related Work section).

Notwithstanding advances made by conformant and contingent planners and continual planners to cope with a dynamic and partially observable environment, these planners can only work in sole pursuit of reaching a user-specified goal state. In contrast, our utility-optimizing self-motivated cognitive agents have the *choice* of functioning as goal-directed planners in certain settings.

Some comments on expressivity are in order as well. The planning domain definition language (PDDL) (McDermott, 1998; Fox and Long, 2003), supported by many planners (including Contingent-FF, Pond, CLG, etc.), is the prevailing standard for defining planning domains, especially for systems participating in the International Planning Competitions. PDDL adopts a STRIPS-like formalism with an operator-centric view. The syntax of PDDL actions is similar to that in our operators (e.g., parameters, preconditions, and effects), and PDDL also allows for durative actions and quantitative preconditions and effects, which can be handled by a general procedural attachment method.

Some devices available in PDDL, but not in our framework, are type constraints on parameters (which, however, are easily re-cast as preconditions), conditional effects (which in principle can be dispensed with through use of multiple variants of actions), and quantified ($\forall$ and $\exists$) preconditions and effects. On the other hand, unlike PDDL, our framework allows for quantified conditional formulas that are used to expand state descriptions, occluded and unoccluded predicates that govern the use of the CWA, and the propositional attitudes of *knowing−that*, *knowing−whether*, and *wanting*. The latter are essential for formalizing knowledge-acquisition actions and for answering questions about the agent's own attitudes, as is desirable for a *communicative* agent.

Problem	**SMCAF**			
	Run Time		#acts	
	avg	min / max	avg	min / max
ma-10-1-5	41.75	0.52 / 338.04	47.56	1 / 385
ma-10-1-10	53.92	0.95 / 350.07	78.1	1 / 426
ma-10-1-15	64.61	0.93 / 397.89	80.28	1 / 519
ma-10-2-5	68.21	3.5 / 377.52	78.95	4 / 444
ma-10-2-10	93.17	4.17 / 617.31	112.12	5 / 749
ma-10-2-15	112.56	3.22 / 705.74	143.62	4 / 945
ma-10-3-5	132.55	12.34 / 434.68	156.0	13 / 527
ma-10-3-10	135.54	9.53 / 668.54	166.3	11 / 877
ma-10-3-15	160.32	6.27 / 772.98	202.26	7 / 907
ma-10-4-5	165.31	10.99 / 817.93	193.68	12 / 960
ma-10-4-10	191.31	10.42 / 877.08	231.72	12 / 1050
ma-10-4-15	239.05	13.37 / 628.09	280.18	15 / 773
ma-10-5-5	255.41	27.44 / 730.04	301.82	33 / 885
ma-10-5-10	310.90	22.07 / 800.67	385.58	27 / 960
ma-10-5-15	282.47	16.5 / 878.35	358.92	20 / 1162
ma-10-6-5	291.59	29.02 / 697.70	350.1	36 / 828
ma-10-6-10	320.61	71.90 / 978.49	325.58	59 / 945
ma-10-6-15	351.49	37.79 / 1021.05	366.84	39 / 1038
ma-10-7-5	374.32	127.78 / 1077.98	380.32	127 / 980
ma-10-7-10	429.75	61.48 / 1050.73	442.54	77 / 1077
ma-10-7-15	491.94	82.75 / 1531.86	498.02	88 / 1658

Table 6.10: Our Framework on Multiagent-10-x-b

6.3 Cognitive Robots

We next turn to the area of *cognitive robotics* (e.g., (Levesque and Lakemeyer, 2008; Fritz and McIlraith, 2008; Sardiña et al., 2004)), a term used by many to distinguish robots that reason symbolically from ones more concerned with proficient motion or manipulation, as in the case of industrial robots and behavioral robots (see Section 6.4).[1] Work in this area has addressed quite challenging issues in planning, including conditional and sensing actions, iteration, incomplete knowledge, qualitative user preferences, uncertainty or nondeterminism, exogenous events, and reasoning about the planning agent's own mental states, to be anticipated as a result of contemplated actions. A noteworthy tool in some of this work has been the Golog agent programming language (Reiter, 2001). Golog layers formal procedural constructs (tests, nondeterministic choice, sequencing, conditionals, while-do, and procedure definitions) on top of the Situation Calculus. We do not discuss cognitive robots with significant question-answering or other dialogue capabilities here, deferring discussion of such robots and virtual agents to Section 6.5 instead.

Shakey (Fikes and Nilsson, 1971; Nilsson, 1984) is the earliest example of a cognitive robot. It used the STRIPS planner and resolution theorem proving to solve problems involving navigation through various rooms and moving around boxes. Not only could Shakey explain its actions by reference to its goal stack, but it could also optionally find and construct plans from previously stored generalized triangle tables. In fact, Shakey was able to realize speed-ups by a factor of 5 by using STRIPS in conjunction with triangle tables.

TJ (Torrance, 1994) is a navigation robot supporting typewritten discourse with human users. It used a template-based pattern matcher to interpret type-

[1] However, the term has also been adopted by robotics researchers who emphasize the use of higher-level programming languages for specifying perception and physical action, with no particular commitment to knowledge-based reasoning.

written utterances of a user; in response to a user utterance – be it a command, question or sentence, TJ could learn names of locations, find a path between two locations, navigate to a different location, or answer the user's question about its current location and its navigation. We might have placed our remarks on TJ in Section 6.5 because of this robot's verbal abilities. However, since its interpretation of inputs is limited to direct updates of its map data and of its navigation plans consisting of trajectories specified in segments with stopping criteria, TJ falls short of the sort of symbolic understanding and reasoning exemplified by the agents discussed in that section.

Several SplinterBot mobile robots have been programmed as cognitive robots. For example, Laird et al. (2011) described two versions of this robot, where the more complex version (with several hundred Soar production rules) is able to "clean rooms"; this consists of moving specific types of colored blocks (at most one per room) in a line of rooms to a storage area at the end of the line of rooms, constructing its own map of room layout and block locations as it goes. The map and block locations are stored declaratively in semantic memory. There is also an episodic memory, consisting of snapshots of the working memory contents, taken whenever the robot performs an external action (other than rolling forward). This allows for retrieval of locations of previously seen blocks.

A particularly impressive recent cognitive robot is Leonardo (Breazeal et al., 2009). This engaging, gremlin-like robot (and a virtual counterpart) can perform certain cooperative tasks and certain rule-learning tasks in table-top worlds consisting of children's blocks, boxes, and visual barriers. The beliefs of the robot are comprised of time-stepped object trajectories whose elements (snapshots) are perceptual vectors such as (*block*, *red*, *triangle*, (10, 0, 0)), obtained on the basis of special reflective markers. Leonardo can also represent a human collaborator's beliefs in this way, allowing for gaps in perception due to occlusion or due to temporary absence of the collaborator.

Leonardo further uses goal-directed schemas similar to propositional STRIPS operators, aimed at simple goals such as that the human should have a cookie or chips (all represented by blocks), and it can ascribe these schemas to human collaborators. This allows for simple mind-modeling, and helping a person with a faulty model, caused by switching contents of boxes in the person's absence, to achieve a goal. The robot has also been adapted to the task of learning behavioral rules such as "put a marble on each blue block and each red block" based on an automated demonstration or human demonstration, allowing for the fact that the human demonstration has a different visual perspective, and may be limited to just the blocks close to the human. In terms of behavior and elementary mind-modeling, Leonardo is perhaps the most interesting cognitive robot, but its quasi-symbolic knowledge representations of trajectories and goal schemas are more restricted in scope than those in Golog-based or other logic-based robots.

Some other examples of cognitive robots are enumerated as follows:

- Tino (de Giacomo et al., 1996), which used propositional dynamic logic embedded in a description logic for navigation planning;
- Diablo (Baral et al., 1998), which used first-order metric interval temporal logic in its successful execution of a challenging tidy-up task;
- a Robocup player (Ferrein et al., 2004) based on an online, decision-theoretic version of Golog;
- DORO (Carbone et al., 2005), which employed Temporal Concurrent Golog in its successful solution of NIST "rescue" scenarios, where victims need to be found in varied environments, either autonomously or with partial or heavy intervention by a human teleoperator;
- Dora (Aydemir et al., 2010), a robot that creates navigation goals and plans motivated by gaps in its knowledge about the layout of rooms it roams in;

as such it is said to exhibit curiosity and self-awareness;

- ARMAR (Krüger et al., 2011; Petrick et al., 2010), a humanoid robot that combines ideas from STRIPS, affordance theory, and the event Calculus in its motion and manipulation planning;

- RHINO (Burgard et al., 1999), a compelling museum tour guide based in part on Golog, but able to deal effectively with unpredictably moving humans (while it was itself in motion), augment its map, communicate its current goals through a display, and appropriately play prerecorded messages.

The capabilities of these robots evidently intersect those of agents in our framework, and many of them address issues such as uncertainty, physical perception, and learning that our framework currently excludes. However, robot development by its nature requires heavy investment in the design of perceptual and motion capabilities. Therefore, this leaves less scope for developing more general reasoning or planning capabilities, such are required for modeling a world containing an assortment of object types with various types of properties and uses, and agents whose beliefs, wants and intentions need to be taken into account.

In practice, cognitive robots have largely focused on motion planning, with behavior driven mostly by externally supplied goals rather than internal motivation. Admittedly, robocup, tidy-up, rescue, and exploratory robots can be said to be motivated by a drive to optimize certain success scores. While cognitive robots allow for incomplete advance knowledge of the disposition of obstacles and target objects, they are typically incapable of distinguishable knowledge-based verbal interaction, nor is there a significant capacity for general reasoning about mundane objects, or about their own or other agents' beliefs and wants. Leonardo makes interesting progress in the direction of the latter, nonetheless.

6.4 Behavioral Agents

A behavioral agent is one whose activities are determined by immediate environmental features and the agent's own current state. Their relevance to our enterprise lies in their reactivity, in the sense that they exploit opportunities or avert threats as they arise. However, they generally lack symbolic reasoning and planning, apart from, perhaps, an exploratory lookahead in the state space. We have already noted that reinforcement learning (RL) agents are of this type. While RL agents learn a behavioral policy as a result of extensive exploration and the concomitant reward/punishment experience, such a policy is simply a way of reacting to the current state. The disadvantage of this way of coping with an environment containing risks and opportunities is twofold. First, the learned policies are very domain-specific. Second, it does not scale up well to a complex world where a wide variety of entities and situations may be encountered thus contributing to an extremely large number of successors of a state. The advantage of deliberation – symbolic thinking – is precisely its generality.

Autonomous agents in game worlds are typically programmed as purely behavioral agents. However, the game agents described in (Dinerstein et al., 2008) plan ahead using sequences of actions in the form of motion segments extracted from motion trajectories demonstrated to the agent by a human teacher (e.g., in the case of a submarine agent, showing how to cross a pod of whales without collision). These sequences are retrieved on the basis of features of the current situation, and a choice is made based on expected fitness resulting from the action sequence.

Another interesting game agent is reported in (Davies and Mehdi, 2006), based on Bratman's belief-desire-intention model (Bratman, 1990). Beliefs in this case consist of data objects that store the agent's status (health condition, location, current weapon, etc.), enemy locations, navigation points, and other items. The

agent perceives navigation points, useful items such as health packs, and other agents within its cone of vision. However, while desires and intentions are mentioned, agent behavior still seems to be largely reactive: navigate to a health pack when in diminished health, retreat from enemies encountered while seeking a health pack, attack enemies encountered when in good health, explore territory by default, etc.

To summarize, although limited sorts of cognition and planning have made their way into game agent research, the knowledge representations and strategies used tend to be very domain-specific, and do not provide the agents with cognitive, let alone reflective, capabilities in any strong sense. Furthermore, there is no provision for intelligent verbal interaction.

Certain mechanized game devices can be grouped with the preceding types of game agents. For example, the camera-equipped table-soccer machine KiRo (Tacke et al., 2004), like game agents that use a lookahead, considers multiple action sequences it might carry out (or that might occur if the opponent gains control of the ball), and evaluates their expected utility before choosing an action. Like our agents, KiRo replans after each action (motor activation of one of the rods of the soccer table). Despite these similarities, we regard this robot as behavioral rather than cognitive, since its reasoning does not use knowledge, either in the form of symbolic action descriptions or symbolic rules or facts.

Most humanoid robots, despite their life-like features, are also purely behavioral, and as such incapable of using or acquiring symbolic knowledge. For instance, the behavioral system of MIT's well-known Kismet robot (Breazeal and Brooks, 2004), a predecessor of the more advanced Leonardo robot described in the preceding section, is structured as a decision tree, where each node consists of several competing IF-THEN rules, and the strengths of the competing IFs determine the choice of THEN. The IF-part of each rule tests for perceptual inputs and internal state parameters, and the THEN-part selects a successor rule set. The

rule sets bottom out in modes of behavior such as looking happy or sad, turning towards or away from a person or object, or making sounds to attract attention. Kismet exhibits the expected sorts of behavior as a function of circumstances; for example, it intensifies its focus on satisfying a drive (viz. achieving social interaction, sleep, or toy-viewing) that has persisted for a while, and also gets more prone to disgust when encountering stimuli that do not relieve the urgent drive.

The creators of pet-like (e.g., SONY's AIBO commercialized in 1999) and humanoid robots argue vigorously for emotional intelligence in robots, hoping to make them socially engaging entities. However, for them, this seems to mean primarily expressing emotions, as a means to communicate understanding, success, difficulty, etc. Regrettably, intuiting or inferring how others feel and what they want is largely neglected, as it would require mental modeling that falls outside of these nonthinking robots.

The humanoid robots under past and present development in research labs and commercial enterprises across the world are too numerous to mention. Although their experiments reiterate the important role of emotions in robots, there is no inkling of interest in adding genuine thinking capabilities. Notwithstanding moves towards higher cognitive functions, the main emphasis continues to be on improved perceptual and motion capabilities, while interactions with objects and humans is preprogrammed and, in the case of verbal interaction, scripted.

6.5 Agents That Plan and Communicate Verbally

Several past and current agent systems and frameworks more nearly resemble ours than any of the planning systems, cognitive robotics projects, or behavioral agent systems mentioned above.

A remarkable early system was Winograd's SHRDLU (Winograd, 1972). It was not only able to plan and act in a simulated world of children's blocks on a table, but did so while taking instructions and answering questions in English via a keyboard interface. Its question-answering and planning capabilities within its blocks domain were quite advanced, due to its proceduralized linguistic and planning knowledge. Unfortunately, the complexity and lack of transparency of the procedural representations, and the subtlety of procedural interactions, seem to have forestalled follow-up work targeting broader or more ambitious domains.

An even more impressive agent with planning and dialogue capabilities was Vere and Bickmore's Homer (Vere and Bickmore, 1990), which was reviewed in Section 2.4. Homer was a small simulated robotic submarine navigating a 2-dimensional Seaworld, featuring an island with docking and fueling facilities, icebergs, a barge, and a variety of other mostly stationary objects. The agent was able to form and execute plans to navigate to particular places, pick up and deliver parcels, see and identify objects when encountering them, take pictures of objects in passing, and infer larger-scale event types (such as passing an object) from multiple smaller-scale ones (such as having the object ahead of it at one point and behind it at another). Like SHRDLU, it was able to take its instructions, answer questions, and report on its plans, actions and observations in English, but used a larger vocabulary of about 800 words, and showed a somewhat greater degree of self-awareness.

Despite Homer's admirable integrated range of abilities, Homer was difficult to extend into a more sophisticated agent. This difficulty possibly resulted from the shortcomings in its knowledge representations. Aside from its ad-hoc symbolic representations, there was mixing of domain symbolism with internal book-keeping symbols in its knowledge representations. For example, Vere and Bickmore's definition of *go* (from one place to another, as a state transition) contains the line $(.token \quad = \quad (VERE.GENSYM))$, which really refers to internal variable

management. Mostly importantly, as Vere and Bickmore acknowledged, Homer lacks initiative, preferences, desires and values – all the attributes that would truly anthropomorphize an agent.

Compared with SHRDLU's and Homer's sophisticated goal-directed planning in geometrical worlds, and their rather advanced English capabilities, our agents are certainly less fully developed. However, our project was never concerned with engineering an elaborate simulation or, at this point, complex linguistic skills, but rather with providing a flexible framework for defining self-motivated cognitive agents that can effectively reason about their world (including themselves), can use their knowledge to plan for long-term cumulative rewards, and can answer simple questions about their world and themselves. Thus we, as well as some undergraduates in our Machines and Consciousness courses, have been able to implement various kinds of self-motivated agents and even classical planners in our framework. The English question-answering (QA) capabilities, though elementary in form, allow for inference, cover both physical properties and mental attitudes, and are easy to extend to new domains. Though SHRDLU and Homer were distinguished AI milestones, their domain-specific procedural aspects in both planning and dialogue confined them to their particular domains, and forfeited the elaboration-tolerance property that, as John McCarthy maintained, seems crucial for scaling up AI.

Two more recent, highly developed agent projects that overlap conceptually with ours are the GLAIR/Cassie project (Shapiro and Bona, 2010; Shapiro, 1998) and the TRIPS project (Ferguson and Allen, 1998). GLAIR is the architectural framework; Cassie, the name of successive versions of an agent based on that architecture. Cassie, as a virtual agent or as a Nomad robot (with foveal vision, sonar, and wheels), operated in a world of large colored cubes and balls, interpreted as other robots and two people. It could follow commands such as "go to the green robot" and "follow Bill," and commented on its actions by saying, for

example, “I found Bill. I went to Bill. I am near Bill. I am following Bill.” Cassie could also answer questions such as “who are you?”, and “who have you talked to?”, thus exhibiting basic self-awareness. Its planning relied on formal rules such as that if an agent is to follow an object, then it needs to find the object in order to be looking at it, and then go to it in order to be near it. The acting executive described in (Shapiro and Bona, 2010) performs goal-regression, though the various versions of Cassie seem to have relied mostly on predefined plans, and on policies about when certain acts should be performed. The effects of actions are always believed by Cassie to hold.

The knowledge representation is partially equivalent to FOL, but allows unquoted formulas as terms, leading to an approach to modals not unlike ours. The GLAIR architecture is more ambitious than ours, in its allowance for perceptual and motor layers beneath the conscious level, and Cassie features an episodic memory allowing QA about past interactions. The latter remains an unimplemented goal in our framework, although our group has previously demonstrated episodic memory in a related QA system focused on self-awareness (see (Schubert, 2005), drawing on earlier work by Kaplan (1999); see also (Morbini and Schubert, 2008)). Distinctive characteristics of our framework once again include the forward-directed, reward-optimizing planning, the systematic elaboration of world descriptions through forward inference, and the use of the restricted CWA to allow uniform handling of properties of objects at hand (or at a remove) and of attitudes, for both inference and QA.

The TRIPS system (Ferguson and Allen, 1998), like GLAIR/Cassie, is the result of many years of development. It stands out as an end-to-end system that not only performs problem solving and planning in a fairly complex simulated world, but does so collaboratively, interacting quite robustly with various human users via spoken language. The simulated world involves emergency evacuation of people from several locations on an island, using air and ground transport. A well-

developed display shows the island and routes being followed, highlights salient entities, shows the speech recognizer output, and allows for certain graphic user interface interactions.

The system incorporates both generic and domain-specific components. For example, it employs a general-purpose grammar and chart parser, but also maps generic verb frames to domain-specific and again frame-like knowledge representations. Though geared towards the evacuation domain, many operations are relatively domain-independent at an abstract level, relating to the construction of a shared plan, such as proposing, accepting, or rejecting potential actions, modifying aspects of actions(e.g., resources to be used), and retracting parts of plans. The use of generic components has allowed Allen's research group to quickly develop new prototype applications, such as patient health monitoring (Ferguson et al., 2010), and collaborative learning of web tasks (Jung et al., 2008).

As no one yet knows how to achieve broad-coverage, domain-independent language understanding, TRIPS employs production-system-like heuristic rules to map preliminary logical forms derived from parse trees into plausible intentions underlying the user's utterances. The intentions are typically to propose an addition to, or alteration of, the current plan, or to ask a question, for instance, how much time a tentatively added plan segment will take, or about the availability of vehicles.

Our framework certainly lacks the TRIPS system's collaborative and focused problem-solving and linguistic skills and well-developed virtual and prototype application domains and interfaces. However, it has the distinctive features we have pointed out before. Our framework enables ME to reason about its own and other agents' attitudes, act to try to optimize its own future rewards, and make uniform inferential use of general, logically represented knowledge for planning and QA, whereas reasoning in TRIPS consists primarily of instantiating recipes that promise to achieve some current objectives. Moreover, the transparent declarative

form in which knowledge and plans are represented in our framework simplifies the instantiation of new agents and objects, and undergraduates in our courses have been able to create their own simulated worlds in term projects easily.

6.6 Some General Architectures for Intelligent Autonomous Systems

Lastly, we comment on the relationship between our agent framework and some well-known architectural proposals for intelligent agents. One such proposal is the BDI architecture of (Bratman, 1990), already reviewed in Section 2.3 and briefly mentioned in Section 6.4 in connection with a BDI-based game agent. For Bratman, a key advantage of forming and committing to certain plans was that this makes reasoning about the world tractable, by pruning away the great majority of possible futures that are open to the agent, given its behavioral repertoire. While the importance of computational pragmatism in this sense is indisputable, for us the most important advantage of plans is that they allow estimation of cumulative rewards, and thereby rational choices. Motivation in our agents is in a sense more fundamental than in BDI agents. Instead of regarding desires as primary, our framework grounds desires (and along with them, plans and intentions) in a system of rewards and costs. Consequently, rather than necessarily remaining committed to plans and intentions until they are fulfilled or rendered unsustainable by the circumstances, our agents may very well be deflected from the current plan of action by new knowledge and new opportunities and threats.

In the metareasoning literature, a common view of agent architecture is one that assumes a metareasoning layer monitoring and controlling a domain-oriented reasoner, which in turn controls action (Cox and Raja, 2008). We think that enabling an agent to monitor and reason about its own performance as a reasoner,

learner, and actor in the world is indeed an important objective, but we regard a competent planning capability as a prior requirement. Furthermore, based on our own studies in metareasoning, we view the various levels of reasoning as tightly intertwined, rather than as separate, layered processes (Morbini and Schubert, 2008). However, this architectural issue is of limited relevance to our agents, since they do not monitor or retrospectively assess their actions, let alone their own reasoning and planning. Our agents perform metareasoning only in the sense of being able to make inferences about their own and other agents' attitudes, and that aspect of their reasoning plays the same role as object-level reasoning in verifying preconditions of contemplated actions, projecting resultant states, and answering questions.

Several architectural frameworks bear the imprint of Allen Newell's cognitive perspective (Newell, 1990). One is Soar (Laird et al., 1987), a programming environment for building rule-based expert systems, equipped with a working memory, semantic memory, and episodic memory, as in the SplinterBot robots described in Section 6.3. As an agent architecture, Soar is amenable to carrying out multiple tasks in time-shared fashion, and thus has been used in various simulation-like studies for single and multiple agents, such as collision avoidance for autonomous air vehicles, or simulation of events such as weapon use, damage, fire, smoke, and flooding on ships in a naval fleet.

Nevertheless, it appears that the Soar architecture is less well-suited to complex reasoning, planning, and dialogue for two reasons. First, productions – like the procedural knowledge employed in SHRDLU and other early agents – are inclined to encode knowledge in a rather opaque way, resulting in a heterogenous mix of objects and tasks in working memory that makes for a challenging debugging environment. Second, the knowledge representations used in semantic and episodic memory are traditionally limited to simple attribute-value constructs, with no systematic allowance for logical connectives, quantifiers, or semantically well-founded

representations of attitudes and other modalities. Therefore, the kind of systematic, reasoned lookahead and QA performed by agents in our framework seems to us rather difficult to achieve within the Soar architecture.

Another framework in the tradition of Newell is ICARUS (Langley, 2006). The emphasis here is on perceptually grounded concepts and physical actions, and on goal-oriented means-ends analysis. Concepts allow for types and attribute-value pairs to be defined. Skills are defined analogously to planning operators (with allowance for hierarchical decomposition), and can be acquired by abstraction from successful action sequences previously constructed by means-ends analysis. An example of a program readily built within ICARUS is one for delivering packages within a simulated driving environment, and skill-learning can be demonstrated within such an environment. However, there is as yet no built-in provision for forward search in a problem space, or for attitudes, logical inference, or question-answering.

As a third architecture partially indebted to Newell, we take note of ACT-R (Anderson et al., 2004), the product of a lengthy cognitive modeling tradition. The emphasis in this architecture is on simulating human problem-solving and learning in empirically testable detail, and even via functional MRI. Declarative memory is assumed to be a type of relational semantic network, and as such is more or less compatible with logical representations. Various functional modules are posited, corresponding to particular cortical regions, such as a visual module for visually identifying objects, a manual module for controlling the hands, a declarative module for retrieving information from declarative memory, and an intentional module for tracking current goals and intentions. The modules store information in their associated buffers, and the contents of these buffers are accessible by a central production system, which can in turn modify the contents of the buffers.

Interesting models of performance on various memory tasks, learning tasks,

and problem solving tasks including Towers of Hanoi have been implemented within the ACT-R framework and its ACT predecessor. There has been work on designing cognitive ACT-R-based agents as well in recent years. For instance, Best and Lebiere (2006) described initial work on virtual agents intended to act as virtual opponents to military trainees in urban combat. Productions specify actions such as evading or shooting at an enemy as a function of available escape routes, current goals, and the assigned role of the agent in a group action. Furthermore, maps can be acquired autonomously, and agents pass user-authored text messages such as "go on the count of three." Plans executed by agents are scripted as schematic chunks whose execution is situation-dependent, with allowance for reactive interruption by newly perceived threats. However, only small, partial scenarios appear to have been implemented, and the agents are certainly constrained by the coordinated behaviors scripted by the programmers and by the goals assigned to them, rather than being self-motivated. They cannot in any genuine sense consider and answer questions, nor do they think ahead and make knowledge-based inferences. In contrast, the use of a well-reasoned lookahead and inferential mechanisms enables ME not only to infuse self-motivated forethought into reactive-level behavior, but also to introspect as well as reason about other agents' mental states.

7 Conclusion

This dissertation began by illuminating the gap between the paradigm of traditional planners and reasoners and the paradigm of self-motivated agents in AI. Deprived of self-awareness and such mental attitudes as beliefs, desires and intentions, traditional planners and reasoners are strictly externally motivated and thus dedicated solely to the fulfillment of user-given goals, without regard for their own cumulative utility. On the other hand, self-motivated (behavioral) agents have conventionally focused on acquiring utility-optimizing mappings from states to actions; however, due to their non-symbolic but feature-based representations, they are incapable of reasoned anticipation and planned achievement of future states and rewards. In essence, the latter agents can behave only reflexively, rather than reflectively.

We firmly believe that general human-like intelligence entails planning and reasoning. Furthermore, internal motivation is inextricably intertwined with human-like intelligence, along with one's desire to proficiently respond to opportunities, threats, and failures in one's environment in a utility-optimizing (or cost-minimizing) manner. Motivated by our goal to integrate self-motivation into planning and reasoning, we presented a general framework for creating self-motivated cognitive agents that are capable of planning flexibly in an incompletely known world, making inferences over a substantial knowledge base, and engaging in

question-answering dialogue, all while pursuing their own rewards.

Our agent *ME*, as exemplified empirically, plans deliberately and continually, and acts opportunistically in accord with its reasoned expectations about future rewards and penalties. It does so by drawing on its specific and general knowledge about itself and its environment, including both introspective knowledge and knowledge about mental states of other agents. *ME* can also perform bounded forward inference in any given state. By prudently appealing to a restricted version of the CWA, *ME* further demonstrates its self-awareness and epistemic knowledge via question-answering with the human user.

We reiterate our meaningful research contributions presented in this dissertation:

1. *We have bridged a gap in AI research.* Our framework can be viewed as an integration of the planning-based agent paradigm and the behavioral agent paradigm. The former agent largely neglects the current state, except to the extent that the current state supports or fails to support steps towards some goal state; the latter agent chooses a best action for its current state as per its learned policy which reflects only past reward/punishment experiences. When devising and evaluating possible plans of actions using its projective lookahead and knowledge, our self-motivated cognitive agent is not only focused on the current state, but also future-oriented striving to optimize its cumulative utility. The use of a well-reasoned lookahead and inferential mechanisms enables our agent not only to introspect as well as reason about other agents' mental states or about object properties, but also to infuse self-motivated forethought into reactive-level behavior.

2. *We have demonstrated the feasibility of combining planning, inference and dialogue in a completely self-motivated cognitive agent.* Although many AI systems exhibit some of these capabilities, none, to our knowledge, have in-

tegrated them all. Our self-motivated cognitive agent can derive knowledge-driven inferences, plan flexibly and act opportunistically in an incompletely known world, and engage in distinguishable knowledge-based verbal interaction, all while aiming to optimize its own cumulative utility. Notably, by judicious application of a restricted version of the CWA, our agent further proves its self-awareness and epistemic knowledge to the human user via verbal interaction.

3. *We have created a general, flexible, and easy-to-use self-motivated cognitive agent framework.* We have shown that our general framework is flexible enough that it can be easily, and has been, used to design self-motivated cognitive agents in simulated worlds and ported to purely goal-directed domains as well. Students in our Machines and Consciousness course for three consecutive offerings have been able to define and experiment with their own self-motivated cognitive agents and interesting simulated worlds in their term projects. In addition, as discussed in Sections 5.3.3, 6.1 and 6.2, we successfully simulated strictly goal-directed planning behavior, and represented and solved such planning problems as the Towers of Hanoi, a Logistics domain, a Colorballs domain, and a Multiagent traveling domain using our framework. Notably, our framework performed impressively better than some state-of-the-art contingent planners that build full contingent plans in the Colorballs domain, even solving problems that stumped them. The multiagent extensions we have made to our framework to enable multiple, coexisting self-motivated cognitive agents further bolster the generality and versatility of our framework.

7.1 Future Work

We have established that our framework for self-motivated cognitive agents is compelling, distinctive and versatile as it stands. It also lays the foundation for promising future work involving more human-like agents capable of self-motivated dialogues about the real world.

Ultimately, we envision an explicitly self-aware and self-motivated conversation agent, communicating about the real world, not a simulated one, and perhaps at a later stage equipped with perceptual and motor abilities. The agent's behavior will be driven by a continually evolving plan. The lifting of our approach to deal with dialogues about the real world, driven by continual, self-motivated planning, will involve some major challenges.

Allowance for Uncertainty

Several extensions to the current framework would be desirable. One is allowance for degrees of uncertainty about the truth of some predications and about the effects of actions. Thus in exploring future states, not only alternative actions, but also alternative outcomes of those actions (with probabilities) would be considered, and the plans formed would generally be *conditional* plans. Also, if ME is uncertain whether the preconditions of an action hold, it should nonetheless consider attempting that action, especially if the potential rewards of success are high and the risks are low.

Conditional occlusion is another desirable enhancement; e.g., what is contained in a box might be occluded if the lid is closed, but unoccluded if the lid is open. Instead of marking predicates as definitely occluded, we would like to replace the notion of occlusion with explicit metaknowledge. For example, the agent might have a metacognitive axiom to the effect that if it can see the interior of a container

and does not see a particular item in it, then that item is not in the container (cf. (Morbini and Schubert, 2008)).

Addition of a Learning Capacity

A capacity for learning would be another desirable extension of the current framework. This could be effected by experience-based modification of rewards for certain actions or states that satisfy the preconditions of other, particularly rewarding actions (e.g., increasing the rewards for cooking actions since they enable immediately rewarding eating actions; or increasing rewards simply for going to a place where there may be food, if experience indicates that this correlates with higher subsequent rewards). This would be a form of reinforcement learning, akin to the joy of anticipation (or dread) that humans (and other creatures, such as Pavlov's dog) can develop as a result of experience. Abstraction of macro-operators from particularly rewarding sequences of actions (much as in the tradition of (Fikes et al., 1972)) is another possibility.

Transition to a Real-World Dialogue Agent

Dialogue states and world states can be unboundedly varied, and the contributions by an interlocutor are generally unpredictable. Thus dialogue planning needs to be hierarchical and allow for uncertainty. Even so, reasoning and planning could become prohibitively complex when they take into account not only a diversified domain of discourse, but also the discourse context, speech acts, and mental states of the participants, including mutual beliefs, values, goals and plans.

We expect that a partial escape from planning complexity will be enabled by the use of data-driven common-ground inferences, and rule-based suggestions on what to consider saying or doing next, or what goals to consider pursuing, given features of the current situation. Many such suggestions – e.g., to greet

under certain conditions, to answer questions when asked, to help with problems if able to, or to empathize with someone's delight or dismay – are self-evident from human experience and principles of cooperative conversation. Others are perhaps learnable from experience, based on the consequences (such as interlocutor (dis)approval, or new knowledge gained) of choices made in the course of dialogues. However, unlike a policy-driven agent, the opportunistic agent will consider whether the generated suggestions can be advantageously incorporated into the current plan, before actually committing to corresponding plan modifications. In this way it will integrate deliberate and impulsive behavior.

Despite the challenges presented by real-world dialogues, several of the representational and computational techniques we developed as part of the current framework could carry over to such dialogues. One is the use of a STRIPS-like precondition-effect structure in action schemas, perhaps along with start-conditions and stop-conditions as used in our "actual" operators, for modeling actions by other agents and exogenous events in the world. Other transferrable aspects are the procedural attachment conventions and the allowance for attitudinal predications in preconditions and effects. Crucially, we will carry forward the idea of associating intrinsic (or anticipatory) rewards and penalties with actions (verbal or otherwise) and states, as the ultimate basis for the agent's behavior. For example, rather than being compelled by dialogue rules or recipes to respond in fixed ways to particular types of inputs and dialogue situations, the dialogue agent would act cooperatively because it enjoys both speaking and listening, and is pleased upon answering a question to the interlocutor's apparent satisfaction, or upon receiving an item of knowledge that fills a perceived significant gap in its knowledge, and it would feel penalized if reproached by the interlocutor.

Another transferable feature is the method of reflective inference employed by the system and ascribed to other agents. The idea that agents know what they know and do not know is of course familiar, but that an agent x knows the truth

or falsity of broad classes of statements whose subject is x is less familiar. While this assumption cannot be made about all predicates (e.g., whether x hosts certain microorganisms, whether x has read more novels than her neighbor, whether x ate a sandwich on the same date 20 years ago, or will die within 20 years), it can be made about most attitudes (what x wants, likes, fears, hopes, expects, intends, doubts, etc.), about major abilities (what devices x can operate, what languages x can speak, what foods x can prepare, etc.), about ownership, friendship, acquaintanceship, major life events, and much more. Finally, the currently used form of bounded forward inference to enrich the representations of expected conditions in the world, as a result of both contemplated speech acts and potential actions and events in the world under discussion, will be an essential mechanism in the kind of dialogue system we envisage.

Bibliography

Albore, A., H. Palacios, and H. Geffner. 2009. A translation-based approach to contingent planning. In *Proceedings of the 21st international Joint Conference on Artifical intelligence (IJCAI 2009)*, pages 1623–1628.

Anderson, J.R., D. Bothell, M.D. Byrne, S. Douglass, C. Lebiere, and Y. Qin. 2004. An integrated theory of mind. *Psychological Review*, 111:1036–1060.

Anderson, M.L. and T. Oates. 2007. A review of recent research in reasoning and metareasoning. *AI Magazine*, 28(1):7–16.

Anderson, M.L. and D.R. Perlis. 2005. Logic, self-awareness and self-improvement: The metacognitive loop and the problem of brittleness. *Journal of Logic and Computation*, 15(1):21–40.

Aydemir, A., M. Göbelbecker, M. Hanheide, N. Hawes, I. Kruijff-Korbayová, P. Lison, K. Sjöo, J.L. Wyatt, H. Zender, and M. Zillich. 2010. Dora the explorer: A mobile robot motivated by curiosity. In *poster, 4th International Conference on Cognitive Systems (CogSys 2010)*. Zurich.

Baars, B. 1988. *A Cognitive Theory of Consciousness*. Cambridge University Press, New York, NY.

Bacchus, F. and R. Petrik. 1998. Modeling an agent's incomplete knowledge during planning and execution. In A.G. Cohn, L.K. Schubert, and S.C. Shapiro, editors,

Proceedings of the 6th International Conference on Principles of Knowledge Representation and Reasoning (KR 1998), pages 432–443. Morgan Kaufmann Publishers.

Baral, C., L. Floriano, A. Hardesty, D. Morales, M. Nogueira, and T. C. Son. 1998. From theory to practice: the utep robot in the aaai 96 and aaai 97 robot contests. In *Proc. of the 2nd Int. Conf. on Autonomous Agents (Agents 1998)*, pages 32–38. St. Paul,MN.

Baral, C. and T.C. Son. 1997. Approximate reasoning about actions in the presence of sensing and incomplete information. In *Proceedings of the International Logic Programming Symposium (ILPS 1997)*, pages 387–401.

Best, B.J. and C. Lebiere. 2006. Cognitive agents interacting in real and virtual worlds. In R. Sun, editor, *Cognition and Multi-Agent Interaction: From Cognitive Modeling to Social Simulation*, pages 186–218. Cambridge Univ. Press.

Blum, A. and M. Furst. 1995. Fast planning through planning graph analysis. In *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI 1995)*, pages 1636–1642.

Bonet, B. and H. Geffner. 2000. Planning with incomplete information as heuristic search in belief space. In *Proceedings of the 5th International Conference on Artificial Intelligence Planning Systems (AIPS 2000)*, pages 52–61.

Bratman, M.E. 1987. *Intentions, Plans, and Practical Reason*. Harvard University Press, Cambridge, MA.

Bratman, M.E. 1990. What is intention? In P.R. Cohen, J.L. Morgan, and M.E. Pollack, editors, *Intentions in Communication*, pages 15–32. MIT Press.

Breazeal, C. and R. Brooks. 2004. Robot emotion: A functional perspective. In J.M. Fellous and M.A. Arbib, editors, *Who Needs Emotions: The Brain Meets the Robot*, page Chapter 10. Oxford Univ. Press.

Breazeal, C., J. Gray, and M. Berlin. 2009. An embodied cognition approach to mindreading skills for socially intelligent robots. *The Int. J. of Robotics Research*, 28:565–680.

Brenner, M. and B. Nebel. 2009. Continual planning and acting in dynamic multi-agent environments. *Autonomous Agents and Multi-Agent Systems*, 19(3):297–331.

Bryce, D., S. Kambhampati, and D.E. Smith. 2006. Planning graph heuristics for belief space search. *Journal of Artificial Intelligence Research*, 26:35–99.

Burgard, W., A.B. Cremers, D. Fox, D. Hahnel, G. Lakemeyer, D. Schulz, W. Steiner, and S. Thrun. 1999. Experiences with an interactive museum tour-guide robot. *Artificial Intelligence*, 114(1-2):3–55.

Carbone, A., A. Finzi, A. Orlandini, F. Pirri, and G. Ugazio. 2005. Augmenting situation awareness via model-based control in rescue robots. In *Proc. of the IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS 2005)*. Edmonton, Canada.

Cohen, P.R. and H.J. Levesque. 1990a. Intention is choice with commitment. *Artificial Intelligence*, 42(2-3):213–261.

Cohen, P.R. and H.J. Levesque. 1990b. Rational interaction as the basis for communication. In P.R. Cohen, J.L. Morgan, and M.E. Pollack, editors, *Intentions in Communication*, pages 221–256. MIT Press.

Cox, M. and A. Raja. 2008. Metareasoning: A manifesto. In *AAAI-08 Workshop on Metareasoning*, pages 1–4.

Cox, M.T. 2005. Metacognition in computation: A selected research review. *Artificial Intelligence*, 169(2):104–141.

Cox, M.T. 2007. Perpetual self-aware cognitive agents. *Artificial Intelligence*, 28(1):32–45.

Davies, N.P. and Q. Mehdi. 2006. BDI for intelligent agents in computer games. In *Proceedings of the 8th International Conference on Computer Games: AI and Mobile Systems (CGAIMS 2006)*.

de Giacomo, G., L. Iocchi, D. Nardi, and R. Rosati. 1996. Moving a robot: the KR & R approach at work. In *Proc. of the 5th Int. Conf. on Principles of Knowledge Representation and Reasoning (KR 1996)*, pages 198–209. Cambridge, MA.

Dinerstein, J., P.K. Egbert, D. Ventura, and M. Goodrich. 2008. Demonstration-based behavior programming for embodied virtual agents. *Computational Intelligence*, 24(4):235–256.

Ferguson, G. and J.F. Allen. 1998. TRIPS: An integrated intelligent problem-solving assistant. In *Proceedings of the 15th National Conference on Artificial Intelligence (AAAI 1998)*.

Ferguson, G., J. Quinn, C. Horwitz, M. Swift, J. Allen, and L. Galescu. 2010. Towards a personal health management assistant. *J. of Biomedical Informatics*, 43(5):S13–S16.

Ferrein, E., C. Fritz, and G. Lakemeyer. 2004. On-line decision-theoretic Golog for unpredictable domains. In *Proceedings of the 27th German Conference on Artificial Intelligence (KI 2004)*.

Fikes, R.E., P.E. Hart, and N.J. Nilsson. 1972. Learning and executing generalized robot plans. *Artificial Intelligence*, 3(4):251–288.

Fikes, R.E. and N.J. Nilsson. 1971. STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2:189–208.

Fox, M. and D. Long. 2003. PDDL 2.1: An extension to PDDL for expressing temporal planning domains. *Journal of Artificial Intelligence Research*, 20:61–124.

Fritz, C. and S.A. McIlraith. 2008. Planning in the face of frequent exogenous events. In *Proc. of the 1st International Symposium on Search Techniques in Artificial Intelligence and Robotics (at AAAI 2008)*. Chicago, IL.

Georgeff, M.P. and A.L. Lansky. 1986. A system for reasoning in dynamic domains: Fault diagnosis on the space shuttle. Technical Report 375, AI Center, SRI International, 333 Ravenswood Ave., Menlo Park, CA 94025.

Georgeff, M.P. and A.L. Lansky. 1987. Reactive reasoning and planning. In *Proceedings of the 6th National Conference on Artificial Intelligence*.

Georgeff, M.P., A.L. Lansky, and P. Bessiere. 1985. Fast planning through planning graph analysis. In *Proceedings of the 9th International Joint Conference on Artificial Intelligence (IJCAI 1985)*, pages 516–523.

Georgeff, M.P., A.L. Lansky, and M.J. Schoppers. 1986. Reasoning and planning in dynamic domains: An experiment with a mobile robot. Technical Report 380, AI Center, SRI International, 333 Ravenswood Ave., Menlo Park, CA 94025.

Golden, K., O. Etzioni, and D. Weld. 1994. Tractable closed-world reasoning with updates. In *Proceedings of the 4th International Conference on Principles of Knowledge Representation and Reasoning*, pages 178–189.

Hayes-Roth, B. and F. Hayes-Roth. 1979. A cognitive model of planning. *Cognitive Science*, 3:275–310.

Hoffman, J. and R. Brafman. 2005. Contingent planning via heuristic forward search with implicit belief states. In *Proceedings of the 15th International Conference on Automated Planning and Scheduling (ICAPS 2005)*, pages 71–80.

Hoffmann, J. and B. Nebel. 2001. The FF planning system: Fast plan generation through heuristic search. *Journal of Artificial Intelligence Research*, 14:253–302.

Jung, H., J. Allen, L. Galescu, N. Chambers, M. Swift, and W. Taysom. 2008. Utilizing natural language for one-shot task learning. *J. of Logic and Computation*, 18(3):475–493.

Kaplan, A.N. 1999. Reason maintenance in a hybrid reasoning system. In *Proc. of the 1st Workshop in Computational Semantics (ICoS-1)*.

Kautz, H.A. and B. Selman. 1992. Planning as satisfiability. In *Proceedings of the 10th European Conference on Artificial Intelligence (ECAI 1992)*.

Krüger, N., J. Piater, C. Geib, R. Petrick, M. Steedman, F. Wörgötter, A. Ude, T. Asfour, D. Kraft, D. Omrcen, A. Agostini, and R. Dillmann. 2011. Object-action complexes: Grounded abstractions of sensory-motor processes. *Robotics and Autonomous Systems*, 59(10):740–757.

Kushmerick, N., S. Hanks, and D.S. Weld. 1995. An algorithm for probabilistic planning. *Artificial Intelligence*, 76:239–286.

Laird, J.E., N. Derbinsky, and J. Voigt. 2011. Performance evaluation of declarative memory systems in soar. In *Proc. of the 20th Behavior Representation in Modeling & Simulation Conference*, pages 33–40. Sundance, UT.

Laird, J.E., A. Newell, and P.S. Rosenbloom. 1987. SOAR: an architecture for general intelligence. *Artificial Intelligence*, 33(1):1–64.

Langley, P. 2006. Cognitive architectures and general intelligent systems. *AI Magazine*, 27(2):33–44.

Lespérance, Y. and H. Levesque. 1995. Indexical knowledge and robot action: A logical account. *Artificial Intelligence*, 72(1-2):69–115.

Levesque, H. and G. Lakemeyer. 2008. Cognitive robotics. In F. van Harmelen, V. Lifschitz, and B. Porter, editors, *Handbook of Knowledge Representation*, page Chapter 24. Elsevier.

Levesque, H.J., P.R. Cohen, and J.H.T. Nunes. 1990. On acting together. In *Proceedings of the 8th National Conference on Artificial Intelligence (AAAI 1990)*, pages 94–99.

Liu, D.H. 2008. A survey of planning in intelligent agents: from externally motivated to internally motivated systems. Technical Report TR-2008-936, Department of Computer Science, University of Rochester.

Liu, D.H. and L.K. Schubert. 2009. Incorporating planning and reasoning into a self-motivated, communicative agent. In *Proceedings of the 2nd Conference on Artificial General Intelligence (AGI 2009)*.

Liu, D.H. and L.K. Schubert. 2010. Combining self-motivation with logical planning and inference in a reward-seeking agent. In *Proceedings of the 2nd International Conference on Agents and Artificial Intelligence (ICAART 2010)*, volume 2, pages 257–263.

McCarthy, J. 1968. Programs with common sense. In M.L. Minsky, editor, *Semantic Information Processing*, pages 403–418. MIT Press.

McCarthy, J. 1979. First order theories of individual concepts and propositions. In *Machine Intelligence 9*, pages 129–147.

McCarthy, J. 1995. Making robots conscious of their mental states. In *Machine Intelligence 15*, pages 3–17.

McDermott, D. 1998. *The planning domain definition language manual*.

Minsky, M. L. 1968. Matter, mind, and models. In M.L. Minsky, editor, *Semantic Information Processing*, pages 425–432. MIT Press.

Morbini, F. and L. Schubert. 2008. Metareasoning as an integral part of commonsense and autocognitive reasoning. In *AAAI-08 Workshop on Metareasoning*.

Newell, A. 1990. *Unified Theories of Cognition*. Harvard University Press, Cambridge, MA.

Nilsson, N.J. 1984. Shakey the robot. Technical Report 323, AI Center, SRI International.

Palacios, H. and H. Geffner. 2007. From conformant into classical planning: Efficient translations that may be complete too. In *Proceedings of the 17th International Conference on Automated Planning and Scheduling (ICAPS 2007)*, pages 264–271.

Penberthy, J.S. and D.S. Weld. 1992. UCPOP: A sound, complete, partial order planner for ADL. In B. Nebel, C. Rich, and W. Swartout, editors, *Proceedings of the 3rd International Conference on Principles of Knowledge Representation and Reasoning (KR 1992)*, pages 103–114. Morgan Kaufmann Publishers.

Petrick, R., N. Adermann, T. Asfour, M. Steedman, and R. Dillmann. 2010. Connecting knowledge-level planning and task execution on a humanoid robot using Object-Action Complexes. In *poster, 4th International Conference on Cognitive Systems (CogSys 2010)*. Zurich.

Rao, A.S. and M.P. Georgeff. 1991. Modeling rational agents within a BDI-architecture. In J. Allen, R. Fikes, and E. Sandewall, editors, *Proceedings of the 2nd International Conference on Principles of Knowledge Representation and Reasoning (KR 1991)*, pages 473–484. Morgan Kaufmann Publishers.

Rao, A.S. and M.P. Georgeff. 1992. An abstract architecture for rational agents. In C. Rich, W. Swartout, and B. Nebel, editors, *Proceedings of the 3rd International Conference on Principles of Knowledge Representation and Reasoning (KR 1992)*, pages 439–449. Morgan Kaufmann Publishers.

Rao, A.S. and M.P. Georgeff. 1995. BDI-agents: from theory to practice. In *Proceedings of the First International Conference on Multiagent Systems*.

Reiter, R., editor. 2001. *Knowledge in Action: Logical Foundations for Specifying and Implementing Dynamical Systems*. MIT Press, Cambridge, MA.

Sardiña, S., G. De Giacomo, Y. Lespérance, and H. Levesque. 2004. On ability to autonomously execute agent programs with sensing. In *Proc. of the 4th International Workshop on Cognitive Robotics (CoRobo 2004)*.

Scherl, R., H. Levesque, and Y. Lespérance. 1995. The situation calculus with sensing and indexical knowledge. In M. Koppel and E. Shamir, editors, *Proceedings of the 4th Bar-Ilan Symposium on Foundations of Artificial Intelligence (BISFAI 1995)*, pages 86–95.

Schubert, L. 2005. Some knowledge representation and reasoning requirements for self-awareness. In M. Anderson and T. Oates, editors, *Metacognition in Computation: Papers from the 2005 AAAI Spring Symposium*, pages 106–113. AAAI Press.

Shapiro, S.C. 1998. Embodied Cassie. In *Cognitive Robotics: Papers from the 1998 AAAI Fall Symposium*, pages 136–143. Menlo Park, CA.

Shapiro, S.C. and J.P. Bona. 2010. The GLAIR cognitive architecture. *International Journal of Machine Consciousness*, 2(2):307–332.

Tacke, M., T. Weigel, and B. Nebel. 2004. Decision-theoretic planning for playing table soccer. In *Proceedings of the 27th German Conference on Artificial Intelligence (KI 2004)*.

Torrance, M.C. 1994. *Natural Communication with Robots*. Master's thesis, Department of Electrical Engineering and Computer Science, MIT.

Vere, S. 1983. Planning in time: Windows and durations for activities and goals. *IEEE Tran. on Pattern Analysis and Machine Intelligence*, PAMI-5(3):246–267.

Vere, S. and T. Bickmore. 1990. A basic agent. *Computational Intelligence*, 6(1):41–60.

Vere, S.A. 1991. Organization of the basic agent. *SIGART Bulletin*, 2(4):164–168.

Weld, D.S. 1999. Recent advances in AI planning. *AI Magazine*, 20(2):93–123.

Winograd, T. 1971. Procedures as a representation for data in a computer program for understanding natural language. Technical Report 235, Massachusetts Institute of Technology.

Winograd, T. 1972. *Understanding Natural Language*. Academic Press.